electrical-engineering-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `electrical_engineering_ai999x.about()` for the same answer.

Every capability electrical-engineering-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import electrical_engineering_ai999x

result = electrical_engineering_ai999x.solve('series_resistance', R='10, 20, 30')
result.show()
# equivalent = 60 ohm, count = 3
```

By topic

topic	capabilities
AC circuits / AC power	Complex, real, reactive and apparent power from voltage and current phasors, Power-factor correction capacitor
AC circuits / energy storage	Energy stored in an inductor
AC circuits / filters	First-order RC filter: cutoff, gain and phase
AC circuits / impedance	Impedance of a series RLC circuit, Impedance of a parallel RLC circuit
AC circuits / phasors	Phasor from polar to rectangular form, Phasor from rectangular to polar form, Sum of sinusoids by phasor addition
AC circuits / resonance	Resonance, Q factor and bandwidth of a series RLC circuit, Resonance, Q factor and bandwidth of a parallel RLC circuit
AC circuits / three-phase power	Balanced three-phase load (wye or delta)
AC circuits / transformers	Ideal transformer: voltages, currents and reflected resistance
AC circuits / waveforms	RMS and average value of a periodic waveform
Control systems / compensation	Maximum phase lead of a lead compensator
Control systems / electromechanical systems	Steady-state speed and current of a permanent-magnet DC motor
Control systems / frequency response	Phase margin and gain-crossover frequency, Gain margin and phase-crossover frequency, Frequency response (Bode magnitude and phase) at one frequency
Control systems / stability	Routh-Hurwitz stability test, Closed-loop poles with unity negative feedback

topic	capabilities
Control systems / state space	Transfer function of a single-input single-output state-space model, Controllability and observability of a single-input single-output system
Control systems / steady-state accuracy	Steady-state error of a unity-feedback system
Control systems / time response	Step-response specifications of a standard second-order system
DC circuits / bridges	Wheatstone bridge output
DC circuits / dividers	Voltage divider, Current divider
DC circuits / mesh and nodal analysis	Mesh currents from the mesh equations (Kirchhoff's voltage law), Node voltages from the nodal equations (Kirchhoff's current law), Node voltages of a DC resistive circuit from its netlist
DC circuits / network transformations	Delta-to-wye (Pi-to-T) resistor transformation, Wye-to-delta (T-to-Pi) resistor transformation
DC circuits / series and parallel combinations	Equivalent resistance of resistors in series, Equivalent resistance of resistors in parallel, Equivalent capacitance of capacitors in series, Equivalent capacitance of capacitors in parallel, Equivalent inductance of inductors in series, Equivalent inductance of inductors in parallel, Equivalent resistance of a series-parallel resistor network
DC circuits / Thevenin and Norton equivalents	Thevenin equivalent of a DC circuit between two nodes, Norton equivalent of a DC circuit between two nodes, Maximum power transfer
Digital logic / Boolean algebra	Evaluate a Boolean expression, Equivalence of two Boolean expressions
Digital logic / data conversion	Analog-to-digital conversion: output code and resolution
Digital logic / minimization	Truth table and minimal sum of products of a Boolean expression, Minimal sum of products from minterms (with don't-cares)
Digital logic / number systems	Number base conversion (binary, octal, decimal, hexadecimal), Two's complement representation of a signed integer, Value of a two's complement bit pattern, Binary to Gray code and Gray code to binary
Diodes and transistors / BJT biasing	DC operating point of a voltage-divider biased BJT
Diodes and transistors / diodes	Operating point of a diode in series with a resistor (Shockley equation)
Diodes and transistors / MOSFET	MOSFET drain current in saturation (square law)
Diodes and transistors / Zener regulator	Zener diode shunt regulator
Operational amplifiers / basic stages	Inverting amplifier gain and output, Non-inverting amplifier gain and output
Operational amplifiers / difference amplifier	Difference amplifier output (four-resistor op-amp circuit)
Operational amplifiers / integrator	Op-amp integrator output for a constant input
Operational amplifiers / summing amplifier	Inverting summing amplifier output
Transients / first-order circuits	First-order (RC or RL) transient response, Time for a first-order response to reach a value

topic	capabilities
Transients / second-order circuits	Step response of a series RLC circuit

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
series_resistance	Equivalent resistance of resistors in series	R	equivalent, count
parallel_resistance	Equivalent resistance of resistors in parallel	R	equivalent, count
series_capacitance	Equivalent capacitance of capacitors in series	C, unit	equivalent, count, unit
parallel_capacitance	Equivalent capacitance of capacitors in parallel	C, unit	equivalent, count, unit
series_inductance	Equivalent inductance of inductors in series	L, unit	equivalent, count, unit
parallel_inductance	Equivalent inductance of inductors in parallel	L, unit	equivalent, count, unit
resistor_network	Equivalent resistance of a series-parallel resistor network	network	equivalent, resistors
mesh_currents	Mesh currents from the mesh equations (Kirchhoff's voltage law)	A, b	currents, first
nodal_voltages	Node voltages from the nodal equations (Kirchhoff's current law)	A, b	voltages, first
dc_node_voltages	Node voltages of a DC resistive circuit from its netlist	netlist	node_voltages, source_currents, power_delivered
thevenin_equivalent	Thevenin equivalent of a DC circuit between two nodes	netlist, node_a, node_b	Vth, Rth
norton_equivalent	Norton equivalent of a DC circuit between two nodes	netlist, node_a, node_b	In, Rn
delta_to_wye	Delta-to-wye (Pi-to-T) resistor transformation	Rab, Rbc, Rca	Ra, Rb, Rc
wye_to_delta	Wye-to-delta (T-to-Pi) resistor transformation	Ra, Rb, Rc	Rab, Rbc, Rca
routh_hurwitz	Routh-Hurwitz stability test	coefficients	rhp_roots, stable, first_column
closed_loop_poles	Closed-loop poles with unity negative feedback	num, den, K	poles, max_real_part, stable, characteristic
phase_margin	Phase margin and gain-crossover frequency	num, den	phase_margin, gain_crossover
gain_margin	Gain margin and phase-crossover frequency	num, den	gain_margin, gain_margin_db, phase_crossover
bode_point	Frequency response (Bode magnitude and phase) at one frequency	num, den, w	magnitude, magnitude_db, phase_deg, real, imag

id	operation	inputs	outputs
steady_state_error	Steady-state error of a unity-feedback system	num, den, input	steady_state_error, error_constant, system_type
second_order_step	Step-response specifications of a standard second-order system	zeta, wn	percent_overshoot, overshoot, peak_time, rise_time, t10, t90, settling_time, damped_frequency
series_rlc_impedance	Impedance of a series RLC circuit	R, L, C, f	impedance, phase_deg, real, imag, XL, XC
parallel_rlc_impedance	Impedance of a parallel RLC circuit	R, L, C, f	impedance, phase_deg, real, imag, XL, XC
polar_to_rectangular	Phasor from polar to rectangular form	magnitude, angle_deg	real, imag
rectangular_to_polar	Phasor from rectangular to polar form	real, imag	magnitude, angle_deg
phasor_sum	Sum of sinusoids by phasor addition	amplitudes, phases_deg	amplitude, phase_deg, real, imag
complex_power	Complex, real, reactive and apparent power from voltage and current phasors	V, V_phase, I, I_phase	P, Q, S, power_factor, nature
power_factor_correction	Power-factor correction capacitor	P, V, f, pf_initial, pf_target	C, C_uF, Qc, Q_initial, Q_final
three_phase_load	Balanced three-phase load (wye or delta)	VL, R, X, connection	phase_voltage, phase_current, line_current, P, Q, S, power_factor
summing_amplifier	Inverting summing amplifier output	V, R, Rf	Vout, gains
difference_amplifier	Difference amplifier output (four-resistor op-amp circuit)	V1, V2, R1, R2, R3, R4	Vout, gain_inverting, gain_noninverting
bjt_divider_bias	DC operating point of a voltage-divider biased BJT	VCC, R1, R2, RC, RE, beta, VBE	IC, IC_mA, IB, IE, VB, VE, VC, VCE
zener_regulator	Zener diode shunt regulator	Vin, R, Vz, RL	VL, IR, IL, IZ, IZ_mA, PZ
series_rlc_step	Step response of a series RLC circuit	R, L, C, V, t	vC, i, i_mA, alpha, omega0, zeta, response
boolean_evaluate	Evaluate a Boolean expression	expression, values	value
truth_table_sop	Truth table and minimal sum of products of a Boolean expression	expression	variables, minterms, maxterms, canonical_sop, minimal_sop, terms, literals
minterms_to_sop	Minimal sum of products from minterms (with don't-cares)	minterms, variables, dont_cares	minimal_sop, terms, literals, variables
boolean_equivalence	Equivalence of two Boolean expressions	expression1, expression2	equivalent, counterexample
base_conversion	Number base conversion (binary, octal, decimal, hexadecimal)	value, base	decimal, binary, octal, hexadecimal
twos_complement_encode	Two's complement representation of a signed integer	value, bits	pattern, hexadecimal, unsigned

id	operation	inputs	outputs
twos_complement_decode	Value of a two's complement bit pattern	pattern	value, bits
gray_code	Binary to Gray code and Gray code to binary	value, direction	result, decimal
voltage_divider	Voltage divider	Vin, R1, R2	Vout, I
current_divider	Current divider	I, R1, R2	I1, I2
max_power_transfer	Maximum power transfer	Vth, Rth	RL, Pmax, efficiency
wheatstone_bridge	Wheatstone bridge output	Vs, R1, R2, R3, R4	Vout, balanced, R4_balance
rc_filter	First-order RC filter: cutoff, gain and phase	R, C, f, filter	fc, gain, gain_db, phase_deg
series_rlc_resonance	Resonance, Q factor and bandwidth of a series RLC circuit	R, L, C	f0, Q, bandwidth, f_low, f_high
parallel_rlc_resonance	Resonance, Q factor and bandwidth of a parallel RLC circuit	R, L, C	f0, Q, bandwidth, f_low, f_high
first_order_transient	First-order (RC or RL) transient response	V0, Vf, tau, t	v, fraction_complete
inverting_amplifier	Inverting amplifier gain and output	Rin, Rf, Vin	gain, Vout
noninverting_amplifier	Non-inverting amplifier gain and output	Rg, Rf, Vin	gain, Vout
opamp_integrator	Op-amp integrator output for a constant input	R, C, Vin, t, V0	Vout, slope
transformer	Ideal transformer: voltages, currents and reflected resistance	Np, Ns, Vp, RL	Vs, Is, Ip, Rin, turns_ratio
mosfet_saturation	MOSFET drain current in saturation (square law)	kn, WL, VGS, VTH, VDS	ID, ID_mA, VOV, gm
diode_circuit	Operating point of a diode in series with a resistor (Shockley equation)	Vs, R, Is, n, VT	I, I_mA, VD, VR
rms_waveform	RMS and average value of a periodic waveform	shape, amplitude, offset	rms, average, ac_rms, crest_factor
adc_conversion	Analog-to-digital conversion: output code and resolution	Vin, Vref, bits	code, binary, lsb, lsb_mV, quantized
state_space_to_tf	Transfer function of a single-input single-output state-space model	A, B, C, D	num, den, order
controllability_observability	Controllability and observability of a single-input single-output system	A, B, C	controllability_rank, observability_rank, controllable, observable
inductor_energy	Energy stored in an inductor	L, I	energy, flux_linkage
first_order_time_to_value	Time for a first-order response to reach a value	V0, Vf, tau, v	t, time_constants
lead_compensator	Maximum phase lead of a lead compensator	z, p	phase_max_deg, w_max, gain_at_w_max_db

id	operation	inputs	outputs
dc_motor	Steady-state speed and current of a permanent-magnet DC motor	V, R, Kt, Ke, T, b	speed, rpm, current, input_power, output_power, efficiency

Equivalent resistance of resistors in series — `series_resistance`

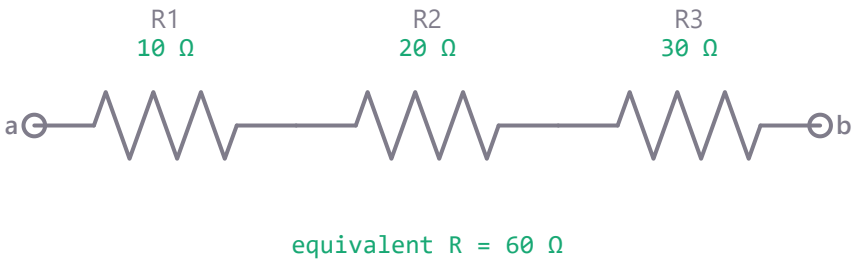
The single resistance equivalent to two or more resistors connected in series.

$$R_{eq} = \sum_{i} R_i$$

input	meaning	condition
`R`	the resistance values in ohms (a list, e.g. '10, 20, 30')	

output	meaning
`equivalent`	equivalent resistance
`count`	number of resistors

- **Method:** the sum of the values
- **Assumes:** every value positive; at least two values
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The 3 elements of the worked example connected in series: R1 = 10 Ω, R2 = 20 Ω, R3 = 30 Ω. Labelled with the equivalent the package returned: 60 Ω. Marked: equivalent = 60 Ω.

```
electrical_engineering_ai999x.solve('series_resistance', R='10, 20, 30')
# -> equivalent = 60 ohm, count = 3   (SOLVED)
```

```
electrical_engineering_ai999x.solve('series_resistance', R=[1000, 2200])
# -> equivalent = 3200 ohm, count = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the equivalent resistance of 10 ohm, 20 ohm and 30 ohm resistors in series.	SOLVED	equivalent = 60 ohm, count = 3

question	outcome	result
What is the total resistance of 1 kohm and 2.2 kohm in series?	SOLVED	equivalent = 3200 ohm, count = 2
Find the equivalent resistance of 10, 0 and 5 ohm resistors in series.	INVALID_INPUT	Equivalent resistance of resistors in series: every resistor must be a positive finite number.
Find the equivalent resistance of 3 resistors of 10 ohm in series.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electrical_engineering_ai999x.tem

Equivalent resistance of resistors in parallel — `parallel_resistance`

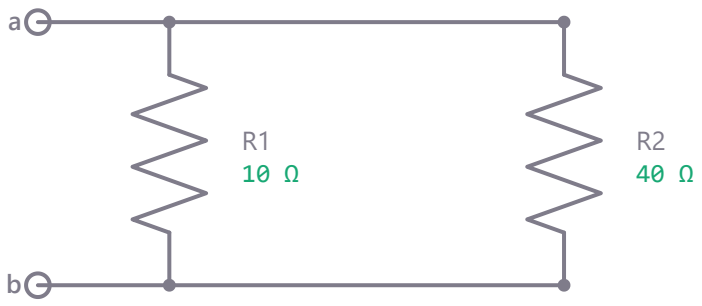
The single resistance equivalent to two or more resistors connected in parallel.

$$\frac{1}{R_{eq}} = \sum_{i} \frac{1}{R_i}$$

input	meaning	condition
`R`	the resistance values in ohms (a list, e.g. '10, 20, 30')	

output	meaning
`equivalent`	equivalent resistance
`count`	number of resistors

- **Method:** the reciprocal of the sum of reciprocals
- **Assumes:** every value positive; at least two values
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



`equivalent R = 8 Ω`

The 2 elements of the worked example connected in parallel: R1 = 10 Ω, R2 = 40 Ω. Labelled with the equivalent the package returned: 8 Ω. Marked: equivalent = 8 Ω.

```
electrical_engineering_ai999x.solve('parallel_resistance', R='10, 40')
# -> equivalent = 8 ohm, count = 2   (SOLVED)
```

```
electrical_engineering_ai999x.solve('parallel_resistance', R=[30, 60, 20])
# -> equivalent = 10 ohm, count = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the equivalent resistance of 10 ohm and 40 ohm resistors in parallel.	SOLVED	equivalent = 8 ohm, count = 2
What is the total resistance of 30, 60 and 20 ohm in parallel?	SOLVED	equivalent = 10 ohm, count = 3
What is the total resistance of 1 kohm and 1 kohm in parallel?	SOLVED	equivalent = 500 ohm, count = 2
Find the equivalent resistance of 10 ohm and -40 ohm resistors in parallel.	INVALID_INPUT	Equivalent resistance of resistors in parallel: every resistor must be a positive finite number.

Equivalent capacitance of capacitors in series — `series_capacitance`

The single capacitance equivalent to two or more capacitors connected in series.

$$\frac{1}{C_{eq}} = \sum_{i} \frac{1}{C_i}$$

input	meaning	condition
`C`	the capacitance values, all in one unit (F, mF, uF, nF ...; a list, e.g. '10, 22, 47')	
`unit`	the unit the values are written in (the result is in the same unit)	

output	meaning
`equivalent`	equivalent capacitance, in the unit of the inputs
`count`	number of capacitors
`unit`	the unit of the result

- **Method:** the reciprocal of the sum of reciprocals
- **Assumes:** every value positive; at least two values
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The 2 elements of the worked example connected in series: C1 = 3 uF, C2 = 6 uF. Labelled with the equivalent the package returned: 2 uF. Marked: equivalent = 2 uF.

```
electrical_engineering_ai999x.solve('series_capacitance', C='3, 6', unit='uF')
# -> equivalent = 2, count = 2, unit = uF   (SOLVED)
```

```
electrical_engineering_ai999x.solve('series_capacitance', C=[10, 10, 10, 10])
# -> equivalent = 2.5, count = 4, unit = the unit of the inputs   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the equivalent capacitance of 3 uF and 6 uF capacitors in series.	SOLVED	equivalent = 2, count = 2, unit = uF
What is the total capacitance of 10 nF, 10 nF and 5 nF in series?	SOLVED	equivalent = 2.5, count = 3, unit = nF
Find the equivalent capacitance of 100 nF and 0.1 uF capacitors in series.	SOLVED	equivalent = 0.05, count = 2, unit = uF
Find the equivalent capacitance of 3 uF and 0 uF capacitors in series.	INVALID_INPUT	Equivalent capacitance of capacitors in series: every capacitor must be a positive finite number.

Equivalent capacitance of capacitors in parallel — `parallel_capacitance`

The single capacitance equivalent to two or more capacitors connected in parallel.

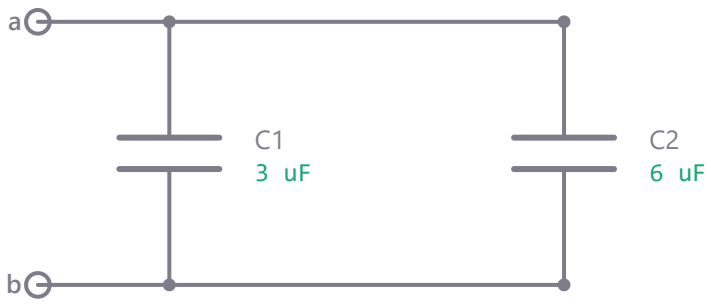
$$C_{eq} = \sum_i C_i$$

input	meaning	condition
`C`	the capacitance values, all in one unit (F, mF, uF, nF ...; a list, e.g. '10, 22, 47')	
`unit`	the unit the values are written in (the result is in the same unit)	

output	meaning
`equivalent`	equivalent capacitance, in the unit of the inputs
`count`	number of capacitors

output	meaning
`unit`	the unit of the result

- **Method:** the sum of the values
- **Assumes:** every value positive; at least two values
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The 2 elements of the worked example connected in parallel: C1 = 3 uF, C2 = 6 uF. Labelled with the equivalent the package returned: 9 uF. Marked: equivalent = 9 uF.

```
electrical_engineering_ai999x.solve('parallel_capacitance', C='3, 6', unit='uF')
# -> equivalent = 9, count = 2, unit = uF   (SOLVED)
```

```
electrical_engineering_ai999x.solve('parallel_capacitance', C=[1, 2.2])
# -> equivalent = 3.2, count = 2, unit = the unit of the inputs   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the equivalent capacitance of 3 uF and 6 uF capacitors in parallel.	SOLVED	equivalent = 9, count = 2, unit = uF
What is the total capacitance of 100 pF, 220 pF and 470 pF in parallel?	SOLVED	equivalent = 790, count = 3, unit = pF

Equivalent inductance of inductors in series — `series_inductance`

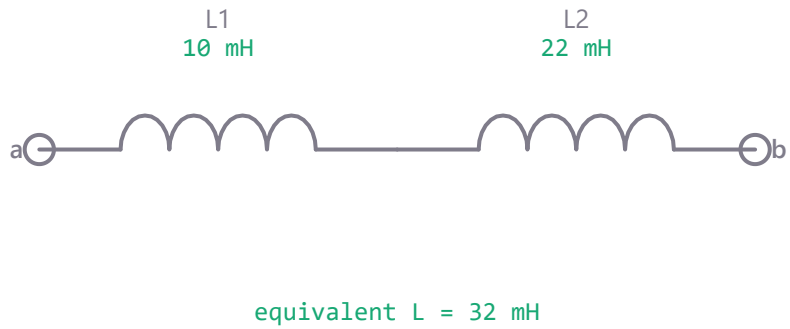
The single inductance equivalent to two or more inductors connected in series.

$$L_{eq} = \sum_{i} L_i$$

input	meaning	condition
`L`	the inductance values, all in one unit (H, mH, uH, nH ...; a list, e.g. '10, 22, 47')	
`unit`	the unit the values are written in (the result is in the same unit)	

output	meaning
`equivalent`	equivalent inductance, in the unit of the inputs
`count`	number of inductors
`unit`	the unit of the result

- **Method:** the sum of the values
- **Assumes:** every value positive; at least two values
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The 2 elements of the worked example connected in series: L1 = 10 mH, L2 = 22 mH. Labelled with the equivalent the package returned: 32 mH. Marked: equivalent = 32 mH.

```
electrical_engineering_ai999x.solve('series_inductance', L='10, 22', unit='mH')
# -> equivalent = 32, count = 2, unit = mH   (SOLVED)
```

```
electrical_engineering_ai999x.solve('series_inductance', L=[1, 2, 3])
# -> equivalent = 6, count = 3, unit = the unit of the inputs   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the equivalent inductance of 10 mH and 22 mH inductors in series.	SOLVED	equivalent = 32, count = 2, unit = mH
What is the total inductance of 1 mH, 2 mH and 3 mH in series?	SOLVED	equivalent = 6, count = 3, unit = mH

Equivalent inductance of inductors in parallel — `parallel_inductance`

The single inductance equivalent to two or more inductors connected in parallel.

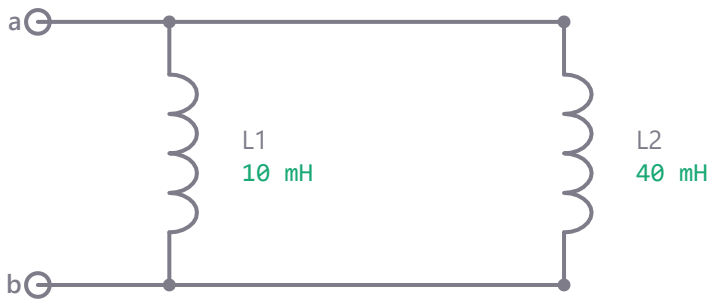
$$\frac{1}{L_{eq}} = \sum_i \frac{1}{L_i}$$

input	meaning	condition
`L`	the inductance values, all in one unit (H, mH, uH, nH ...; a list, e.g. '10, 22, 47')	

input	meaning	condition
`unit`	the unit the values are written in (the result is in the same unit)	

output	meaning
`equivalent`	equivalent inductance, in the unit of the inputs
`count`	number of inductors
`unit`	the unit of the result

- **Method:** the reciprocal of the sum of reciprocals
- **Assumes:** every value positive; at least two values
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The 2 elements of the worked example connected in parallel: L1 = 10 mH, L2 = 40 mH. Labelled with the equivalent the package returned: 8 mH. Marked: equivalent = 8 mH.

```
electrical_engineering_ai999x.solve('parallel_inductance', L='10, 40', unit='mH')
# -> equivalent = 8, count = 2, unit = mH   (SOLVED)
```

```
electrical_engineering_ai999x.solve('parallel_inductance', L=[6, 3])
# -> equivalent = 2, count = 2, unit = the unit of the inputs   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the equivalent inductance of 10 mH and 40 mH inductors in parallel.	SOLVED	equivalent = 8, count = 2, unit = mH
What is the total inductance of 6 uH and 3 uH in parallel?	SOLVED	equivalent = 2, count = 2, unit = uH

Equivalent resistance of a series-parallel resistor network — `resistor_network`

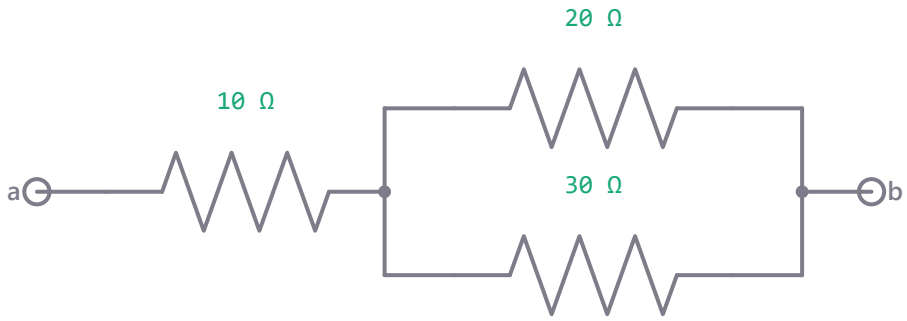
The equivalent resistance of a network written with + for series and || for parallel, e.g. 10 + (20 || 30).

$$R_s = R_1 + R_2, \quad R_p = \left(\frac{1}{R_1} + \frac{1}{R_2}\right)^{-1}$$

input	meaning	condition
`network`	the network in ohms: + joins in series, || joins in parallel ('||' binds tighter than '+'), parentheses group	

output	meaning
`equivalent`	equivalent resistance
`resistors`	number of resistors in the network

- **Method:** recursive reduction of the expression tree: series groups add, parallel groups add reciprocally
- **Assumes:** every resistance positive; at least two resistors; '||' binds tighter than '+'
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The resistor network written in the worked example, 10 + (20 || 30) (+ series, || parallel, resistances in ohms), drawn between terminals a and b. Labelled with what the package returned: equivalent resistance 22 Ω over 3 resistors. Marked: R_eq = 22 Ω.

```
electrical_engineering_ai999x.solve('resistor_network', network='10 + (20 || 30)')
# -> equivalent = 22 ohm, resistors = 3   (SOLVED)
```

```
electrical_engineering_ai999x.solve('resistor_network', network='(100 || 100) + 50')
# -> equivalent = 100 ohm, resistors = 3   (SOLVED)
```

```
electrical_engineering_ai999x.solve('resistor_network', network='6 || 3 + 4')
# -> equivalent = 6 ohm, resistors = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the equivalent resistance of 10 + (20 || 30) ohm.	SOLVED	equivalent = 22 ohm, resistors = 3
What is the total resistance of (100 || 100) + 50 ohms?	SOLVED	equivalent = 100 ohm, resistors = 3

question	outcome	result
Find the equivalent resistance of 4 || (6 + 6) ohm.	SOLVED	equivalent = 3 ohm, resistors = 3
Find the equivalent resistance of 10 + (20 || 0) ohm.	INVALID_INPUT	Equivalent resistance of a series-parallel resistor network: every resistance must be positive.
Find the equivalent resistance of 10 + (20 || 30 ohm.	INVALID_INPUT	Equivalent resistance of a series-parallel resistor network: the network expression ends too early (unbalanced parenthes

Mesh currents from the mesh equations (Kirchhoff's voltage law) — `mesh_currents`

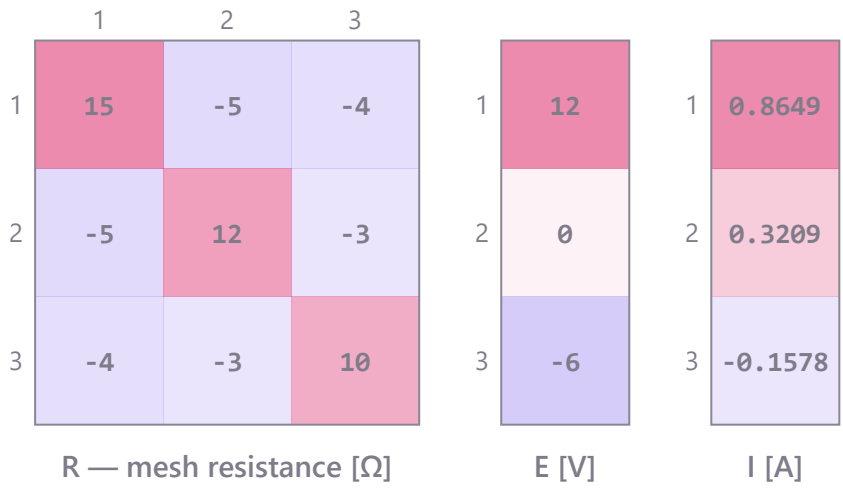
Solve the mesh equations R $\boldsymbol{i} = \boldsymbol{v}$ of a circuit: the resistance matrix and the source-voltage vector give every mesh current.

$$\mathbf{R}\,\mathbf{i} = \mathbf{v}$$

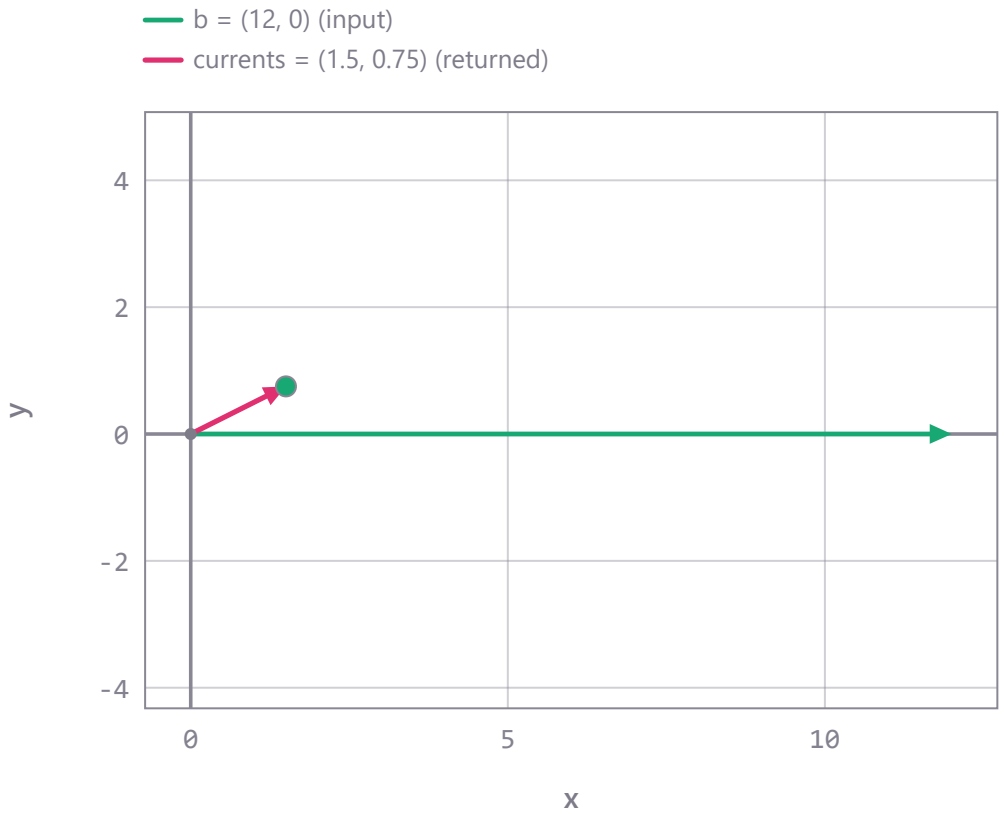
input	meaning	condition
`A`	mesh resistance matrix in ohms (self-resistances on the diagonal, minus the shared resistances off it)	
`b`	net source voltage around each mesh in volts, in the mesh-current direction	

output	meaning
`currents`	the mesh currents in amperes, in the order of the rows
`first`	the first mesh current

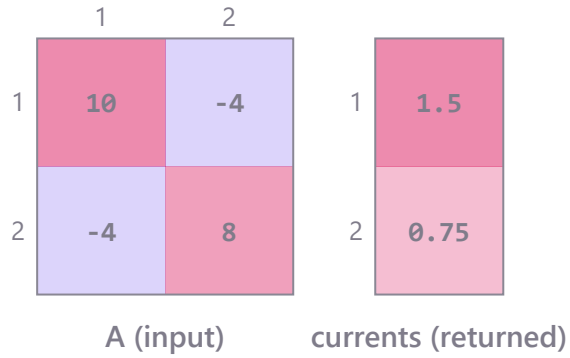
- **Method:** Gaussian elimination with partial pivoting
- **Assumes:** square matrix; one source voltage per mesh; a non-singular matrix
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Mesh analysis of a three-loop circuit as matrices: the mesh-resistance matrix R (self resistances on the diagonal, shared resistances negative off it) and the loop EMFs E are the stated circuit (R I = E); the mesh currents I are the solution returned by the package: I1 = 0.864914 A, I2 = 0.320943 A, I3 = -0.157752 A (a negative current flows against the assumed direction). Each panel is shaded by its own largest magnitude; negative values in the second colour.



The plane vectors of the worked example as arrows from the origin: input b = (12, 0); returned by electrical-engineering-ai999x: currents = (1.5, 0.75); also returned first = 1.5. Other inputs: A = [[10, -4], [-4, 8]]. Marked: the tip of each returned vector.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); currents (returned). Returned with them by electrical-engineering-ai999x: first = 1.5 A. Other inputs: b = [12, 0].

```
electrical_engineering_ai999x.solve('mesh_currents', A='[[10, -4], [-4, 8]]', b='[12, 0]')
# -> currents = [1.5, 0.7500000000000001], first = 1.5 A   (SOLVED)
```

```
electrical_engineering_ai999x.solve('mesh_currents', A='4 -2; -2 6', b='10 0')
# -> currents = [3.0, 1.0], first = 3 A   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve the mesh equations with resistance matrix [[10, -4], [-4, 8]] and source vector	SOLVED	currents = [1.5, 0.7500000000000001], first = 1.5 A

question	outcome	result
[12, 0].		
Find the mesh currents for resistance matrix [[4, -2], [-2, 6]] and voltage vector [10, 0].	SOLVED	currents = [3.0, 1.0], first = 3 A
Solve the mesh equations with resistance matrix [[1, 2], [2, 4]] and source vector [1, 1].	INVALID_INPUT	Mesh currents from the mesh equations (Kirchhoff's voltage law): the system is singular: the equations do not determine
Solve the mesh equations with resistance matrix [[10, -4], [-4, 8]].	NEEDS_INPUT	mesh_currents needs b (net source voltage around each mesh in volts, in the mesh-current direction). Example: solve('mes

Node voltages from the nodal equations (Kirchhoff's current law) — `nodal_voltages`

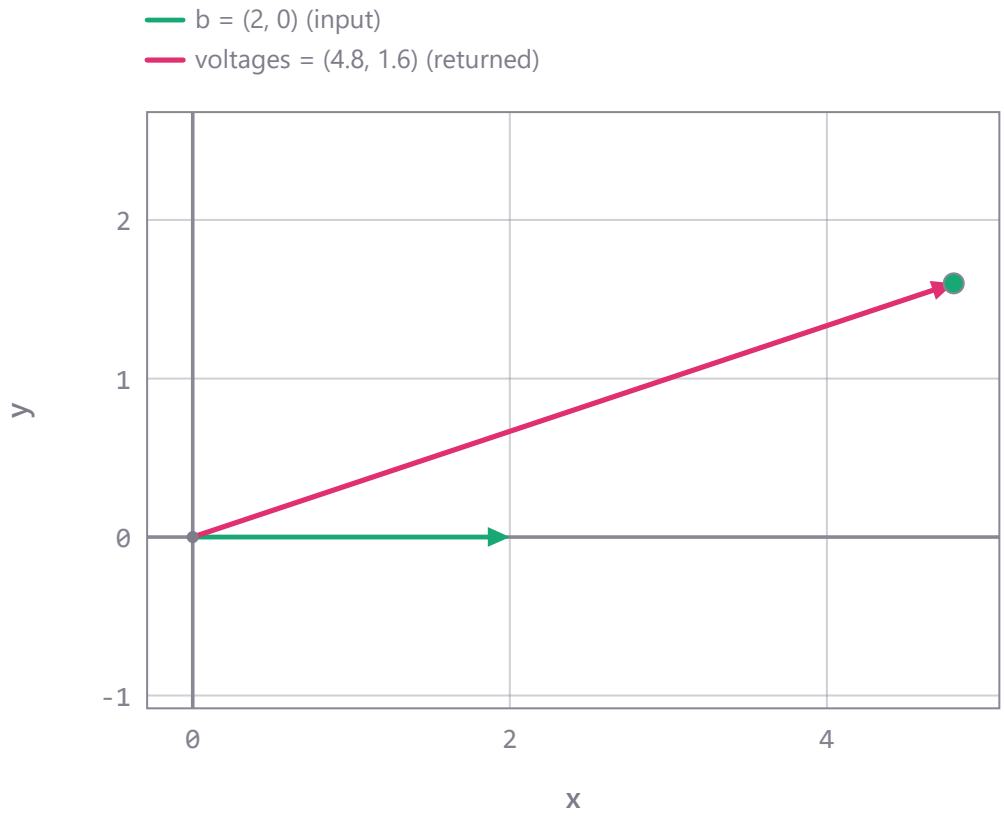
Solve the nodal equations G $\boldsymbol{v} = \boldsymbol{i}$ of a circuit: the conductance matrix and the injected-current vector give every node voltage.

$$\mathbf{G}\,\mathbf{v} = \mathbf{i}$$

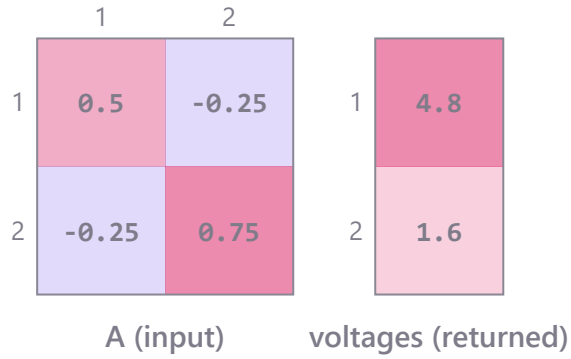
input	meaning	condition
`A`	nodal conductance matrix in siemens (sums of conductances on the diagonal, minus the shared conductances off it)	
`b`	current injected into each node by sources, in amperes	

output	meaning
`voltages`	the node voltages in volts, in the order of the rows
`first`	the first node voltage

- **Method:** Gaussian elimination with partial pivoting
- **Assumes:** square matrix; one injected current per node; a non-singular matrix
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input b = (2, 0); returned by electrical-engineering-ai999x: voltages = (4.8, 1.6); also returned first = 4.8. Other inputs: A = [[0.5, -0.25], [-0.25, 0.75]]. Marked: the tip of each returned vector.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); voltages (returned). Returned with them by electrical-engineering-ai999x: first = 4.8 V. Other inputs: b = [2, 0].

```
electrical_engineering_ai999x.solve('nodal_voltages', A='[[0.5, -0.25], [-0.25, 0.75]]', b='[2,
0]')
# -> voltages = [4.8, 1.6], first = 4.8 V   (SOLVED)
```

```
electrical_engineering_ai999x.solve('nodal_voltages', A=[[0.3, -0.1], [-0.1, 0.2]], b=[1, 0.5])
# -> voltages = [5.0, 5.0], first = 5 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve the nodal equations with conductance matrix [[0.5, -0.25], [-0.25, 0.75]] and current vector [2, 0].	SOLVED	voltages = [4.8, 1.6], first = 4.8 V
Find the node voltages for conductance matrix [[0.3, -0.1], [-0.1, 0.2]] and current vector [1, 0.5].	SOLVED	voltages = [5.0, 5.0], first = 5 V
Solve the nodal equations with conductance matrix [[0.5, -0.25], [-0.25, 0.75]] and current vector [2, 0, 1].	INVALID_INPUT	Node voltages from the nodal equations (Kirchhoff's current law): the vector needs 2 entries, one per row of the matrix.

Node voltages of a DC resistive circuit from its netlist — `dc_node_voltages`

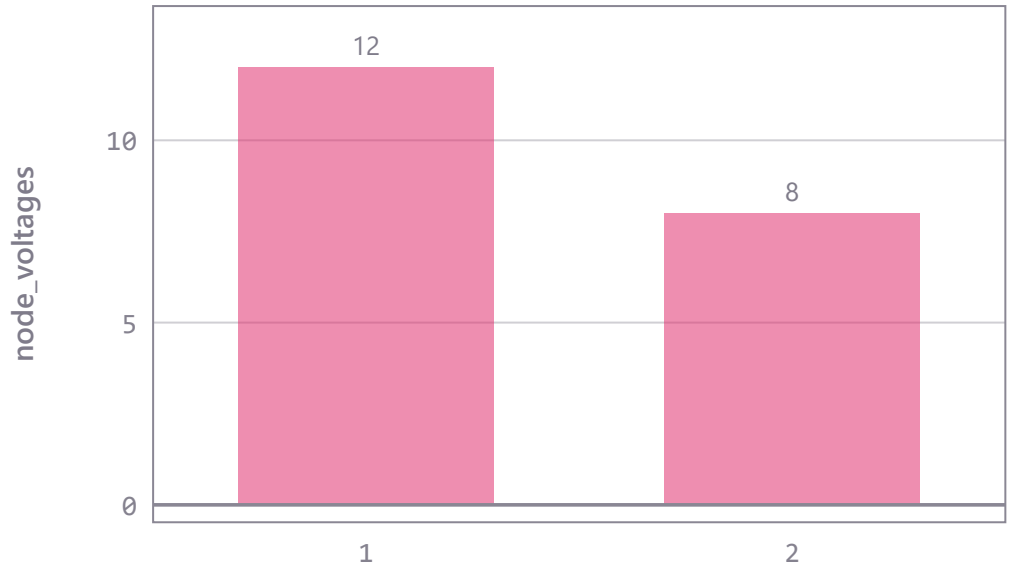
Every node voltage, every voltage-source current and the power delivered, for a circuit of resistors and independent sources written as a netlist.

$$\begin{bmatrix} G & B \\ B^T & 0 \end{bmatrix} \begin{bmatrix} v \\ i \end{bmatrix} = \begin{bmatrix} i_s \\ e \end{bmatrix}$$

input	meaning	condition
`netlist`	elements separated by ';': R<name> <node> <node> <ohms>, V<name> <node+> <node-> <volts>, I<name> <node> <node> <amperes> (flowing from the first node through the source to the second); node 0 is ground	

output	meaning
`node_voltages`	voltage of every node relative to node 0, in volts
`source_currents`	current through each voltage source from its + node to its - node, in amperes
`power_delivered`	total power delivered by the sources (= power dissipated in the resistors)

- **Method:** modified nodal analysis solved by Gaussian elimination
- **Assumes:** a ground node 0; every resistance positive; a unique solution (no floating node, no loop of voltage sources)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



voltage of every node relative to node 0, in volts (node_voltages) returned by electrical-engineering-ai999x for the worked example (netlist = V1 1 0 12; R1 1 2 1000; R2 2 0 2000), one bar per entry: 1 = 12, 2 = 8.

```
electrical_engineering_ai999x.solve('dc_node_voltages', netlist='V1 1 0 12; R1 1 2 1000; R2 2 0 2000')
# -> node_voltages = {'1': 12.0, '2': 8.0}, source_currents = {'V1': -0.004}, power_delivered = 0.048 W   (SOLVED)
```

```
electrical_engineering_ai999x.solve('dc_node_voltages', netlist='I1 0 1 0.01; R1 1 0 100; R2 1 2 200; R3 2 0 300')
# -> node_voltages = {'1': 0.8333333333333334, '2': 0.5000000000000001}, source_currents = {}, power_delivered = 0.00833333 W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the node voltages of the circuit: V1 1 0 12; R1 1 2 1000; R2 2 0 2000.	SOLVED	node_voltages = {'1': 12.0, '2': 8.0}, source_currents = {'V1': -0.004}, power_delivered = 0.048 W
Use nodal analysis on the circuit: I1 0 1 0.01; R1 1 0 100; R2 1 2 200; R3 2 0 300.	SOLVED	node_voltages = {'1': 0.8333333333333334, '2': 0.5000000000000001}, source_currents = {}, power_delivered = 0.00833333 W
Find the node voltages of the circuit: V1 1 0 12; R1 1 2 1000; R2 3 4 2000.	INVALID_INPUT	Node voltages of a DC resistive circuit from its netlist: the system is singular: the equations do not determine every u
Find the node voltages of the circuit: V1 1 0 12; R1 1 2 -5; R2 2 0 2000.	INVALID_INPUT	Node voltages of a DC resistive circuit from its netlist: resistor 'R1' must have a positive resistance.

Thevenin equivalent of a DC circuit between two nodes — `thevenin_equivalent`

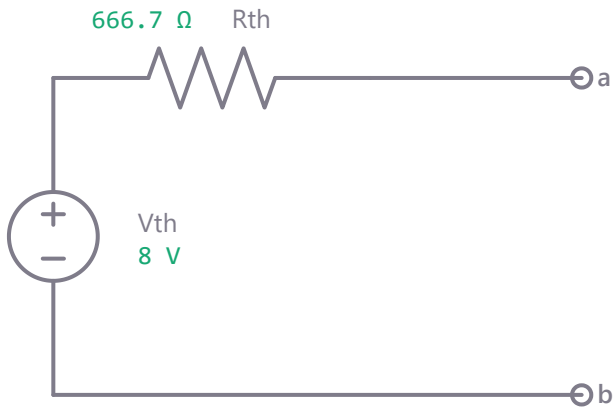
The Thevenin voltage and resistance seen between two nodes of a resistive circuit with independent sources.

$$V_{th} = v_a - v_b\big|_{open}, \quad R_{th} = \frac{v_a - v_b}{1\,\mathrm{A}}\Big|_{sources\ off}$$

input	meaning	condition
`netlist`	elements separated by ';': R<name> <node> <node> <ohms>, V<name> <node+> <node-> <volts>, I<name> <node> <node> <amperes>; node 0 is ground	
`node_a`	the terminal node (the + side of the equivalent)	
`node_b`	the reference terminal node	

output	meaning
`Vth`	Thevenin (open-circuit) voltage, node a relative to node b
`Rth`	Thevenin resistance

- **Method:** open-circuit voltage by modified nodal analysis; resistance by a 1 A test current with the independent sources set to zero
- **Assumes:** both nodes in the circuit; no ideal voltage source directly across the terminals; a unique solution (no floating node)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The Thévenin equivalent the package returned for the circuit V1 1 0 12; R1 1 2 1000; R2 2 0 2000 seen between nodes 2 (a) and 0 (b): a source Vth = 8 V in series with Rth = 666.667 Ω. Marked: Vth = 8 V, Rth = 666.667 Ω.

```
electrical_engineering_ai999x.solve('thevenin_equivalent', netlist='V1 1 0 12; R1 1 2 1000; R2 2 0
2000', node_a='2', node_b='0')
# -> Vth = 8 V, Rth = 666.667 ohm   (SOLVED)
```

```
electrical_engineering_ai999x.solve('thevenin_equivalent', netlist='I1 0 1 0.01; R1 1 0 100; R2 1 2
200', node_a='2', node_b='0')
# -> Vth = 1 V, Rth = 300 ohm   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Thevenin equivalent between nodes 2 and 0 of the circuit: V1 1 0 12; R1 1 2 1000; R2 2 0 2000.	SOLVED	Vth = 8 V, Rth = 666.667 ohm
What is the Thevenin equivalent between nodes 3 and 0 of the circuit: V1 1 0 10; R1 1 2 100; R2 2 0	SOLVED	Vth = 3.33333 V, Rth = 66.6667 ohm

question	outcome	result
100; R3 2 3 50; R4 3 0 200.		
Find the Thevenin equivalent between nodes 1 and 0 of the circuit: V1 1 0 12; R1 1 2 1000; R2 2 0 2000.	INVALID_INPUT	Thevenin equivalent of a DC circuit between two nodes: an ideal voltage source is across the terminals: the Thevenin res
Find the Thevenin equivalent of the circuit: V1 1 0 12; R1 1 2 1000; R2 2 0 2000.	NEEDS_INPUT	thevenin_equivalent needs node_a, node_b (the terminal node (the + side of the equivalent); the reference terminal node)

Norton equivalent of a DC circuit between two nodes — `norton_equivalent`

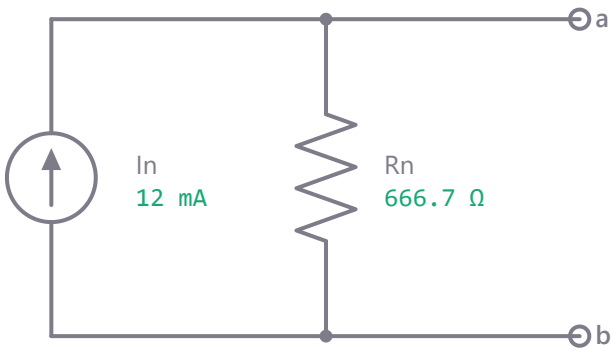
The Norton current and resistance seen between two nodes of a resistive circuit with independent sources.

$$I_N = \frac{V_{th}}{R_{th}}, \quad R_N = R_{th}$$

input	meaning	condition
`netlist`	elements separated by ';': R<name> <node> <node> <ohms>, V<name> <node+> <node-> <volts>, I<name> <node> <node> <amperes>; node 0 is ground	
`node_a`	the terminal node (the + side of the equivalent)	
`node_b`	the reference terminal node	

output	meaning
`In`	Norton (short-circuit) current from node a to node b
`Rn`	Norton resistance (equal to the Thevenin resistance)

- **Method:** Thevenin voltage and resistance by modified nodal analysis, then $I_N = V_th / R_th$
- **Assumes:** both nodes in the circuit; no ideal voltage source directly across the terminals; a unique solution (no floating node)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The Norton equivalent the package returned for the circuit V1 1 0 12; R1 1 2 1000; R2 2 0 2000 seen between nodes 2 (a) and 0 (b): a current In = 0.012 A in parallel with Rn = 666.667 Ω. Marked: In = 0.012 A, Rn = 666.667 Ω.

```
electrical_engineering_ai999x.solve('norton_equivalent', netlist='V1 1 0 12; R1 1 2 1000; R2 2 0
2000', node_a='2', node_b='0')
# -> In = 0.012 A, Rn = 666.667 ohm   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Norton equivalent between nodes 2 and 0 of the circuit: V1 1 0 12; R1 1 2 1000; R2 2 0 2000.	SOLVED	In = 0.012 A, Rn = 666.667 ohm
Find the Norton equivalent between nodes 1 and 0 of the circuit: V1 1 0 12; R1 1 0 1000.	INVALID_INPUT	Norton equivalent of a DC circuit between two nodes: an ideal voltage source is across the terminals: the Thevenin resis

Delta-to-wye (Pi-to-T) resistor transformation — `delta_to_wye`

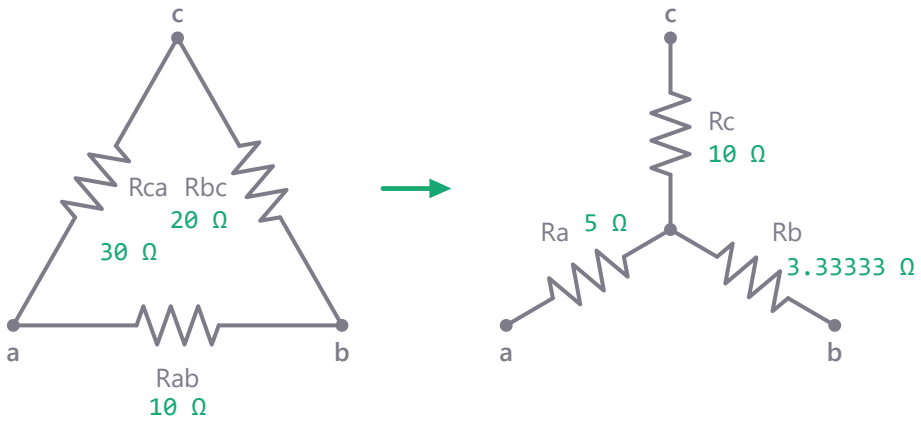
The three wye (star) resistances equivalent to a delta (triangle) of resistances between terminals a, b, c.

$$R_a = \frac{R_{ab} R_{ca}}{R_{ab} + R_{bc} + R_{ca}}$$

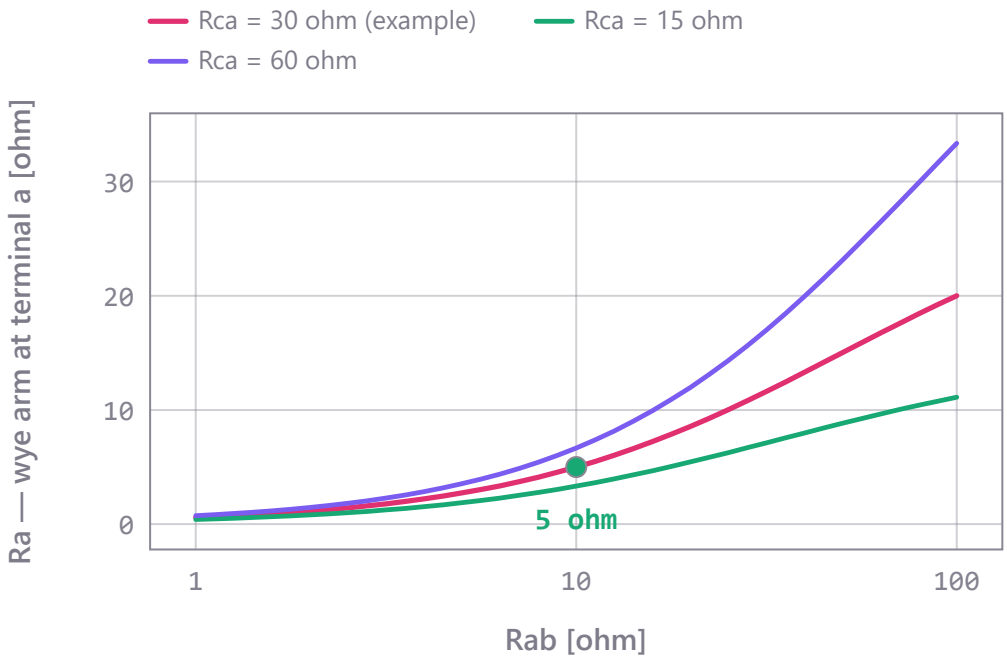
input	meaning	condition
`Rab`	delta resistance between terminals a and b	
`Rbc`	delta resistance between terminals b and c	
`Rca`	delta resistance between terminals c and a	

output	meaning
`Ra`	wye arm at terminal a
`Rb`	wye arm at terminal b
`Rc`	wye arm at terminal c

- **Method:** each arm is the product of the two adjacent delta resistances over their sum
- **Assumes:** every resistance positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Delta network as stated (Rab = 10 Ω, Rbc = 20 Ω, Rca = 30 Ω) and the equivalent wye the package returned (Ra = 5 Ω, Rb = 3.33333 Ω, Rc = 10 Ω), seen between the same terminals a, b, c. Marked: Ra = 5 Ω, Rb = 3.33333 Ω, Rc = 10 Ω.



Ra (wye arm at terminal a) against Rab (delta resistance between terminals a and b) from 1 ohm to 100 ohm on a logarithmic axis. One curve for each value of Rca (delta resistance between terminals c and a): 30 ohm, 15 ohm, 60 ohm. Other inputs as in the worked example: Rbc = 20 ohm. 41 points per curve computed by electrical-engineering-ai999x; marked: the worked example, Rab = 10 ohm, Ra = 5 ohm.

```
electrical_engineering_ai999x.solve('delta_to_wye', Rab=10, Rbc=20, Rca=30)
# -> Ra = 5 ohm, Rb = 3.33333 ohm, Rc = 10 ohm   (SOLVED)
```

```
electrical_engineering_ai999x.solve('delta_to_wye', Rab=30, Rbc=30, Rca=30)
# -> Ra = 10 ohm, Rb = 10 ohm, Rc = 10 ohm   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Convert a delta with Rab = 10 ohm, Rbc = 20 ohm, Rca = 30 ohm to a wye.	SOLVED	Ra = 5 ohm, Rb = 3.33333 ohm, Rc = 10 ohm
Convert a delta network with Rab = 3 kohm, Rbc = 3 kohm, Rca = 3 kohm to an	SOLVED	Ra = 1000 ohm, Rb = 1000 ohm, Rc = 1000 ohm

question	outcome	result
equivalent star.		
Convert a delta with Rab = 10 ohm, Rbc = 20 ohm to a wye.	NEEDS_INPUT	delta_to_wye needs Rca (delta resistance between terminals c and a). Example: solve('delta_to_wye', Rab='10 ohm', Rbc='2

Wye-to-delta (T-to-Pi) resistor transformation — `wye_to_delta`

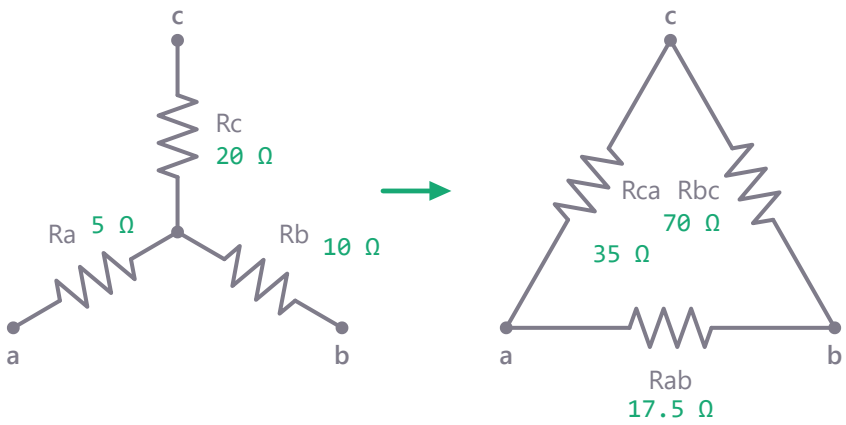
The three delta (triangle) resistances equivalent to a wye (star) of resistances at terminals a, b, c.

$$R_{ab} = \frac{R_a R_b + R_b R_c + R_c R_a}{R_c}$$

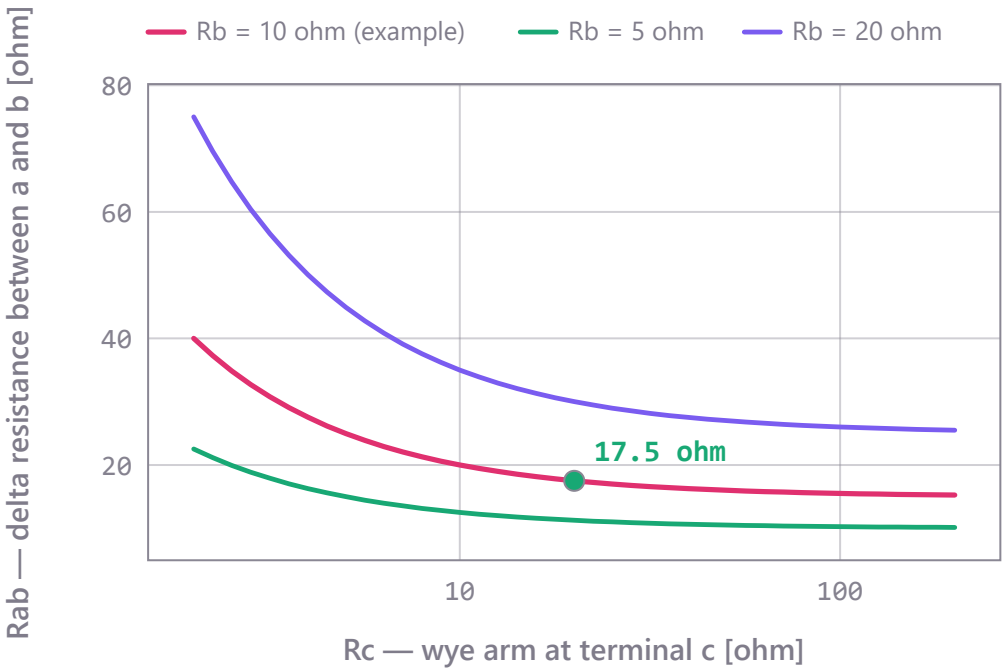
input	meaning	condition
`Ra`	wye arm at terminal a	
`Rb`	wye arm at terminal b	
`Rc`	wye arm at terminal c	

output	meaning
`Rab`	delta resistance between a and b
`Rbc`	delta resistance between b and c
`Rca`	delta resistance between c and a

- **Method:** each delta resistance is the sum of pairwise products of the arms over the opposite arm
- **Assumes:** every resistance positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Wye network as stated (Ra = 5 Ω, Rb = 10 Ω, Rc = 20 Ω) and the equivalent delta the package returned (Rab = 17.5 Ω, Rbc = 70 Ω, Rca = 35 Ω), seen between the same terminals a, b, c. Marked: Rab = 17.5 Ω, Rbc = 70 Ω, Rca = 35 Ω.



Rab (delta resistance between a and b) against Rc (wye arm at terminal c) from 2 ohm to 200 ohm on a logarithmic axis. One curve for each value of Rb (wye arm at terminal b): 10 ohm, 5 ohm, 20 ohm. Other inputs as in the worked example: Ra = 5 ohm. 41 points per curve computed by electrical-engineering-ai999x; marked: the worked example, Rc = 20 ohm, Rab = 17.5 ohm.

```
electrical_engineering_ai999x.solve('wye_to_delta', Ra=5, Rb=10, Rc=20)
# -> Rab = 17.5 ohm, Rbc = 70 ohm, Rca = 35 ohm   (SOLVED)
```

```
electrical_engineering_ai999x.solve('wye_to_delta', Ra=10, Rb=10, Rc=10)
# -> Rab = 30 ohm, Rbc = 30 ohm, Rca = 30 ohm   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Convert a wye with Ra = 5 ohm, Rb = 10 ohm, Rc = 20 ohm to a delta.	SOLVED	Rab = 17.5 ohm, Rbc = 70 ohm, Rca = 35 ohm
Convert a star network with Ra = 1 kohm, Rb = 1 kohm, Rc = 1 kohm to a delta.	SOLVED	Rab = 3000 ohm, Rbc = 3000 ohm, Rca = 3000 ohm
Convert a wye with Ra = 5 ohm, Rb = 0 ohm, Rc = 20 ohm to a delta.	INVALID_INPUT	Wye-to-delta (T-to-Pi) resistor transformation: Rb must be a positive resistance.

Routh-Hurwitz stability test — `routh_hurwitz`

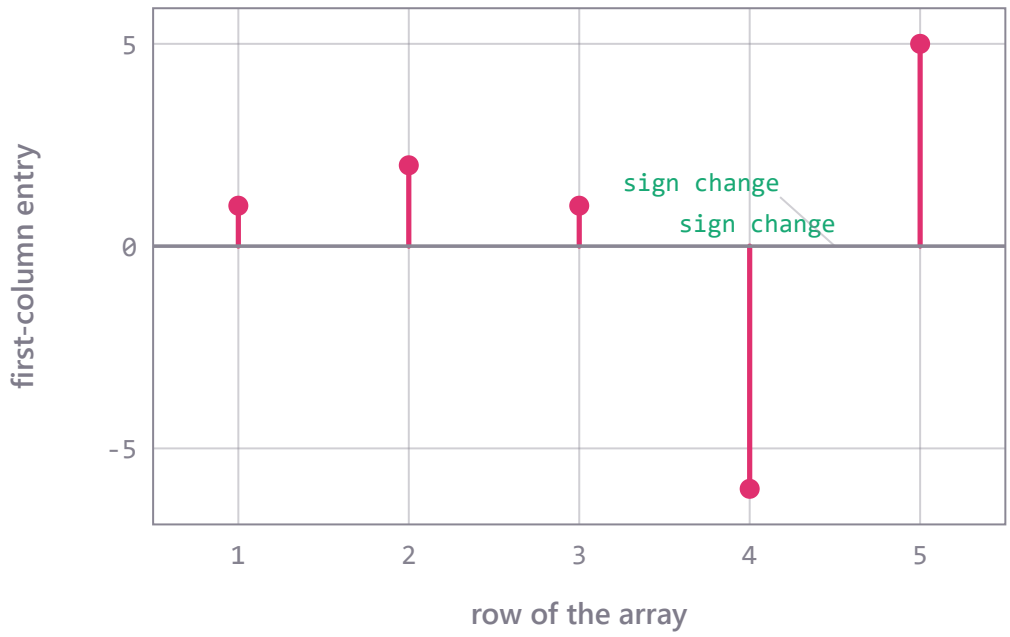
The Routh array's first column, the number of roots in the right half plane and whether a characteristic polynomial is stable.

$$b_i = \frac{a_1\, a_{0,i+1} - a_0\, a_{1,i+1}}{a_1}$$

input	meaning	condition
`coefficients`	characteristic polynomial coefficients, highest power of s first	

output	meaning
`rhp_roots`	number of roots with positive real part (sign changes in the first column)
`stable`	true when every root has a negative real part
`first_column`	the first column of the Routh array

- **Method:** Routh array built row by row; sign changes of its first column counted
- **Assumes:** no zero in the first column of the Routh array (the special cases are not handled)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



First column of the Routh array the package returned for the characteristic polynomial with coefficients 1, 2, 3, 4, 5 (highest power first): 1, 2, 1, -6, 5. 2 sign change(s), so the package returned 2 root(s) in the right half-plane (unstable). Marked: each sign change.

```
electrical_engineering_ai999x.solve('routh_hurwitz', coefficients='1, 2, 3, 4, 5')
# -> rhp_roots = 2, stable = False, first_column = [1.0, 2.0, 1.0, -6.0, 5.0]   (SOLVED)
```

```
electrical_engineering_ai999x.solve('routh_hurwitz', coefficients=[1, 6, 11, 6])
# -> rhp_roots = 0, stable = True, first_column = [1.0, 6.0, 10.0, 6.0]   (SOLVED)
```

```
electrical_engineering_ai999x.solve('routh_hurwitz', coefficients=[1, 1, 2, 24])
# -> rhp_roots = 2, stable = False, first_column = [1.0, 1.0, -22.0, 24.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use the Routh-Hurwitz criterion on s^4 + 2s^3 + 3s^2 + 4s + 5.	SOLVED	rhp_roots = 2, stable = False, first_column = [1.0, 2.0, 1.0, -6.0, 5.0]
Apply the Routh test to the characteristic polynomial s^3 + 6s^2 + 11s + 6 = 0.	SOLVED	rhp_roots = 0, stable = True, first_column = [1.0, 6.0, 10.0, 6.0]
Apply the Routh test to (s+1)(s-2)(s+3).	SOLVED	rhp_roots = 1, stable = False, first_column = [1.0, 2.0, -2.0, -6.0]

question	outcome	result
Use the Routh-Hurwitz criterion on s^3 + 2s^2 + s + 2.	INVALID_INPUT	Routh-Hurwitz stability test: a zero appears in the first column of the Routh array (a special case: roots on or symmetr

Closed-loop poles with unity negative feedback — `closed_loop_poles`

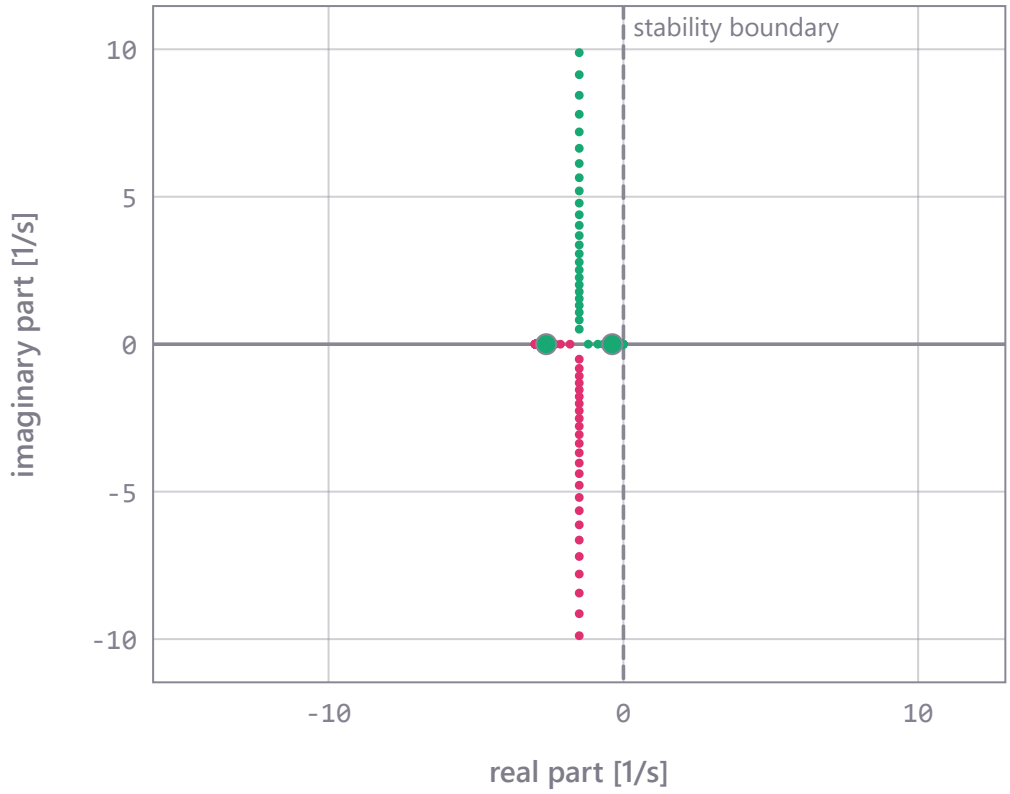
The poles of $KG(s)/(1+KG(s))$: the roots of den(s) + K*num(s), and whether the closed loop is stable.

$$1 + K\,G(s) = 0 \iff \mathrm{den}(s) + K\,\mathrm{num}(s) = 0$$

input	meaning	condition
`num`	numerator coefficients of the open-loop G(s), highest power of s first	
`den`	denominator coefficients of G(s), highest power of s first	
`K`	loop gain multiplying G(s) (default 1)	

output	meaning
`poles`	closed-loop poles (real first by real part, conjugate pairs together)
`max_real_part`	largest real part of the poles (negative when stable)
`stable`	true when every pole has a negative real part
`characteristic`	closed-loop characteristic polynomial den + K num

- **Method:** roots of den(s) + K num(s) by the Aberth-Ehrlich iteration
- **Assumes:** a proper G(s); unity negative feedback
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Closed-loop poles of K·G(s) with G(s) = (1) / (1, 3, 0) under unity feedback, for 61 gains K from 0.01 to 100 (logarithmic steps), each set the package returned. Marked: the poles at K = 1: -2.61803, -0.381966 (stable).

```
electrical_engineering_ai999x.solve('closed_loop_poles', num='1', den='1, 3, 0')
# -> poles = [-2.618033988749895, -0.38196601125010515], max_real_part = -0.381966, stable = True,
characteristic = [1.0, 3.0, 1.0]   (SOLVED)
```

```
electrical_engineering_ai999x.solve('closed_loop_poles', num=[10], den=[1, 6, 5, 0])
# -> poles = [-5.417802089290255, (-0.29109895535487257-1.3270374302059544j),
(-0.29109895535487257+1.3270374302059544j)], max_real_part = -0.291099, stable = True,
characteristic = [1.0, 6.0, 5.0, 10.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the closed-loop poles of G(s) = 2/(s^2 + 3s).	SOLVED	poles = [-2.0, -1.0000000000000002], max_real_part = -1, stable = True, characteristic = [1.0, 3.0, 2.0]
What are the closed-loop poles of G(s) = 1/(s(s+2)) with K = 2?	SOLVED	poles = [(-1-1j), (-1+1j)], max_real_part = -1, stable = True, characteristic = [1.0, 2.0, 2.0]
Find the closed-loop poles of G(s) = 10/(s(s+1)(s+5)).	SOLVED	poles = [-5.417802089290255, (-0.29109895535487257-1.3270374302059544j), (-0.29109895535487257+1.3270374302059544j)], max_real_part = -0.291099, stable = True, characteristic = [1.0, 6.0, 5.0, 10.0]
Find the closed-loop poles of G(s) = (s^3+1)/(s+1).	INVALID_INPUT	Closed-loop poles with unity negative feedback: the transfer function must be proper (numerator degree <= denominator de

Phase margin and gain-crossover frequency — `phase_margin`

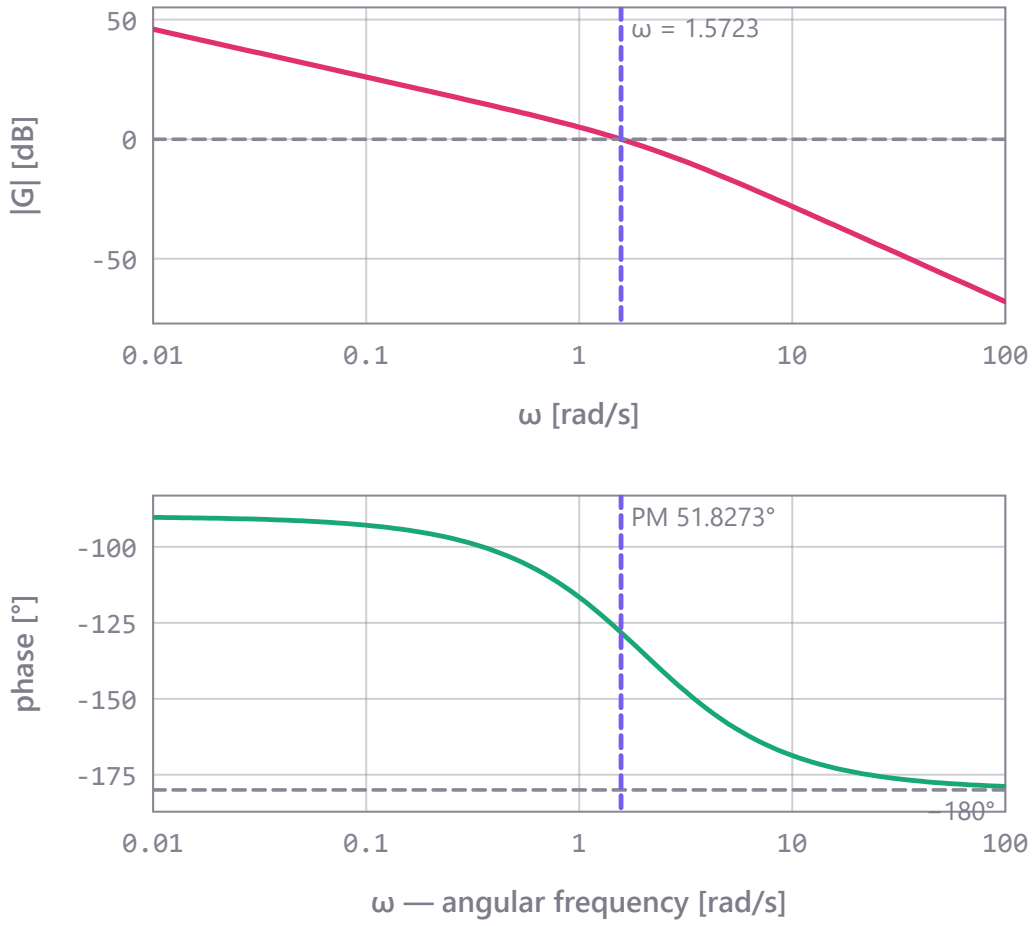
The phase margin of a unity-feedback loop with open-loop G(s): 180 degrees plus the phase of G at the frequency where |G(jw)| = 1.

$$|G(j\omega_c)| = 1, \quad \mathrm{PM} = 180^\circ + \angle G(j\omega_c)$$

input	meaning	condition
`num`	numerator coefficients of the open-loop G(s), highest power of s first	
`den`	denominator coefficients of G(s), highest power of s first	

output	meaning
`phase_margin`	phase margin in degrees
`gain_crossover`	gain-crossover frequency (|G| = 1)

- **Method:** the gain crossover as the positive real root of |N(jw)|^2 - |D(jw)|^2 (Aberth iteration, Newton-polished); PM = (arg G(jw_c) mod 360) - 180
- **Assumes:** a proper G(s); exactly one gain crossover
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Bode magnitude (top) and phase (bottom, unwrapped) of G(s) = (4) / (1, 2, 0) at 121 frequencies from 0.01 to 100 rad/s, each a run of the package's frequency response. Marked: the gain crossover ω = 1.5723 rad/s and the phase margin the package returned, 51.8273°.

```
electrical_engineering_ai999x.solve('phase_margin', num='4', den='1, 2, 0')
# -> phase_margin = 51.8273, gain_crossover = 1.5723 rad/s   (SOLVED)
```

```
electrical_engineering_ai999x.solve('phase_margin', num=[10], den=[1, 6, 5, 0])
# -> phase_margin = 25.3898, gain_crossover = 1.22706 rad/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the phase margin of G(s) = 4/(s(s+2))?	SOLVED	phase_margin = 51.8273, gain_crossover = 1.5723 rad/s
Find the phase margin of G(s) = 10/(s(s+1)(s+5)).	SOLVED	phase_margin = 25.3898, gain_crossover = 1.22706 rad/s
Find the phase margin of G(s) = 0.5/(s+1).	INVALID_INPUT	Phase margin and gain-crossover frequency: |G(jw)| crosses 1 nowhere: this capability needs exactly one gain crossover.

Gain margin and phase-crossover frequency — `gain_margin`

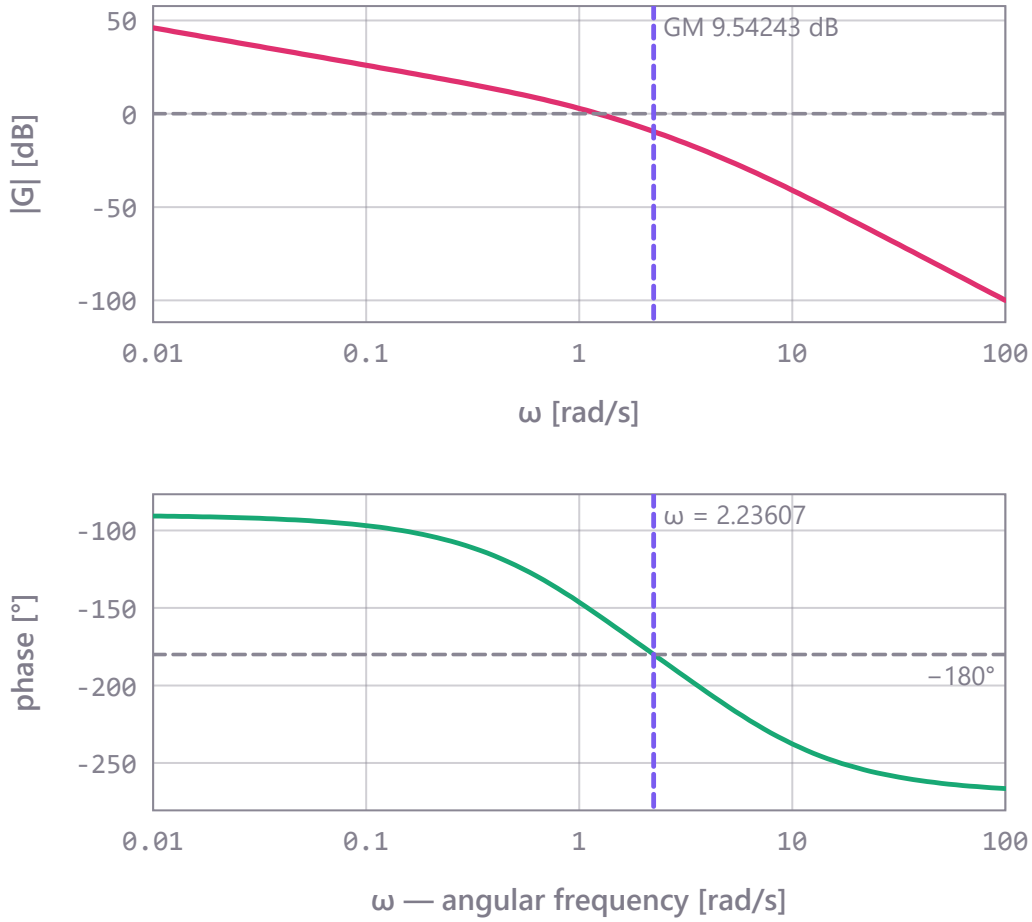
The gain margin of a unity-feedback loop with open-loop G(s): 1/|G(jw)| at the frequency where the phase $is -180$ degrees.

$$\angle G(j\omega_{pc}) = -180^\circ, \quad \mathrm{GM} = \frac{1}{|G(j\omega_{pc})|}$$

input	meaning	condition
`num`	numerator coefficients of the open-loop G(s), highest power of s first	
`den`	denominator coefficients of G(s), highest power of s first	

output	meaning
`gain_margin`	gain margin as a ratio
`gain_margin_db`	gain margin in decibels
`phase_crossover`	phase-crossover frequency (phase -180 degrees)

- **Method:** the phase crossover as the positive real root of Im(N(jw) conj D(jw)) with Re $G < 0$ (Aberth iteration, Newton-polished)
- **Assumes:** a proper G(s); exactly one phase crossover with G negative real
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Bode magnitude (top) and phase (bottom, unwrapped) of G(s) = (10) / (1, 6, 5, 0) at 121 frequencies from 0.01 to 100 rad/s, each a run of the package's frequency response. Marked: the phase crossover ω = 2.23607 rad/s and the gain margin the package returned, 9.54243 dB (factor 3).

```
electrical_engineering_ai999x.solve('gain_margin', num='10', den='1, 6, 5, 0')
# -> gain_margin = 3, gain_margin_db = 9.54243, phase_crossover = 2.23607 rad/s   (SOLVED)
```

```
electrical_engineering_ai999x.solve('gain_margin', num=[6], den=[1, 3, 3, 1])
# -> gain_margin = 1.33333, gain_margin_db = 2.49877, phase_crossover = 1.73205 rad/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the gain margin of G(s) = 10/(s(s+1)(s+5)).	SOLVED	gain_margin = 3, gain_margin_db = 9.54243, phase_crossover = 2.23607 rad/s
What is the gain margin of G(s) = 6/(s+1)^3?	SOLVED	gain_margin = 1.33333, gain_margin_db = 2.49877, phase_crossover = 1.73205 rad/s
Find the gain margin of G(s) = 4/(s(s+2)).	INVALID_INPUT	Gain margin and phase-crossover frequency: the phase never reaches -180 degrees, so the gain margin is infinite: this ca

Frequency response (Bode magnitude and phase) at one frequency — `bode_point`

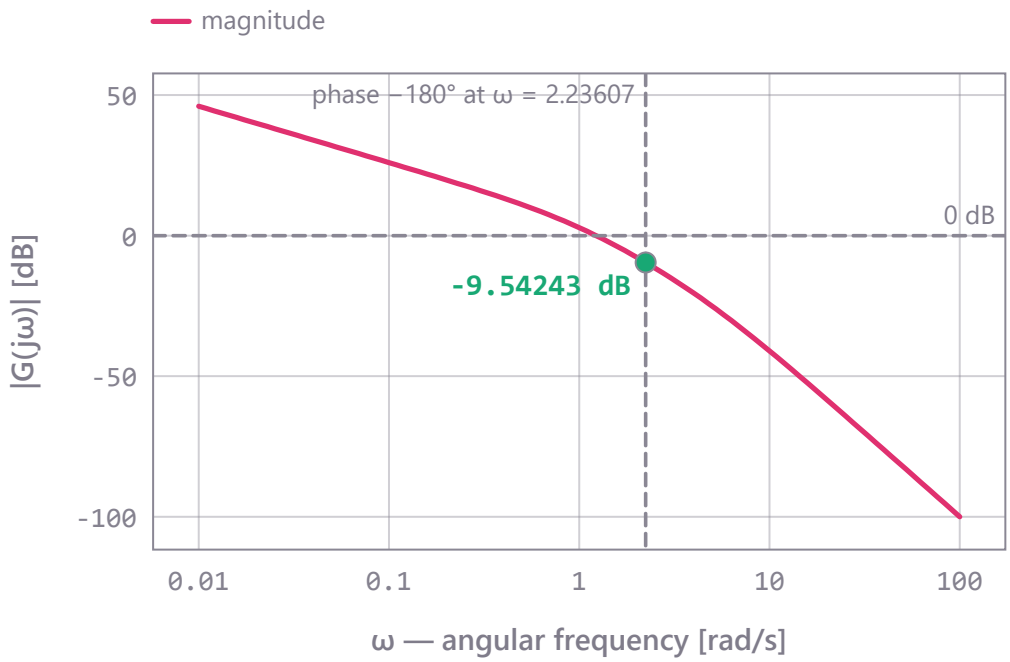
|G(jw)| as a ratio and in decibels, and the phase of G(jw), for a transfer function at an angular frequency w.

$$G(j\omega) = \frac{N(j\omega)}{D(j\omega)}, \quad 20\log_{10}|G(j\omega)|$$

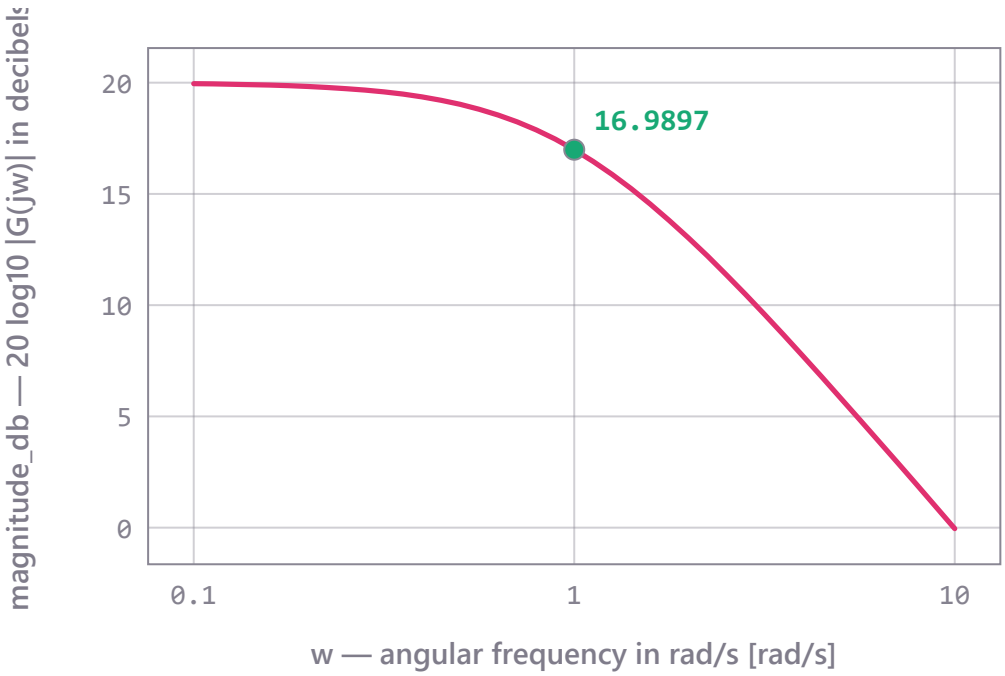
input	meaning	condition
`num`	numerator coefficients of the open-loop G(s), highest power of s first	
`den`	denominator coefficients of G(s), highest power of s first	
`w`	angular frequency in rad/s	

output	meaning
`magnitude`	|G(jw)|
`magnitude_db`	20 log10 |G(jw)| in decibels
`phase_deg`	phase of G(jw) in degrees, in (-180, 180]
`real`	real part of G(jw)
`imag`	imaginary part of G(jw)

- **Method:** Horner evaluation of numerator and denominator at $s = jw$
- **Assumes:** no pole of G on the imaginary axis at w; $w > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Bode magnitude of G(s) = 10 / (s^3 + 6s^2 + 5s), 101 frequencies from 0.01 to 100 rad/s computed by the package. At the phase crossover ω = 2.23607 rad/s the gain is -9.54243 dB, so the gain margin returned by the package is 9.54243 dB (a factor 3); marked: ω = 2.23607 rad/s, |G| = -9.54243 dB.



magnitude_db (20 log10 |G(jw)| in decibels) against w (angular frequency in rad/s) from 0.1 rad/s to 10 rad/s on a logarithmic axis. Other inputs as in the worked example: num = 10, den = 1, 1. 41 points computed by electrical-engineering-ai999x; marked: the worked example, w = 1 rad/s, magnitude_db = 16.9897.

```
electrical_engineering_ai999x.solve('bode_point', num='10', den='1, 1', w=1)
# -> magnitude = 7.07107, magnitude_db = 16.9897, phase_deg = -45, real = 5, imag = -5   (SOLVED)
```

```
electrical_engineering_ai999x.solve('bode_point', num=[100], den=[1, 10, 100], w=10)
# -> magnitude = 1, magnitude_db = 0, phase_deg = -90, real = 0, imag = -1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Bode magnitude of G(s) = 10/(s+1) at 1 rad/s.	SOLVED	magnitude = 7.07107, magnitude_db = 16.9897, phase_deg = -45, real = 5, imag = -5
What is the frequency response of G(s) = 100/(s^2 + 10s + 100) at 10 rad/s?	SOLVED	magnitude = 1, magnitude_db = 0, phase_deg = -90, real = 0, imag = -1
Find the Bode magnitude of G(s) = 1/(s^2 + 4) at 2 rad/s.	INVALID_INPUT	Frequency response (Bode magnitude and phase) at one frequency: G(s) has a pole on the imaginary axis at this frequency.
Find the Bode magnitude of G(s) = 10/(s+1).	NEEDS_INPUT	bode_point needs w (angular frequency in rad/s). Example: solve('bode_point', num='10', den='1, 1', w='1 rad/s').

Steady-state error of a unity-feedback system — `steady_state_error`

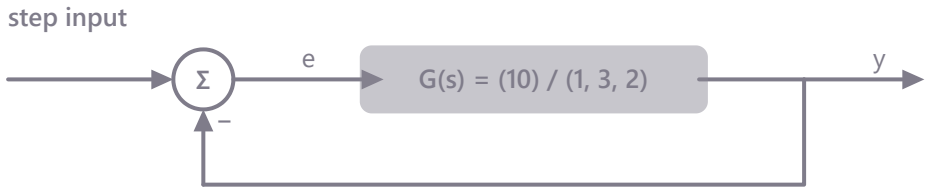
The system type, the error constant (Kp, Kv or Ka) and the steady-state error for a unit step, ramp or parabola input.

$$e_{ss} = \lim_{s \to 0} \frac{s\,R(s)}{1 + G(s)}$$

input	meaning	condition
`num`	numerator coefficients of the open-loop G(s), highest power of s first	
`den`	denominator coefficients of G(s), highest power of s first	
`input`	the test input: unit step $1/s$, unit ramp $1/s^2$ or unit parabola $t^2/2$ $(1/s^3)$	

output	meaning
`steady_state_error`	steady-state error (or 'infinite')
`error_constant`	position (Kp), velocity (Kv) or acceleration (Ka) error constant
`system_type`	number of open-loop poles at the origin

- **Method:** system type from the poles at the origin; error constant lim s^k G(s); $e_{ss} = 1/(1 + Kp)$ for a step, $1/Kv$ for a ramp, $1/Ka$ for a parabola
- **Assumes:** a stable closed loop; unity negative feedback; unit-amplitude test input
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



`steady-state error 0.166667, error constant 5, type 0`

Unity-feedback loop with the stated open-loop G(s) = (10) / (1, 3, 2) and a step input. Labelled with what the package returned: steady-state error 0.166667, error constant 5, system type 0. Marked: e_ss = 0.166667.

```
electrical_engineering_ai999x.solve('steady_state_error', num='10', den='1, 3, 2', input='step')
# -> steady_state_error = 0.166667, error_constant = 5, system_type = 0   (SOLVED)
```

```
electrical_engineering_ai999x.solve('steady_state_error', num=[20], den=[1, 5, 0], input='ramp')
# -> steady_state_error = 0.25, error_constant = 4, system_type = 1   (SOLVED)
```

```
electrical_engineering_ai999x.solve('steady_state_error', num=[20], den=[1, 5, 0], input='step')
# -> steady_state_error = 0, error_constant = infinite, system_type = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the steady-state error of G(s) = 10/((s+1)(s+2)) for a step input.	SOLVED	steady_state_error = 0.166667, error_constant = 5, system_type = 0
What is the steady-state error of G(s) = 20/(s(s+5)) for a ramp input?	SOLVED	steady_state_error = 0.25, error_constant = 4, system_type = 1

question	outcome	result
What is the steady-state error of G(s) = 10/((s+1)(s+2)) for a ramp input?	SOLVED	steady_state_error = infinite, error_constant = 0, system_type = 0
Find the steady-state error of G(s) = 1/((s-1)(s+2)) for a step input.	INVALID_INPUT	Steady-state error of a unity-feedback system: the closed loop is not stable, so there is no steady-state error.
Find the steady-state error of G(s) = 10/((s+1)(s+2)).	NEEDS_INPUT	steady_state_error needs input (the test input: unit step 1/s, unit ramp 1/s^2 or unit parabola t^2/2 (1/s^3)). Example:

Step-response specifications of a standard second-order system — `second_order_step`

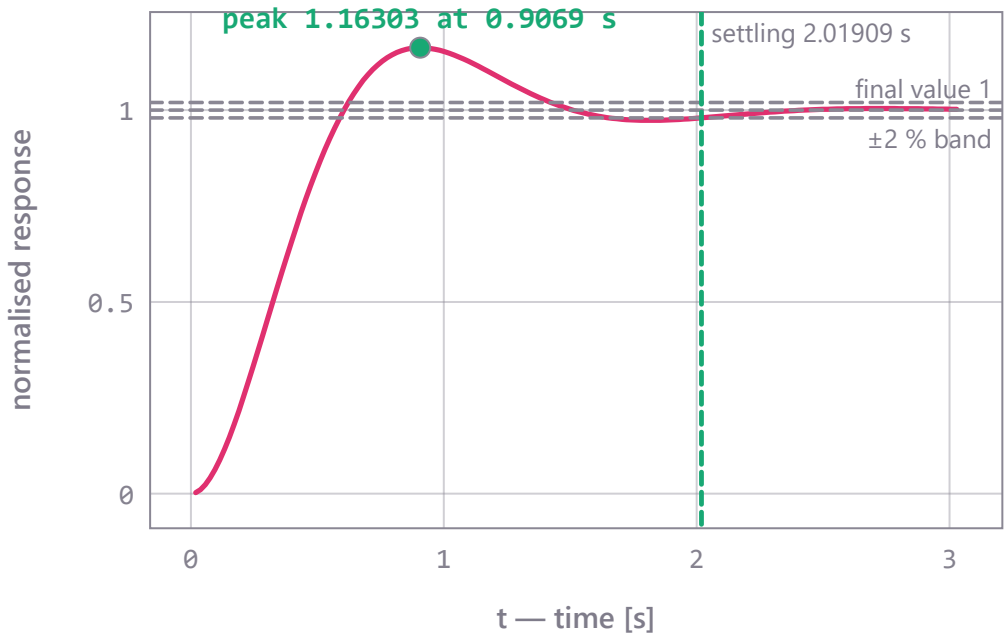
Percent overshoot, peak time, $10-90\%$ rise time and exact 2% settling time of wn^2/(s^2 + 2 zeta wn s + wn^2) for a unit step.

$$M_p = e^{-\zeta\pi/\sqrt{1-\zeta^2}}, \quad t_p = \frac{\pi}{\omega_n\sqrt{1-\zeta^2}}$$

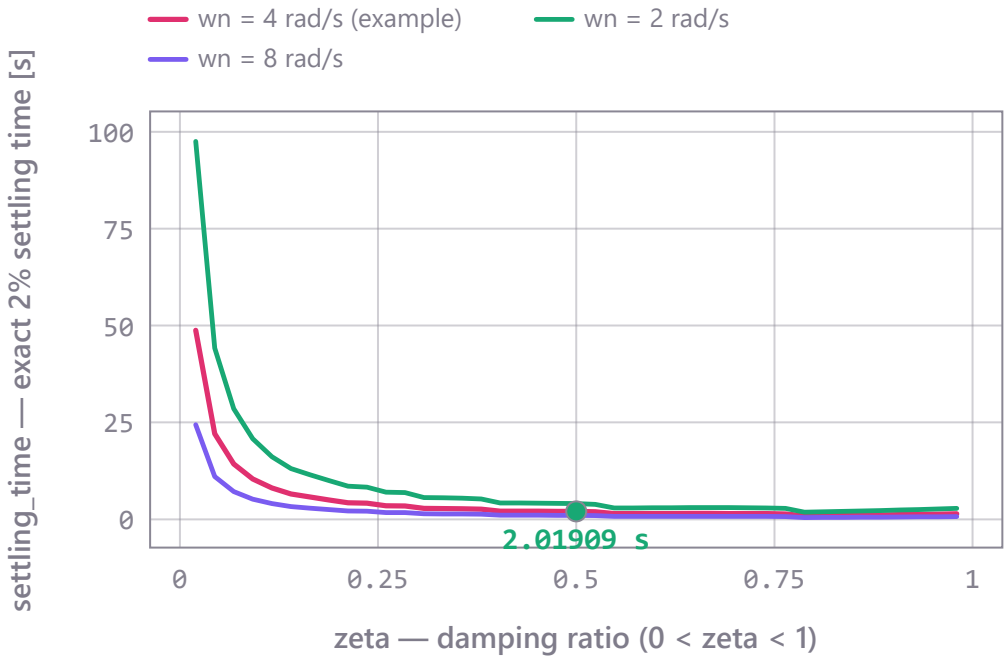
input	meaning	condition
`zeta`	damping ratio $(0 < \zeta < 1)$	
`wn`	undamped natural frequency	

output	meaning
`percent_overshoot`	maximum overshoot in percent
`overshoot`	maximum overshoot as a fraction of the final value
`peak_time`	time of the first peak
`rise_time`	10% to 90% rise time
`t10`	time to reach 10% of the final value
`t90`	time to reach 90% of the final value
`settling_time`	exact 2% settling time (last time the response leaves the band)
`damped_frequency`	damped natural frequency wn sqrt(1 - zeta^2)

- **Method:** closed-form overshoot and peak time; rise and settling times by bisection on the exact response (the last 2% crossing lies between the last peak above the band and the following zero of the error)
- **Assumes:** $0 < \zeta < 1$ (underdamped); unit step from rest; unity DC gain
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Unit step response of a second-order system with ζ = 0.5, ωn = 4 rad/s (as stated), computed as the capacitor voltage the package returned for a series RLC circuit with the same ζ and ωn (L = 1 H, C = 0.0625 F, R = 4 Ω, 1 V step), 161 instants. Marked from the metrics the package returned: overshoot 16.3034 % (peak 1.16303 at t = 0.9069 s), 2 % settling time 2.01909 s, rise time 0.409393 s.



settling_time (exact 2% settling time (last time the response leaves the band)) against zeta (damping ratio (0 < zeta < 1)) from 0.02 to 0.98. One curve for each value of wn (undamped natural frequency): 4 rad/s, 2 rad/s, 8 rad/s. 41 points per curve computed by electrical-engineering-ai999x; marked: the worked example, zeta = 0.5, settling_time = 2.01909 s.

```
electrical_engineering_ai999x.solve('second_order_step', zeta=0.5, wn=4)
# -> percent_overshoot = 16.3034, overshoot = 0.163034, peak_time = 0.9069 s, rise_time = 0.409393
s, t10 = 0.122057 s, t90 = 0.531451 s, settling_time = 2.01909 s, damped_frequency = 3.4641 rad/s
(SOLVED)
```

```
electrical_engineering_ai999x.solve('second_order_step', zeta=0.7, wn=10)
# -> percent_overshoot = 4.59879, overshoot = 0.0459879, peak_time = 0.439911 s, rise_time =
0.21262 s, t10 = 0.0504835 s, t90 = 0.263104 s, settling_time = 0.597879 s, damped_frequency =
7.14143 rad/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the step response specifications of a second-order system with damping ratio 0.5 and natural frequency 4 rad/s.	SOLVED	percent_overshoot = 16.3034, overshoot = 0.163034, peak_time = 0.9069 s, rise_time = 0.409393 s, t10 = 0.122057 s, t90 = 0.531451 s, settling_time = 2.01909 s, damped_frequency = 3.4641 rad/s
A second-order system has zeta = 1.2 and wn = 5 rad/s. Find its step response overshoot.	INVALID_INPUT	Step-response specifications of a standard second-order system: the damping ratio must be between 0 and 1 (an underdampe
A second-order system has damping ratio 0.3. Find its step response settling time.	NEEDS_INPUT	second_order_step needs wn (undamped natural frequency). Example: solve('second_order_step', zeta=0.5, wn='4 rad/s').

Impedance of a series RLC circuit — `series_rlc_impedance`

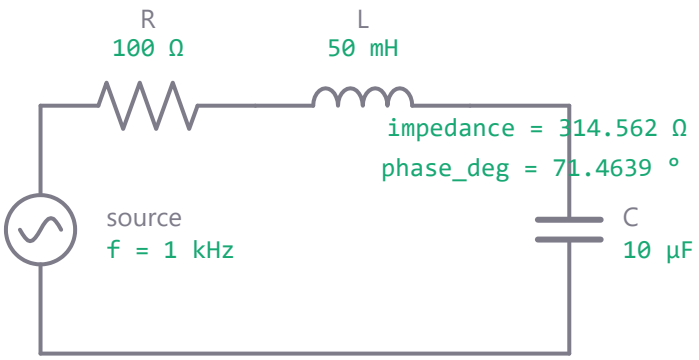
The complex impedance of R, L and C in series at a frequency: magnitude, phase angle and the two reactances.

$$Z = R + j\left(\omega L - \frac{1}{\omega C}\right)$$

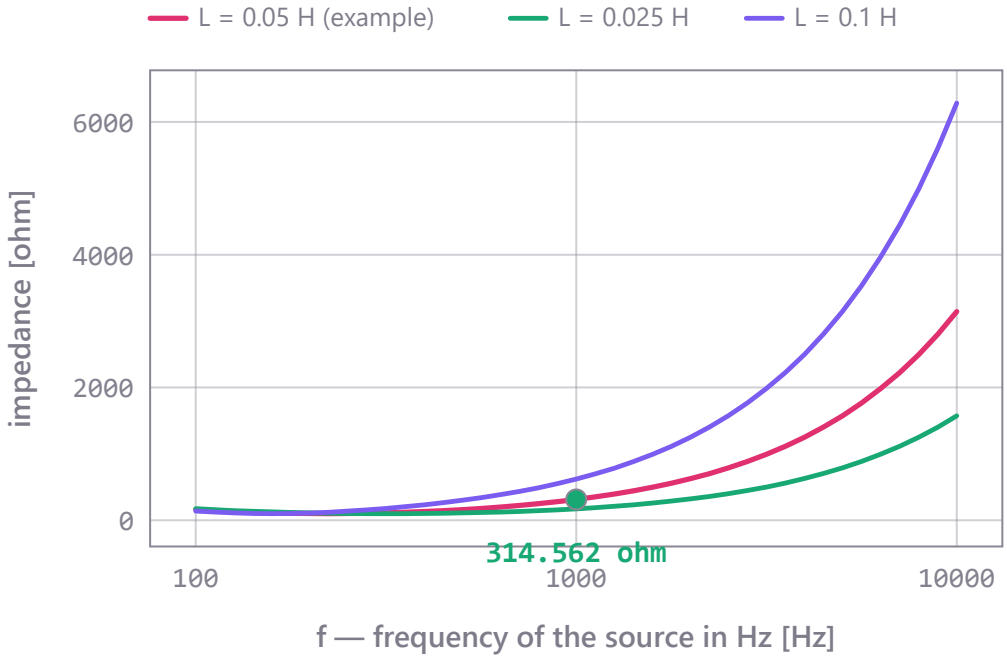
input	meaning	condition
`R`	resistance	
`L`	inductance	
`C`	capacitance	
`f`	frequency of the source in Hz	

output	meaning
`impedance`	magnitude of the impedance |Z|
`phase_deg`	phase angle of Z in degrees (positive: inductive)
`real`	real part of Z
`imag`	imaginary part of Z (net reactance)
`XL`	inductive reactance wL
`XC`	capacitive reactance $1/(wC)$

- **Method:** $Z = R + j(wL - 1/(wC))$ with $w = 2$ pi f
- **Assumes:** $R \geq 0$, $L > 0$, $C > 0$, $f > 0$; sinusoidal steady state
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The series RLC circuit of the worked example (R = 100, L = 0.05, C = 1e-05, f = 1000), labelled with what the package returned: impedance = 314.562; phase_deg = 71.4639. Marked: impedance = 314.562.



impedance (magnitude of the impedance |Z|) against f (frequency of the source in Hz) from 100 Hz to 10000 Hz on a logarithmic axis. One curve for each value of L (inductance): 0.05 H, 0.025 H, 0.1 H. Other inputs as in the worked example: R = 100 ohm, C = 1e-05 F. 41 points per curve computed by electrical-engineering-ai999x; marked: the worked example, f = 1000 Hz, impedance = 314.562 ohm.

```
electrical_engineering_ai999x.solve('series_rlc_impedance', R=100, L=0.05, C=1e-05, f=1000)
# -> impedance = 314.562 ohm, phase_deg = 71.4639, real = 100 ohm, imag = 298.244 ohm, XL = 314.159
ohm, XC = 15.9155 ohm   (SOLVED)
```

```
electrical_engineering_ai999x.solve('series_rlc_impedance', R=3, L=0.012732395447351627,
C=0.00039788735772973834, f=50)
# -> impedance = 5 ohm, phase_deg = -53.1301, real = 3 ohm, imag = -4 ohm, XL = 4 ohm, XC = 8 ohm
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the impedance of a series RLC circuit with R = 100 ohm, L = 50 mH and C = 10	SOLVED	impedance = 314.562 ohm, phase_deg = 71.4639, real = 100 ohm, imag = 298.244 ohm, XL = 314.159 ohm, XC = 15.9155

question	outcome	result
uF at 1 kHz.		ohm
What is the impedance of a series RLC circuit with R = 10 ohm, L = 0.1 H and C = 100 uF at 50 Hz?	SOLVED	impedance = 10.0086 ohm, phase_deg = -2.37677, real = 10 ohm, imag = -0.415062 ohm, XL = 31.4159 ohm, XC = 31.831 ohm
Find the impedance of a series RLC circuit with R = 100 ohm, L = 50 mH and C = 10 uF.	NEEDS_INPUT	series_rlc_impedance needs f (frequency of the source in Hz). Example: solve('series_rlc_impedance', R='100 ohm', L='0.0
Find the impedance of a series RLC circuit with R = 100 ohm, L = 0 mH and C = 10 uF at 1 kHz.	INVALID_INPUT	Impedance of a series RLC circuit: L must be positive.

Impedance of a parallel RLC circuit — `parallel_rlc_impedance`

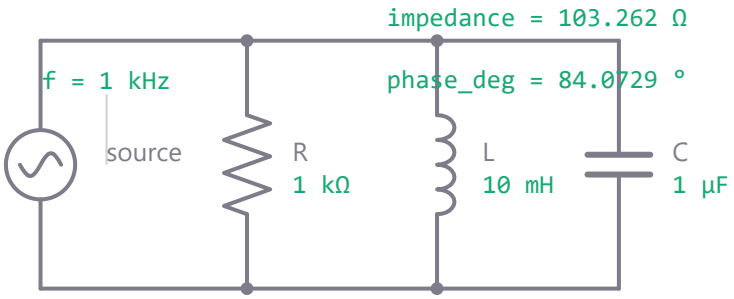
The complex impedance of R, L and C in parallel at a frequency: magnitude, phase angle and the two reactances.

$$\frac{1}{Z} = \frac{1}{R} + j\left(\omega C - \frac{1}{\omega L}\right)$$

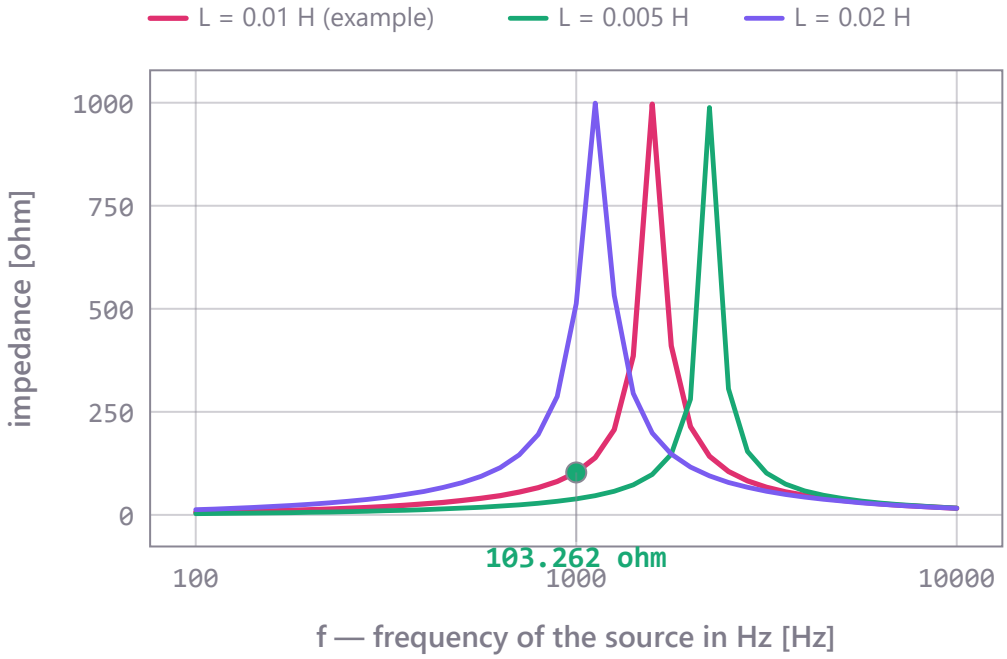
input	meaning	condition
`R`	resistance	
`L`	inductance	
`C`	capacitance	
`f`	frequency of the source in Hz	

output	meaning
`impedance`	magnitude of the impedance |Z|
`phase_deg`	phase angle of Z in degrees (positive: inductive)
`real`	real part of Z
`imag`	imaginary part of Z (net reactance)
`XL`	inductive reactance wL
`XC`	capacitive reactance $1/(wC)$

- **Method:** $Y = 1/R + j(wC - 1/(wL))$, $Z = 1/Y$ with $w = 2$ pi f
- **Assumes:** R, L, C, $f > 0$; sinusoidal steady state
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The parallel RLC circuit of the worked example (R = 1000, L = 0.01, C = 1e-06, f = 1000), labelled with what the package returned: impedance = 103.262; phase_deg = 84.0729. Marked: impedance = 103.262.



impedance (magnitude of the impedance |Z|) against f (frequency of the source in Hz) from 100 Hz to 10000 Hz on a logarithmic axis. One curve for each value of L (inductance): 0.01 H, 0.005 H, 0.02 H. Other inputs as in the worked example: R = 1000 ohm, C = 1e-06 F. 41 points per curve computed by electrical-engineering-ai999x; marked: the worked example, f = 1000 Hz, impedance = 103.262 ohm.

```
electrical_engineering_ai999x.solve('parallel_rlc_impedance', R=1000, L=0.01, C=1e-06, f=1000)
# -> impedance = 103.262 ohm, phase_deg = 84.0729, real = 10.6631 ohm, imag = 102.71 ohm, XL =
62.8319 ohm, XC = 159.155 ohm   (SOLVED)
```

```
electrical_engineering_ai999x.solve('parallel_rlc_impedance', R=50, L=0.00015915494309189535,
C=0.00015915494309189535, f=1000)
# -> impedance = 50 ohm, phase_deg = 0, real = 50 ohm, imag = 0 ohm, XL = 1 ohm, XC = 1 ohm
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the impedance of a parallel RLC circuit with R = 1 kohm, L = 10 mH and C = 1 uF at	SOLVED	impedance = 103.262 ohm, phase_deg = 84.0729, real = 10.6631 ohm, imag = 102.71 ohm, XL = 62.8319 ohm, XC =

question	outcome	result
1 kHz.		159.155 ohm
Find the impedance of a parallel RLC circuit with R = 0 ohm, L = 10 mH and C = 1 uF at 1 kHz.	INVALID_INPUT	Impedance of a parallel RLC circuit: R must be positive.

Phasor from polar to rectangular form — `polar_to_rectangular`

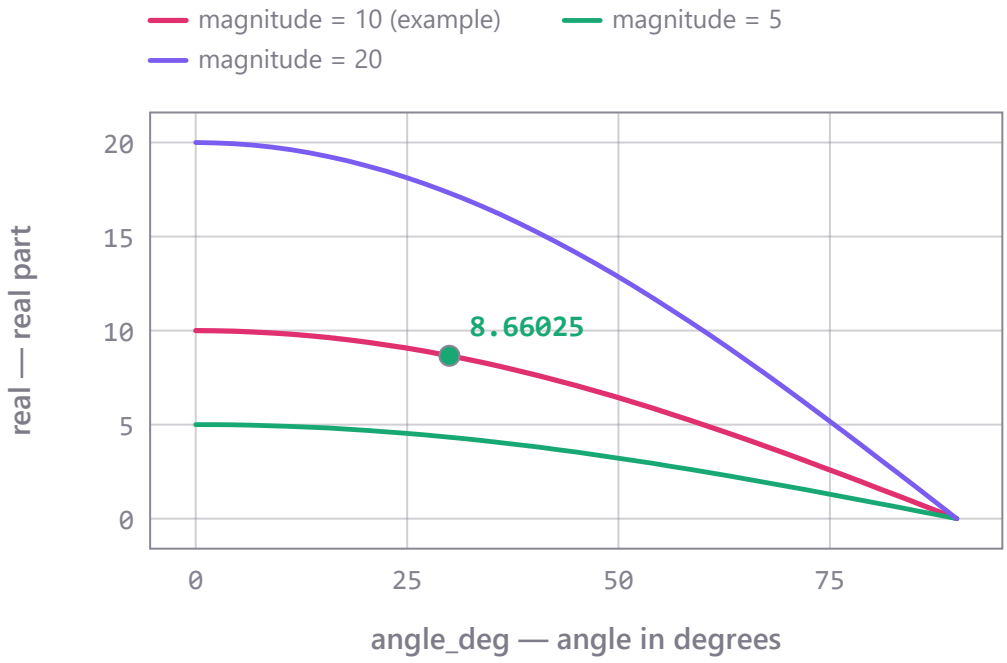
The real and imaginary parts of a phasor given as a magnitude and an angle in degrees.

$$M\angle\theta = M\cos\theta + jM\sin\theta$$

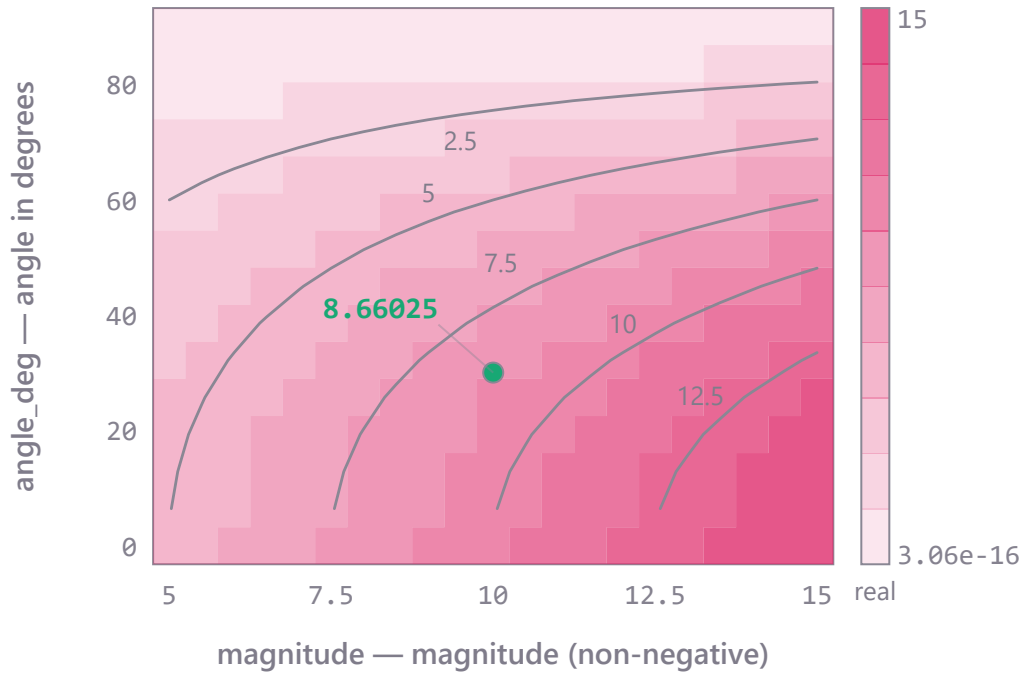
input	meaning	condition
`magnitude`	magnitude (non-negative)	
`angle_deg`	angle in degrees	

output	meaning
`real`	real part
`imag`	imaginary part

- **Method:** $x = M$ cos(theta), $y = M$ sin(theta)
- **Assumes:** magnitude >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



real (real part) against angle_deg (angle in degrees) from 0 to 90. One curve for each value of magnitude (magnitude (non-negative)): 10, 5, 20. 41 points per curve computed by electrical-engineering-ai999x; marked: the worked example, angle_deg = 30, real = 8.66025.



real (real part) over magnitude (magnitude (non-negative)) from 5 to 15 and angle_deg (angle in degrees) from 0 to 90: each of the 21 × 15 cells is one run of electrical-engineering-ai999x, shaded by its value (light 3.0616e-16 to dark 15); iso-lines at real = 2.5, 5, 7.5, 10, 12.5. Marked: the worked example, magnitude = 10, angle_deg = 30, real = 8.66025.

```
electrical_engineering_ai999x.solve('polar_to_rectangular', magnitude=10, angle_deg=30)
# -> real = 8.66025, imag = 5   (SOLVED)
```

```
electrical_engineering_ai999x.solve('polar_to_rectangular', magnitude=2, angle_deg=-90)
# -> real = 1.22465e-16, imag = -2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Convert 10∠30° to rectangular form.	SOLVED	real = 8.66025, imag = 5
Convert the phasor 5 at 53.13 degrees to rectangular form.	SOLVED	real = 3.00001, imag = 3.99999
Convert -3∠45° to rectangular form.	INVALID_INPUT	Phasor from polar to rectangular form: the magnitude must not be negative.

Phasor from rectangular to polar form — `rectangular_to_polar`

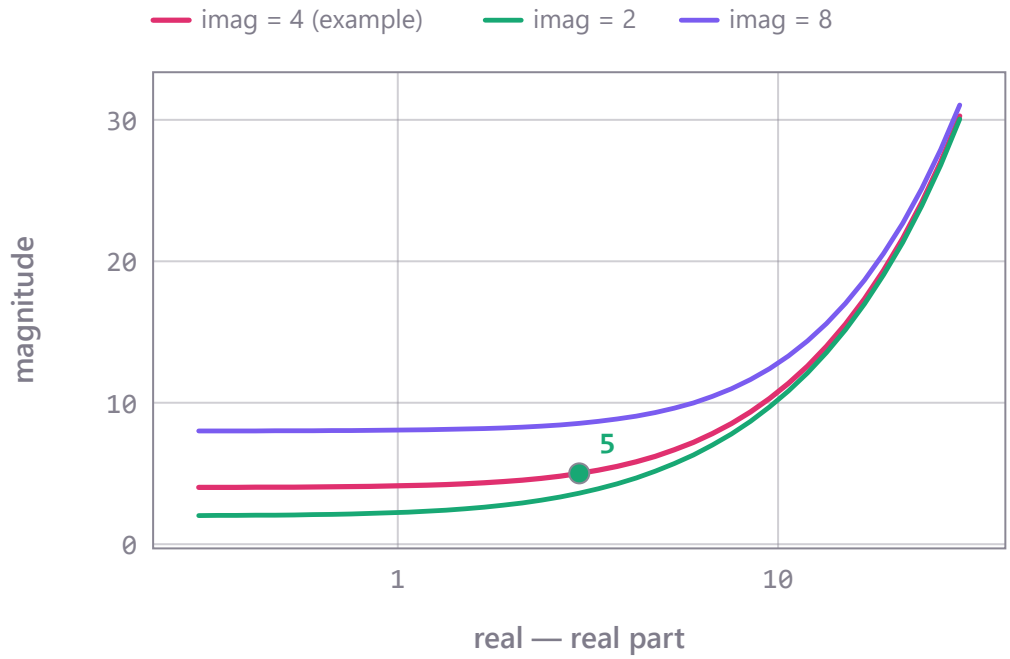
The magnitude and angle (degrees, in (-180, 180]) of a phasor given as real + j imaginary.

$$x + jy = \sqrt{x^2 + y^2} \angle \operatorname{atan2}(y, x)$$

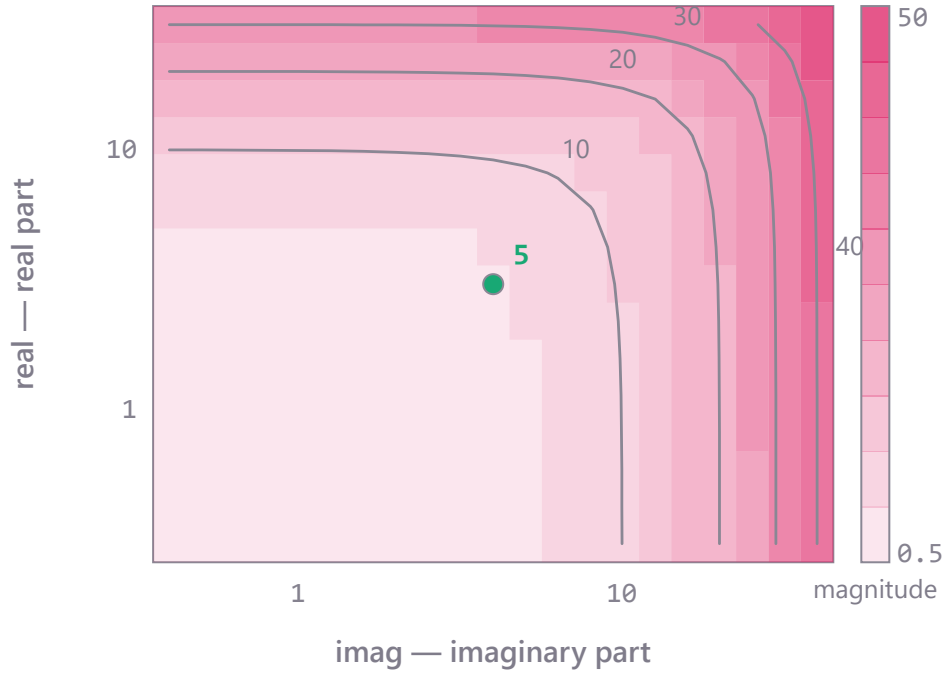
input	meaning	condition
`real`	real part	
`imag`	imaginary part	

output	meaning
`magnitude`	magnitude
`angle_deg`	angle in degrees

- **Method:** M = hypot(x, y), theta = atan2(y, x) (the quadrant kept)
- **Assumes:** not both parts zero
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



magnitude (magnitude) against real (real part) from 0.3 to 30 on a logarithmic axis. One curve for each value of imag (imaginary part): 4, 2, 8. 41 points per curve computed by electrical-engineering-ai999x; marked: the worked example, real = 3, magnitude = 5.



magnitude (magnitude) over imag (imaginary part) from 0.4 to 40 and real (real part) from 0.3 to 30 (logarithmic axes): each of the 21 × 15 cells is one run of electrical-engineering-ai999x, shaded by its value (light 0.5 to dark 50); iso-lines at magnitude = 10, 20, 30, 40. Marked: the worked example, imag = 4, real = 3, magnitude = 5.

```
electrical_engineering_ai999x.solve('rectangular_to_polar', real=3, imag=4)
# -> magnitude = 5, angle_deg = 53.1301   (SOLVED)
```

```
electrical_engineering_ai999x.solve('rectangular_to_polar', real=-2, imag=-2)
# -> magnitude = 2.82843, angle_deg = -135   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Convert 3 + 4j to polar form.	SOLVED	magnitude = 5, angle_deg = 53.1301
Convert -2 - j2 to polar form.	SOLVED	magnitude = 2.82843, angle_deg = -135
Convert 0 + 0j to polar form.	INVALID_INPUT	Phasor from rectangular to polar form: the phasor 0 has no angle.

Sum of sinusoids by phasor addition — `phasor_sum`

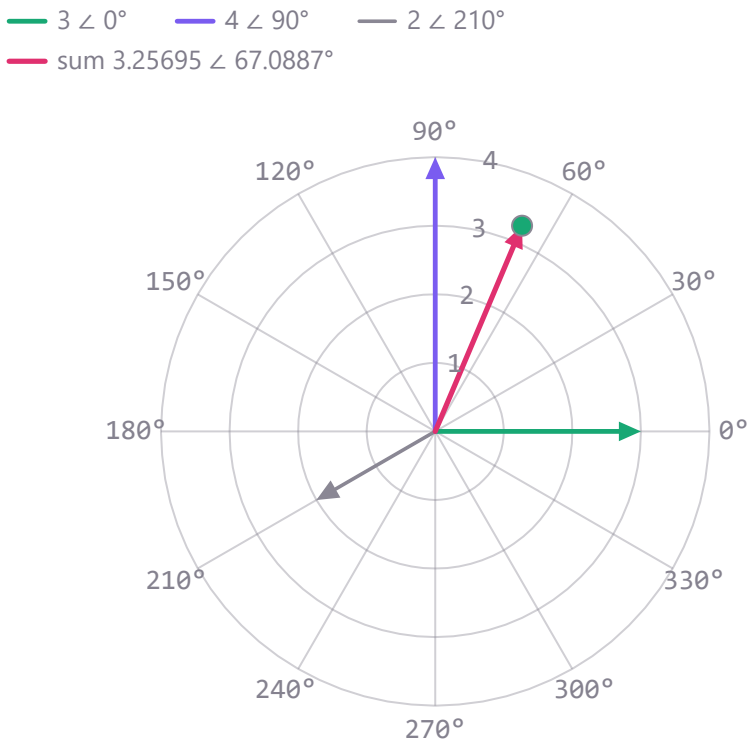
The amplitude and phase of the sum of sinusoids of one frequency, A_k cos(wt + phi_k), by adding their phasors.

$$\sum_k A_k \angle \phi_k = \sum_k A_k e^{j\phi_k}$$

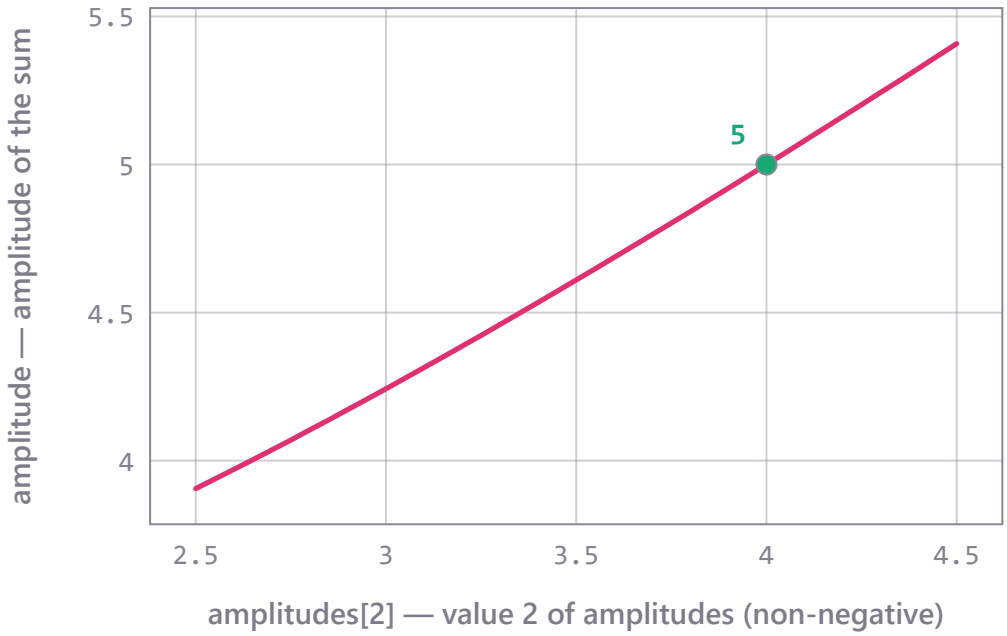
input	meaning	condition
`amplitudes`	amplitudes (non-negative)	
`phases_deg`	phase angles in degrees	

output	meaning
`amplitude`	amplitude of the sum
`phase_deg`	phase of the sum in degrees
`real`	real part of the resultant phasor
`imag`	imaginary part of the resultant phasor

- **Method:** complex addition of the phasors A_k e^{j phi_k}
- **Assumes:** one phase per amplitude; one common frequency
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Three phasors 3∠0°, 4∠90° and 2∠210° on a polar chart (each converted to rectangular form by the package; rings every 1 unit of amplitude) and their sum returned by the package: 3.25695 ∠ 67.0887° = 1.26795 + 3j. Marked: the tip of the sum, real = 1.26795, imaginary = 3.



amplitude (amplitude of the sum) as value 2 of amplitudes (amplitudes (non-negative)) moves from 2.5 to 4.5, the other values as given (3, 4); 41 runs of electrical-engineering-ai999x. Other inputs: phases_deg = 0, 90. Marked: the worked example, amplitudes[2] = 4, amplitude = 5.

```
electrical_engineering_ai999x.solve('phasor_sum', amplitudes='3, 4', phases_deg='0, 90')
# -> amplitude = 5, phase_deg = 53.1301, real = 3, imag = 4   (SOLVED)
```

```
electrical_engineering_ai999x.solve('phasor_sum', amplitudes=[1, 1, 1], phases_deg=[0, 120, -120])
# -> amplitude = 0, phase_deg = 0, real = 0, imag = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the sum of the phasors 3∠0°, 4∠90° and 2∠180°.	SOLVED	amplitude = 4.12311, phase_deg = 75.9638, real = 1, imag = 4
Add the phasors 10∠30° and 5∠-45°.	SOLVED	amplitude = 12.2834, phase_deg = 6.84727, real = 12.1958, imag = 1.46447
Add the phasors -10∠30° and 5∠-45°.	INVALID_INPUT	Sum of sinusoids by phasor addition: amplitudes must not be negative.

Complex, real, reactive and apparent power from voltage and current phasors — `complex_power`

$S = V$ I* for rms phasors: real power P, reactive power Q, apparent power |S|, the power factor and whether it lags or leads.

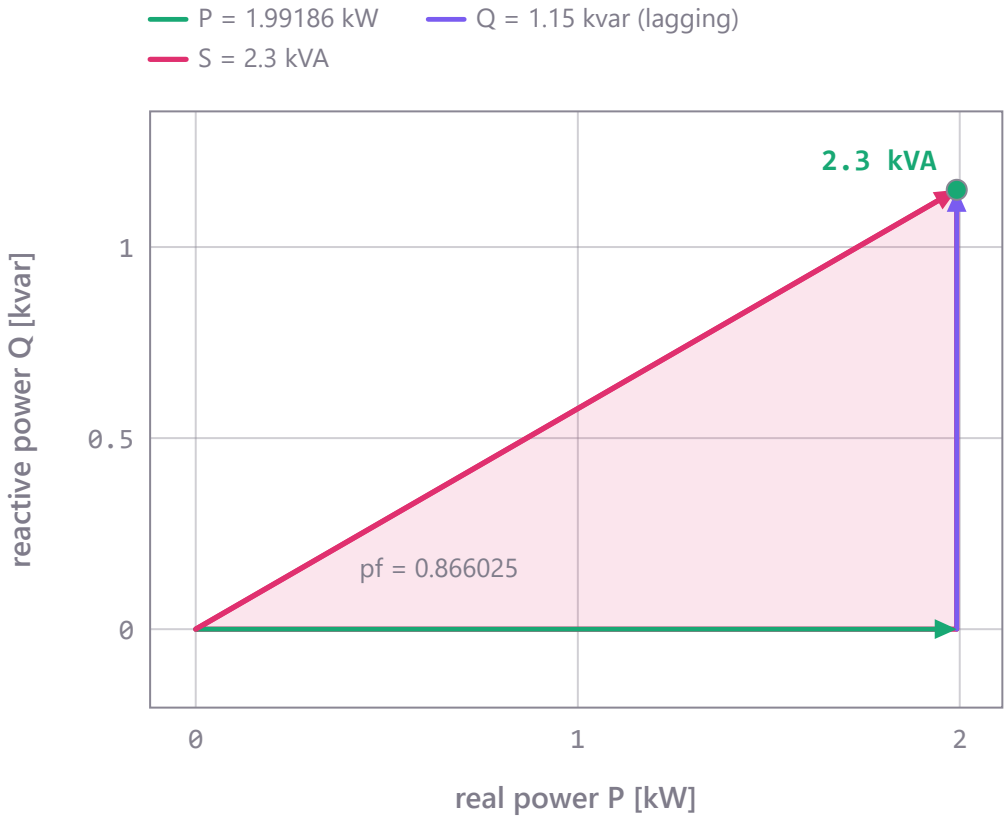
$$S = VI^{*} = P + jQ, \quad \text{pf} = \frac{P}{|S|}$$

input	meaning	condition
V	rms voltage magnitude	

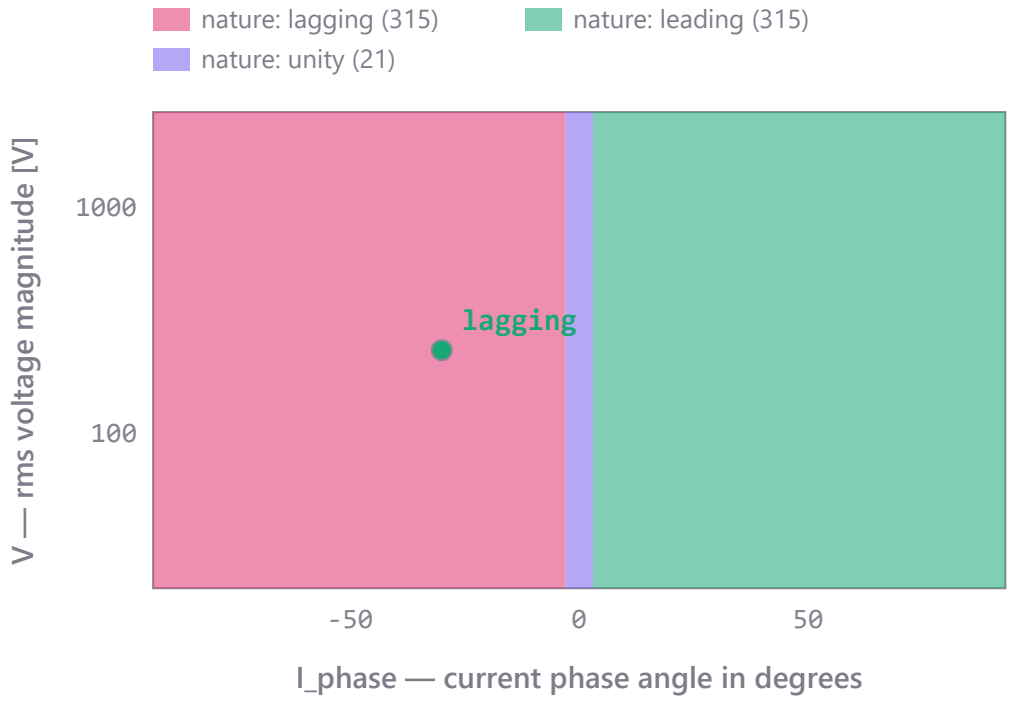
input	meaning	condition
`V_phase`	voltage phase angle in degrees	
`I`	rms current magnitude	
`I_phase`	current phase angle in degrees	

output	meaning
`P`	real (active) power in W
`Q`	reactive power in var (positive: inductive)
`S`	apparent power in VA
`power_factor`	power factor P/|S|
`nature`	lagging (inductive), leading (capacitive) or unity

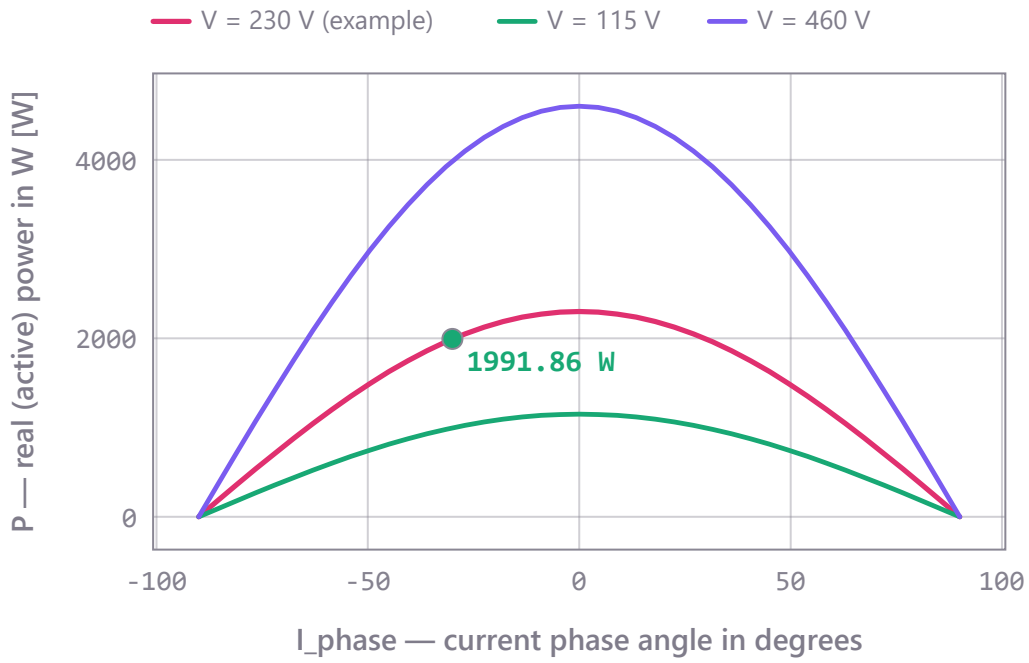
- **Method:** $S = V$ I* (complex conjugate of the current)
- **Assumes:** rms phasors; sinusoidal steady state
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Power triangle of a load with V = 230 V ∠ 0° and I = 10 A ∠ -30°, from the complex power the package returned: real power P = 1991.86 W, reactive power Q = 1150 var (lagging), apparent power S = 2300 VA, power factor 0.866025. Marked: the tip of S.



lagging (inductive), leading (capacitive) or unity (nature) over I_phase (current phase angle in degrees) from -90 to 90 and V (rms voltage magnitude) from 23 V to 2300 V (logarithmic vertical axis): each of the 31 × 21 cells is one run of electrical-engineering-ai999x, coloured by the verdict it returned — lagging: 315 cells, leading: 315 cells, unity: 21 cells. Other inputs as in the worked example: V_phase = 0, I = 10 A. Marked: the worked example, I_phase = -30, V = 230 V → lagging.



P (real (active) power in W) against I_phase (current phase angle in degrees) from -90 to 90. One curve for each value of V (rms voltage magnitude): 230 V, 115 V, 460 V. Other inputs as in the worked example: V_phase = 0, I = 10 A. 41 points per curve computed by electrical-engineering-ai999x; marked: the worked example, I_phase = -30, P = 1991.86 W.

```
electrical_engineering_ai999x.solve('complex_power', V=230, V_phase=0, I=10, I_phase=-30)
# -> P = 1991.86 W, Q = 1150 W, S = 2300 W, power_factor = 0.866025, nature = lagging   (SOLVED)
```

```
electrical_engineering_ai999x.solve('complex_power', V=120, V_phase=10, I=5, I_phase=40)
# -> P = 519.615 W, Q = -300 W, S = 600 W, power_factor = 0.866025, nature = leading   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the complex power for V = 230∠0° V and I = 10∠-30° A.	SOLVED	P = 1991.86 W, Q = 1150 W, S = 2300 W, power_factor = 0.866025, nature = lagging
Find the real power for V = 120∠10° V and I = 5∠40° A.	SOLVED	P = 519.615 W, Q = -300 W, S = 600 W, power_factor = 0.866025, nature = leading
Find the complex power for V = 230∠0° V.	NEEDS_INPUT	complex_power needs I, I_phase (rms current magnitude; current phase angle in degrees). Example: solve('complex_power',

Power-factor correction capacitor — `power_factor_correction`

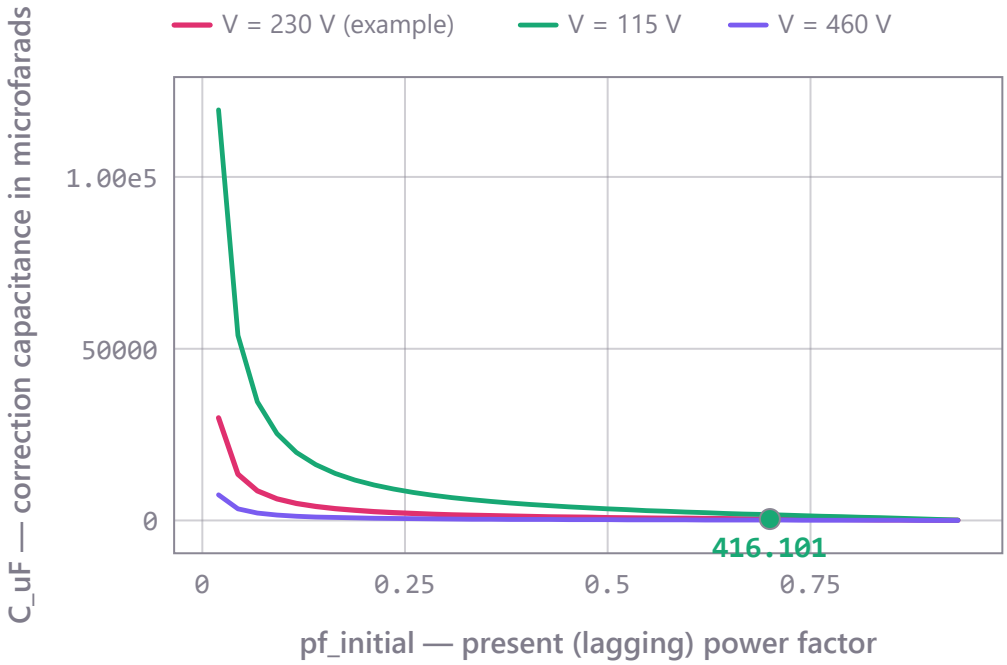
The shunt capacitance (and its reactive power) that raises a lagging load's power factor to a target value.

$$C = \frac{P(\tan\phi_1 - \tan\phi_2)}{2\pi f V^2}$$

input	meaning	condition
`P`	real power of the load	
`V`	rms supply voltage across the capacitor	
`f`	supply frequency	
`pf_initial`	present (lagging) power factor	
`pf_target`	target (lagging) power factor	

output	meaning
`C`	correction capacitance
`C_uF`	correction capacitance in microfarads
`Qc`	reactive power the capacitor supplies, in var
`Q_initial`	reactive power before correction, in var
`Q_final`	reactive power after correction, in var

- **Method:** $Qc = P$ (tan acos $pf1 - tan$ acos pf2), C = Qc / (2 pi f V^2)
- **Assumes:** single-phase, sinusoidal; both power factors lagging; target > present
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C_uF (correction capacitance in microfarads) against pf_initial (present (lagging) power factor) from 0.02 to 0.98. One curve for each value of V (rms supply voltage across the capacitor): 230 V, 115 V, 460 V. Other inputs as in the worked example: P = 10000 W, f = 50 Hz, pf_target = 0.95. 39 points per curve computed by electrical-engineering-ai999x; marked: the worked example, pf_initial = 0.7, C_uF = 416.101.

```
electrical_engineering_ai999x.solve('power_factor_correction', P=10000, V=230, f=50,
pf_initial=0.7, pf_target=0.95)
# -> C = 0.000416101 F, C_uF = 416.101, Qc = 6915.2 W, Q_initial = 10202 W, Q_final = 3286.84 W
(SOLVED)
```

```
electrical_engineering_ai999x.solve('power_factor_correction', P=1000, V=120, f=60, pf_initial=0.6,
pf_target=1.0)
# -> C = 0.000245609 F, C_uF = 245.609, Qc = 1333.33 W, Q_initial = 1333.33 W, Q_final = 0 W
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
A load draws 10 kW at a power factor of 0.7 from a 230 V, 50 Hz supply. What capacitance corrects the power factor to 0.95?	SOLVED	C = 0.000416101 F, C_uF = 416.101, Qc = 6915.2 W, Q_initial = 10202 W, Q_final = 3286.84 W
A motor uses 1 kW at a power factor of 0.6 on a 120 V, 60 Hz line. Find the capacitor that improves the power factor to 1.	SOLVED	C = 0.000245609 F, C_uF = 245.609, Qc = 1333.33 W, Q_initial = 1333.33 W, Q_final = 0 W
A load draws 10 kW at a power factor of 0.9 from a 230 V, 50 Hz supply. What capacitance corrects the power factor to 0.8?	INVALID_INPUT	Power-factor correction capacitor: the target power factor must be higher than the present one.

Balanced three-phase load (wye or delta) — `three_phase_load`

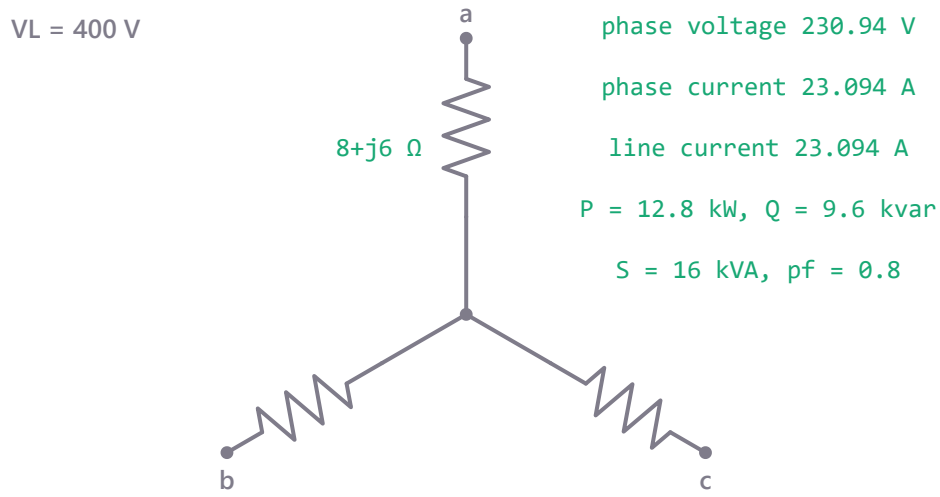
Phase voltage, phase and line currents, real, reactive and apparent power and the power factor of a balanced load with impedance R + jX per phase.

$$P = \sqrt{3}\, V_L I_L \cos\phi$$

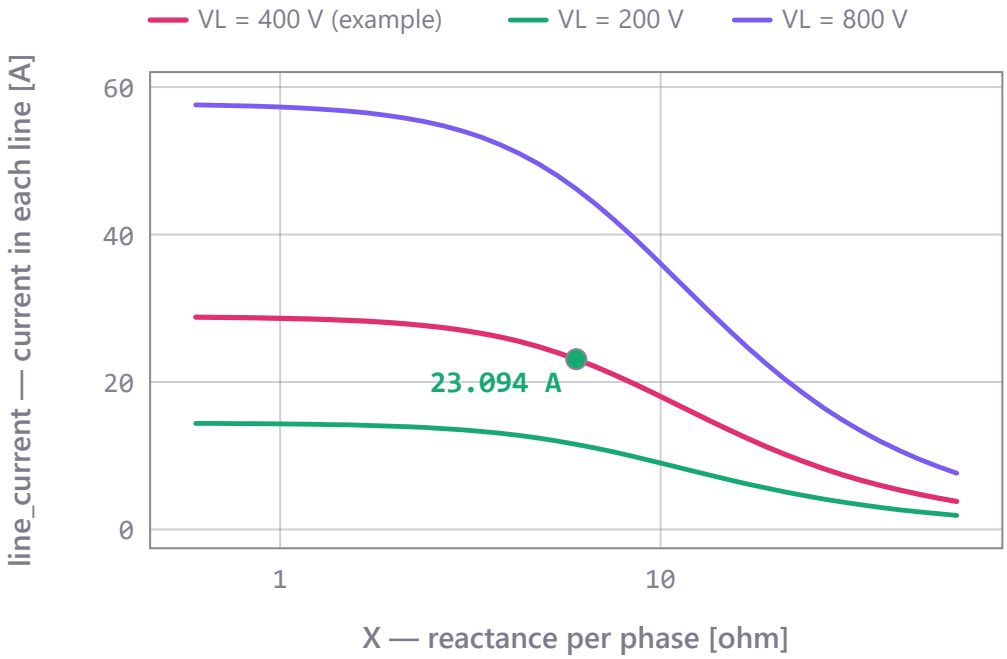
input	meaning	condition
`VL`	rms line-to-line voltage	
`R`	resistance per phase	
`X`	reactance per phase (positive inductive, negative capacitive)	
`connection`	load connection	

output	meaning
`phase_voltage`	voltage across each phase of the load
`phase_current`	current in each phase of the load
`line_current`	current in each line
`P`	total real power in W
`Q`	total reactive power in var
`S`	total apparent power in VA
`power_factor`	power factor R/|Z|

- **Method:** wye: $V_{ph} = \frac{V_L}{\sqrt{3}}$, $I_L = I_p h$; delta: $V_p h = V_L$, $I_L = sqrt(3)$ I_ph; $P = 3\, I_p h^2$ R
- **Assumes:** balanced load and supply; $R \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Balanced three-phase wye load as stated: line voltage VL = 400 V, each phase R = 8 Ω and X = 6 Ω. Labelled with what the package returned: phase voltage 230.94 V; phase current 23.094 A; line current 23.094 A; P = 12.8 kW, Q = 9.6 kvar; S = 16 kVA, pf = 0.8. Marked: line current = 23.094 A.



line_current (current in each line) against X (reactance per phase (positive inductive, negative capacitive)) from 0.6 ohm to 60 ohm on a logarithmic axis. One curve for each value of VL (rms line-to-line voltage): 400 V, 200 V, 800 V. Other inputs as in the worked example: R = 8 ohm, connection = wye. 41 points per curve computed by electrical-engineering-ai999x; marked: the worked example, X = 6 ohm, line_current = 23.094 A.

```
electrical_engineering_ai999x.solve('three_phase_load', VL=400, R=8, X=6, connection='wye')
# -> phase_voltage = 230.94 V, phase_current = 23.094 A, line_current = 23.094 A, P = 12800 W, Q =
9600 W, S = 16000 W, power_factor = 0.8   (SOLVED)
```

```
electrical_engineering_ai999x.solve('three_phase_load', VL=400, R=8, X=6, connection='delta')
# -> phase_voltage = 400 V, phase_current = 40 A, line_current = 69.282 A, P = 38400 W, Q = 28800
W, S = 48000 W, power_factor = 0.8   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
A balanced wye-connected load has R = 8 ohm and X = 6 ohm per phase on a 400 V three-phase supply. Find the line current.	SOLVED	phase_voltage = 230.94 V, phase_current = 23.094 A, line_current = 23.094 A, P = 12800 W, Q = 9600 W, S = 16000 W, power_factor = 0.8
A balanced delta-connected three-phase load with R = 8 ohm and X = 6 ohm per phase is fed at 400 V. Find the total power.	SOLVED	phase_voltage = 400 V, phase_current = 40 A, line_current = 69.282 A, P = 38400 W, Q = 28800 W, S = 48000 W, power_factor = 0.8
A balanced three-phase load with R = 8 ohm and X = 6 ohm per phase is fed at 400 V. Find the line current.	NEEDS_INPUT	three_phase_load needs connection (load connection). Example: solve('three_phase_load', VL='400 V', R='8 ohm', X='6 ohm'

Inverting summing amplifier output — `summing_amplifier`

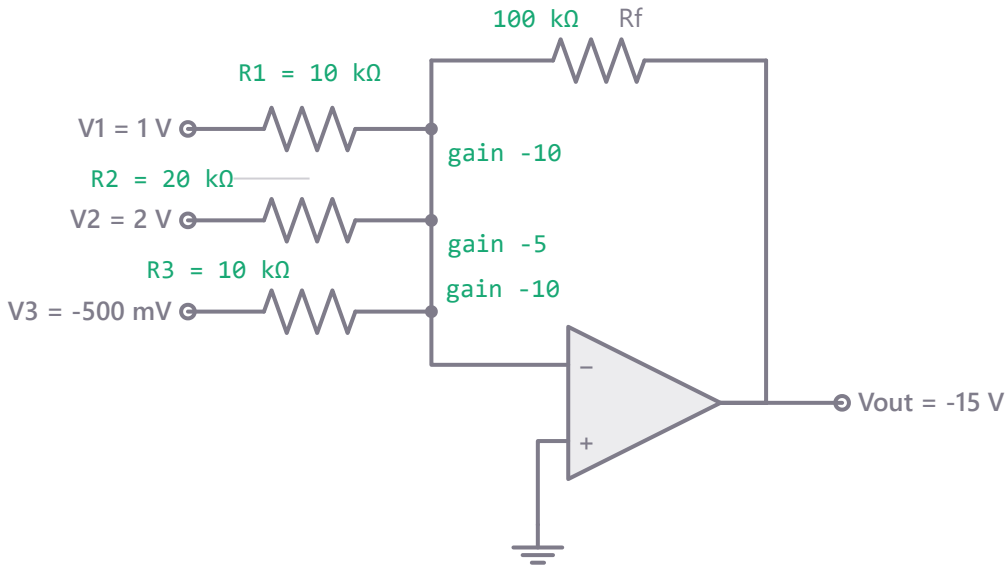
The output of an ideal inverting summing amplifier: Vout = -Rf $(V_1/R_1 + V_2/R_2 + \dots)$.

$$V_{out} = -R_f \sum_k \frac{V_k}{R_k}$$

input	meaning	condition
`V`	input voltages in volts	
`R`	input resistors in ohms (one per input)	
`Rf`	feedback resistor	

output	meaning
`Vout`	output voltage
`gains`	gain of each input, -Rf/R_k

- **Method:** virtual ground at the inverting input; Kirchhoff's current law there
- **Assumes:** ideal op-amp (infinite gain, no saturation); non-inverting input grounded
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Inverting summing amplifier as stated: inputs V1 = 1 V through R1 = 10 kΩ, V2 = 2 V through R2 = 20 kΩ, V3 = -0.5 V through R3 = 10 kΩ, feedback Rf = 100 kΩ. Labelled with what the package returned: the gain of each input (-10, -5, -10) and Vout = -15 V. Marked: Vout = -15 V.

```
electrical_engineering_ai999x.solve('summing_amplifier', V='1, 2, -0.5', R='10000, 20000, 10000',
Rf=100000)
# -> Vout = -15 V, gains = [-10.0, -5.0, -10.0]   (SOLVED)
```

```
electrical_engineering_ai999x.solve('summing_amplifier', V=[0.2, 0.3], R=[1000, 1000], Rf=10000)
# -> Vout = -5 V, gains = [-10.0, -10.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
An inverting summing amplifier has inputs of 1 V, 2 V and -0.5 V through input resistors of 10 kohm, 20 kohm and 10 kohm, and a feedback resistor of 100 kohm. Find the output voltage.	SOLVED	Vout = -15 V, gains = [-10.0, -5.0, -10.0]
A summing amplifier has inputs of 0.2 V and 0.3 V through input resistors of 1 kohm and 1 kohm, and a feedback resistor of 10 kohm. Find the output voltage.	SOLVED	Vout = -5 V, gains = [-10.0, -10.0]

question	outcome	result
A summing amplifier has inputs of 0.2 V and 0.3 V through input resistors of 1 kohm, 1 kohm and 2 kohm, and a feedback resistor of 10 kohm. Find the output voltage.	INVALID_INPUT	Inverting summing amplifier output: give one input resistor for every input voltage.

Difference amplifier output (four-resistor op-amp circuit) — `difference_amplifier`

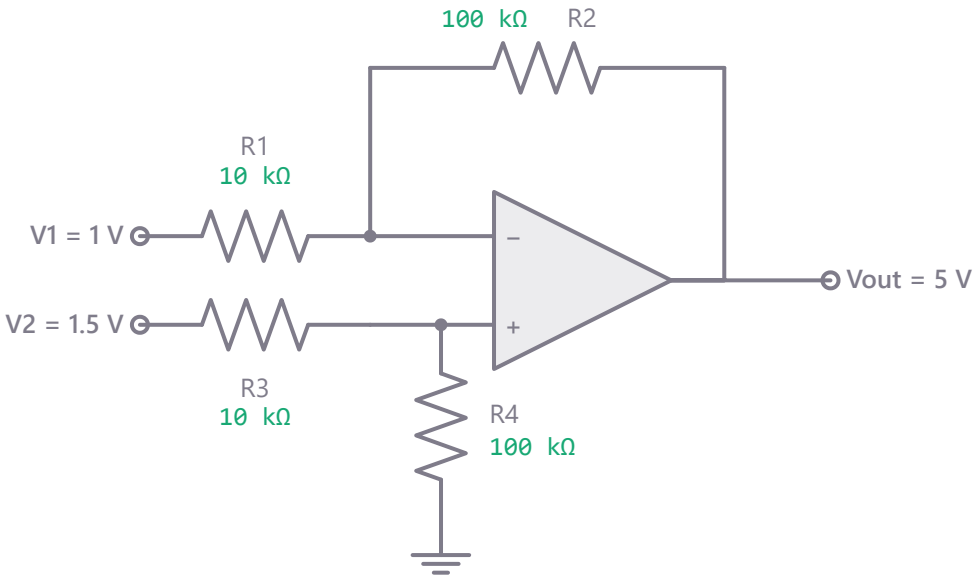
The output of the classic difference amplifier: V1 through R1 to the inverting input, R2 from output to that input, V2 through R3 to the non-inverting input, R4 from it to ground.

$$V_{out} = \frac{R_4}{R_3 + R_4}\left(1 + \frac{R_2}{R_1}\right)V_2 - \frac{R_2}{R_1}V_1$$

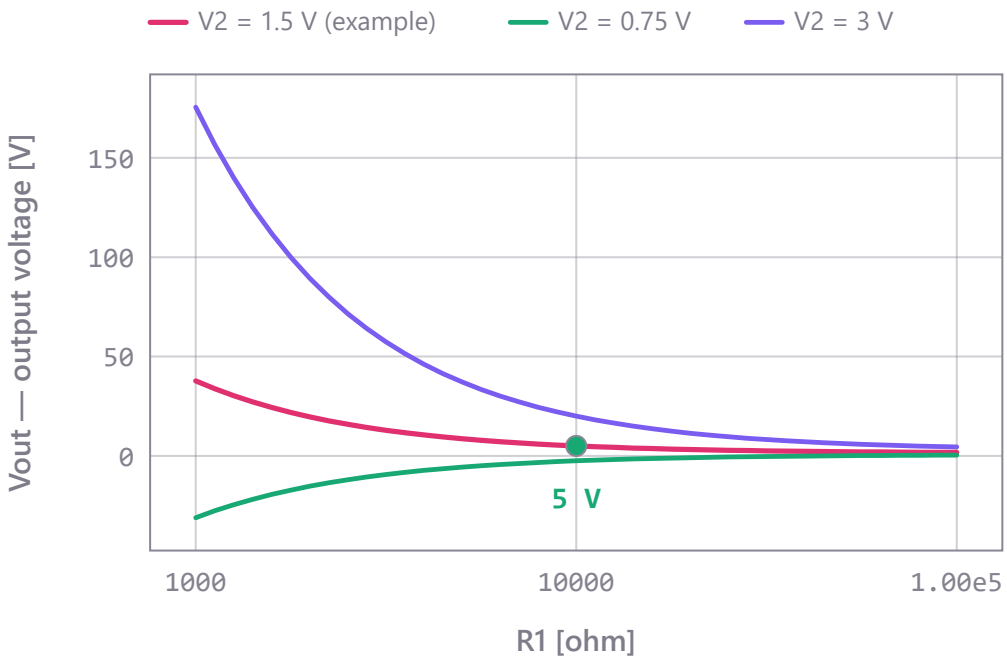
input	meaning	condition
`V1`	voltage at the inverting side	
`V2`	voltage at the non-inverting side	
`R1`	input resistor to the inverting input	
`R2`	feedback resistor	
`R3`	input resistor to the non-inverting input	
`R4`	resistor from the non-inverting input to ground	

output	meaning
`Vout`	output voltage
`gain_inverting`	gain from V1, $-R_2/R_1$
`gain_noninverting`	gain from V2, $R_4/(R_3+R_4)\,(1+R_2/R_1)$

- **Method:** superposition with v+ = v-
- **Assumes:** ideal op-amp (infinite gain, no saturation)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Difference amplifier as stated: V1 = 1 V through R1 = 10 kΩ to the − input with feedback R2 = 100 kΩ; V2 = 1.5 V through R3 = 10 kΩ to the + input with R4 = 100 kΩ to ground. Labelled with what the package returned: Vout = 5 V (gains -10 and 10). Marked: Vout = 5 V.



Vout (output voltage) against R1 (input resistor to the inverting input) from 1000 ohm to 100000 ohm on a logarithmic axis. One curve for each value of V2 (voltage at the non-inverting side): 1.5 V, 0.75 V, 3 V. Other inputs as in the worked example: V1 = 1 V, R2 = 100000.0 ohm, R3 = 10000.0 ohm, R4 = 100000.0 ohm. 41 points per curve computed by electrical-engineering-ai999x; marked: the worked example, R1 = 10000 ohm, Vout = 5 V.

```
electrical_engineering_ai999x.solve('difference_amplifier', V1=1, V2=1.5, R1=10000.0, R2=100000.0,
R3=10000.0, R4=100000.0)
# -> Vout = 5 V, gain_inverting = -10, gain_noninverting = 10   (SOLVED)
```

```
electrical_engineering_ai999x.solve('difference_amplifier', V1=2, V2=1, R1=1000.0, R2=2000.0,
R3=1000.0, R4=1000.0)
# -> Vout = -2.5 V, gain_inverting = -2, gain_noninverting = 1.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
A difference amplifier has R1 = 10 kohm, R2 = 100 kohm, R3 = 10 kohm, R4 = 100 kohm, V1 = 1 V and V2 = 1.5 V. Find the output voltage.	SOLVED	Vout = 5 V, gain_inverting = -10, gain_noninverting = 10
A difference amplifier has R1 = 10 kohm, R2 = 100 kohm, R3 = 10 kohm, V1 = 1 V and V2 = 1.5 V. Find the output voltage.	NEEDS_INPUT	difference_amplifier needs R4 (resistor from the non-inverting input to ground). Example: solve('difference_amplifier',

DC operating point of a voltage-divider biased BJT — `bjt_divider_bias`

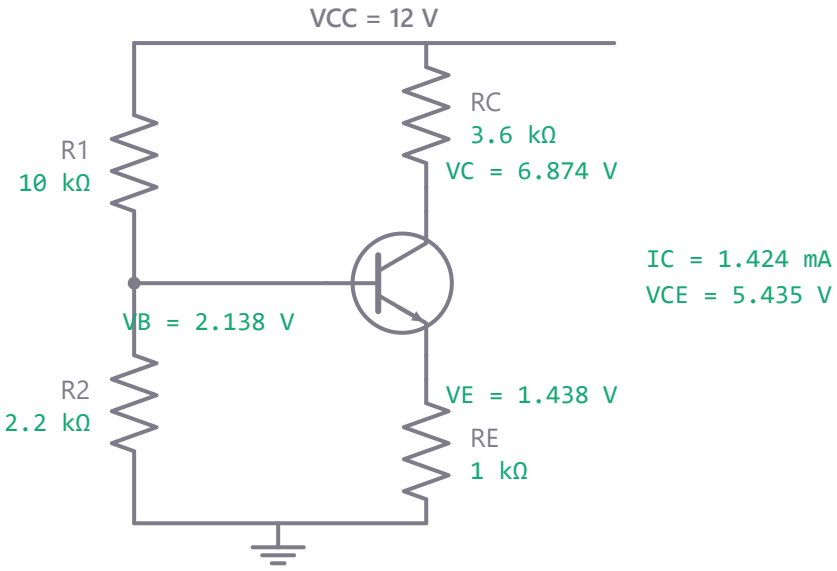
Base, collector and emitter currents and the node voltages of an NPN transistor with divider bias (R1 to VCC, R2 to ground), collector resistor RC and emitter resistor RE.

$$I_B = \frac{V_{TH} - V_{BE}}{R_{TH} + (\beta + 1) R_E}, \quad V_{TH} = V_{CC} \frac{R_2}{R_1 + R_2}$$

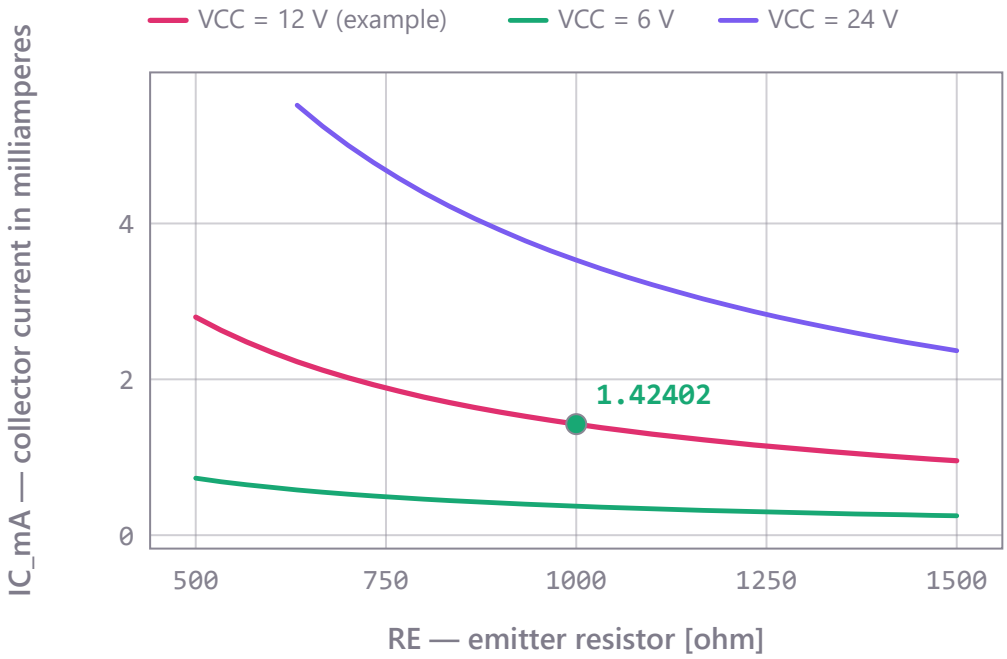
input	meaning	condition
`VCC`	supply voltage	
`R1`	divider resistor from VCC to the base	
`R2`	divider resistor from the base to ground	
`RC`	collector resistor	
`RE`	emitter resistor	
`beta`	DC current gain	
`VBE`	base-emitter voltage in conduction (e.g. 0.7 V)	

output	meaning
`IC`	collector current
`IC_mA`	collector current in milliamperes
`IB`	base current
`IE`	emitter current
`VB`	base voltage
`VE`	emitter voltage
`VC`	collector voltage
`VCE`	collector-emitter voltage

- **Method:** Thevenin equivalent of the divider, then IB = (VTH - VBE)/(RTH + (beta+1) RE) (the exact analysis, base current included)
- **Assumes:** NPN in the forward-active region (VCE >= 0.2 V); constant VBE and beta
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Voltage-divider bias of an npn transistor as stated: VCC = 12 V, R1 = 10 kΩ, R2 = 2.2 kΩ, RC = 3.6 kΩ, RE = 1 kΩ, β = 100, VBE = 0.7 V. Labelled with the operating point the package returned: VB = 2.13826 V, VE = 1.43826 V, VC = 6.87354 V, IC = 0.00142402 A, VCE = 5.43529 V. Marked: IC = 1.42402 mA.



IC_mA (collector current in milliamperes) against RE (emitter resistor) from 500 ohm to 1500 ohm. One curve for each value of VCC (supply voltage): 12 V, 6 V, 24 V. Other inputs as in the worked example: R1 = 10000.0 ohm, R2 = 2200 ohm, RC = 3600 ohm, beta = 100, VBE = 0.7 V. 31 points per curve computed by electrical-engineering-ai999x; marked: the worked example, RE = 1000 ohm, IC_mA = 1.42402.

```
electrical_engineering_ai999x.solve('bjt_divider_bias', VCC=12, R1=10000.0, R2=2200, RC=3600,
RE=1000, beta=100, VBE=0.7)
# -> IC = 0.00142402 A, IC_mA = 1.42402, IB = 1.42402e-05 A, IE = 0.00143826 A, VB = 2.13826 V, VE
= 1.43826 V, VC = 6.87354 V, VCE = 5.43529 V   (SOLVED)
```

```
electrical_engineering_ai999x.solve('bjt_divider_bias', VCC=10, R1=10000.0, R2=10000.0, RC=1000,
RE=1000, beta=99, VBE=0.7)
# -> IC = 0.00405429 A, IC_mA = 4.05429, IB = 4.09524e-05 A, IE = 0.00409524 A, VB = 4.79524 V, VE
= 4.09524 V, VC = 5.94571 V, VCE = 1.85048 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
A voltage-divider biased BJT has VCC = 12 V, R1 = 10 kohm, R2 = 2.2 kohm, RC = 3.6 kohm, RE = 1 kohm, beta = 100 and VBE = 0.7 V. Find the collector current.	SOLVED	IC = 0.00142402 A, IC_mA = 1.42402, IB = 1.42402e-05 A, IE = 0.00143826 A, VB = 2.13826 V, VE = 1.43826 V, VC = 6.87354 V, VCE = 5.43529 V
A voltage-divider biased BJT has VCC = 12 V, R1 = 1 kohm, R2 = 10 kohm, RC = 10 kohm, RE = 100 ohm, beta = 200 and VBE = 0.7 V. Find the collector current.	INVALID_INPUT	DC operating point of a voltage-divider biased BJT: VCE would be below 0.2 V: the transistor is saturated and the active
A voltage-divider biased BJT has VCC = 12 V, R1 = 10 kohm, R2 = 2.2 kohm, RC = 3.6 kohm, RE = 1 kohm and VBE = 0.7 V. Find the collector current.	NEEDS_INPUT	bjt_divider_bias needs beta (DC current gain). Example: solve('bjt_divider_bias', VCC='12 V', R1='10000 ohm', R2='2200 o

Zener diode shunt regulator — `zener_regulator`

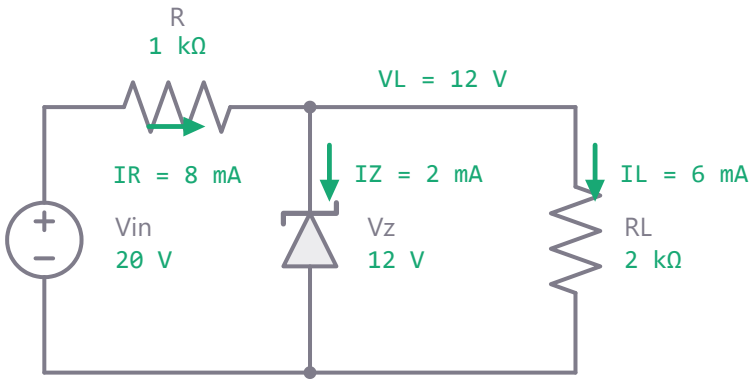
Load voltage and the currents of a Zener shunt regulator (series resistor R from the input, Zener and load in parallel).

$$I_Z = \frac{V_{in} - V_Z}{R} - \frac{V_Z}{R_L}$$

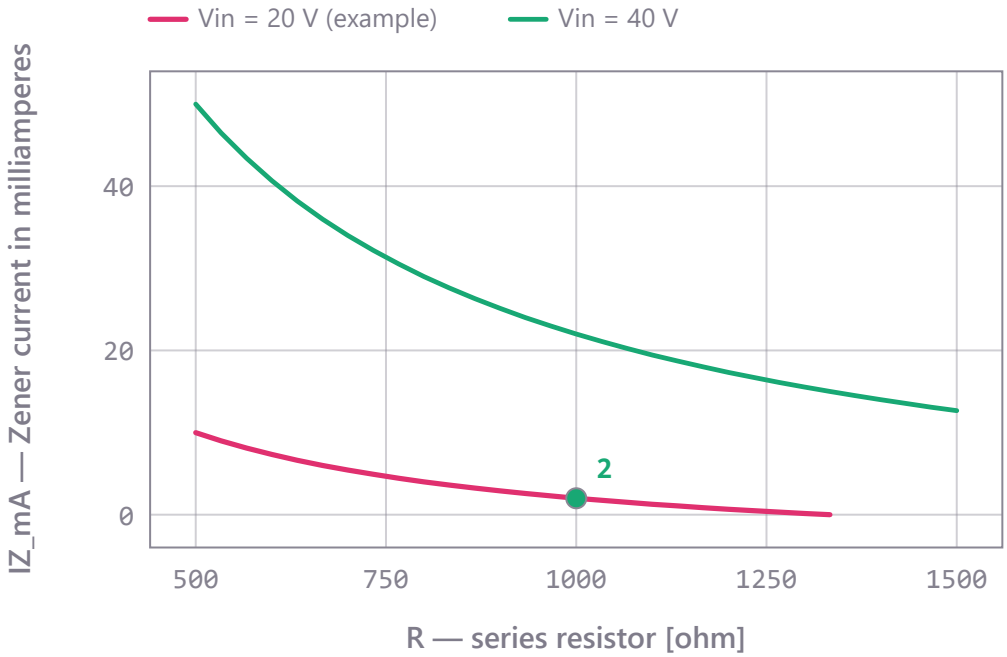
input	meaning	condition
`Vin`	input voltage	
`R`	series resistor	
`Vz`	Zener (breakdown) voltage	
`RL`	load resistance	

output	meaning
`VL`	load voltage
`IR`	current in the series resistor
`IL`	load current
`IZ`	Zener current
`IZ_mA`	Zener current in milliamperes
`PZ`	power dissipated in the Zener

- **Method:** I_R = (Vin - Vz)/R, $I_L = Vz/RL$, $I_Z = I_R - I_L$
- **Assumes:** the Zener is in breakdown (ideal: constant Vz); positive values
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Zener shunt regulator as stated: Vin = 20 V, series R = 1 kΩ, Zener voltage Vz = 12 V, load RL = 2 kΩ. Labelled with what the package returned: VL = 12 V, IR = 0.008 A, IZ = 0.002 A, IL = 0.006 A (Zener power 0.024 W). Marked: VL = 12 V.



IZ_mA (Zener current in milliamperes) against R (series resistor) from 500 ohm to 1500 ohm. One curve for each value of Vin (input voltage): 20 V, 40 V. Other inputs as in the worked example: Vz = 12 V, RL = 2000 ohm. 26 points per curve computed by electrical-engineering-ai999x; marked: the worked example, R = 1000 ohm, IZ_mA = 2.

```
electrical_engineering_ai999x.solve('zener_regulator', Vin=20, R=1000, Vz=12, RL=2000)
# -> VL = 12 V, IR = 0.008 A, IL = 0.006 A, IZ = 0.002 A, IZ_mA = 2, PZ = 0.024 W   (SOLVED)
```

```
electrical_engineering_ai999x.solve('zener_regulator', Vin=12, R=220, Vz=5.1, RL=1000)
# -> VL = 5.1 V, IR = 0.0313636 A, IL = 0.0051 A, IZ = 0.0262636 A, IZ_mA = 26.2636, PZ = 0.133945
W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
A Zener regulator has an input voltage of 20 V, a series resistor of 1 kohm, a Zener voltage of 12 V and a load resistance of 2 kohm. Find the Zener current.	SOLVED	VL = 12 V, IR = 0.008 A, IL = 0.006 A, IZ = 0.002 A, IZ_mA = 2, PZ = 0.024 W
A Zener regulator has an input voltage of 15 V, a series resistor of 1 kohm, a Zener voltage of 12 V and a load resistance of 1 kohm. Find the Zener current.	INVALID_INPUT	Zener diode shunt regulator: without the Zener the load voltage would not reach Vz: the diode is not in breakdown (no re
A Zener regulator has an input voltage of 20 V, a Zener voltage of 12 V and a load resistance of 2 kohm. Find the Zener current.	NEEDS_INPUT	zener_regulator needs R (series resistor). Example: solve('zener_regulator', Vin='20 V', R='1000 ohm', Vz='12 V', RL='20

Step response of a series RLC circuit — `series_rlc_step`

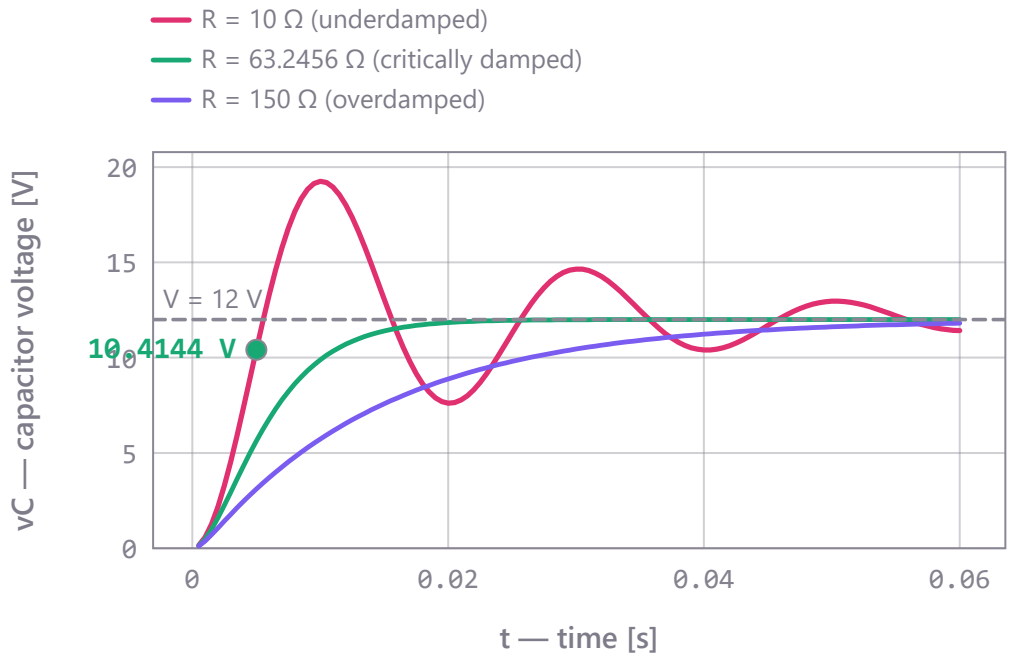
Capacitor voltage and loop current at time t after a DC step V is applied to a series RLC circuit at rest, with the damping classification.

$$LC\,\ddot{v}_C + RC\,\dot{v}_C + v_C = V, \quad \alpha = \frac{R}{2L},\ \omega_0 = \frac{1}{\sqrt{LC}}$$

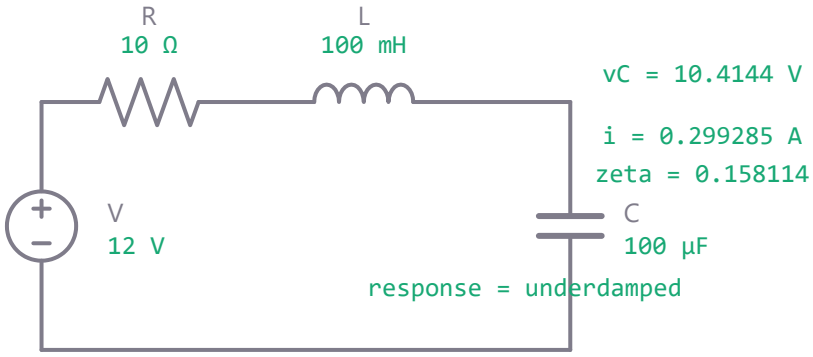
input	meaning	condition
`R`	resistance	
`L`	inductance	
`C`	capacitance	
`V`	step voltage applied at $t = 0$	
`t`	time after the step	

output	meaning
`vC`	capacitor voltage at t
`i`	loop current at t
`i_mA`	loop current in milliamperes
`alpha`	neper frequency $R/(2L)$ in $1/s$
`omega0`	undamped resonant frequency $1/sqrt$(LC)
`zeta`	damping ratio $\alpha/omega0$
`response`	underdamped, critically damped or overdamped

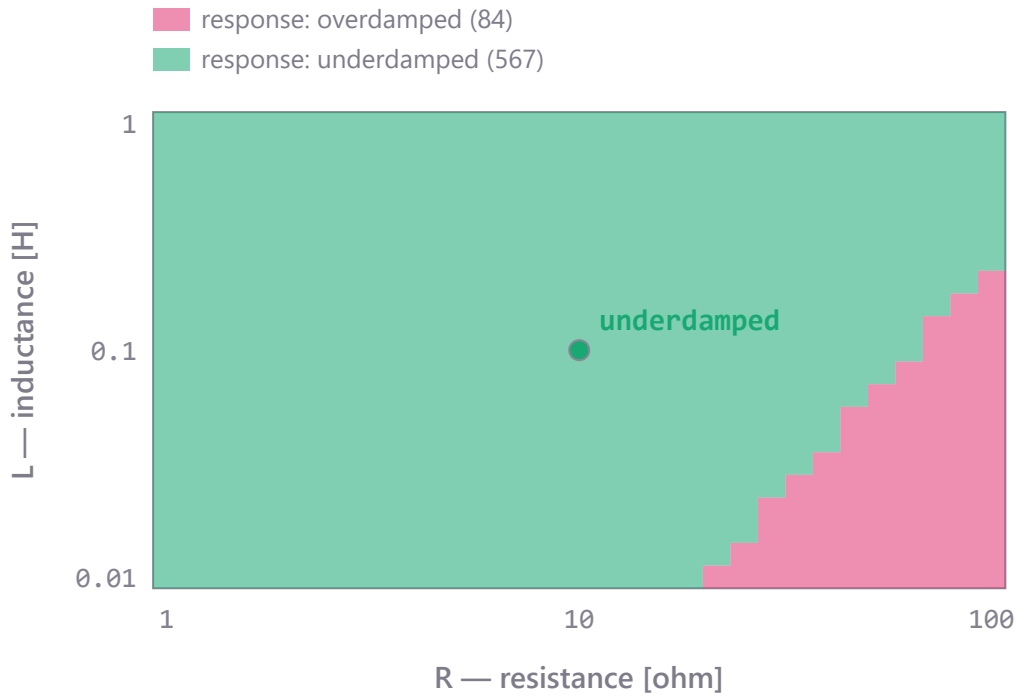
- **Method:** closed-form solution of L C vC'' + R C vC' + $vC = V$ by the roots of the characteristic equation
- **Assumes:** circuit at rest before the step (vC = 0, i = 0); ideal components
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Capacitor voltage of a series RLC circuit switched onto V = 12 V (L = 0.1 H, C = 0.0001 F) for three resistances, 121 instants each computed by the package: R = 10 Ω: ζ = 0.158114, underdamped; R = 63.2456 Ω: ζ = 1, critically damped; R = 150 Ω: ζ = 2.37171, overdamped. Marked: t = 0.005 s, vC = 10.4144 V (R = 10 Ω).



The series RLC circuit of the worked example (R = 10, L = 0.1, C = 0.0001, V = 12, t = 0.005), labelled with what the package returned: vC = 10.4144; i = 0.299285; zeta = 0.158114; response = underdamped. Marked: vC = 10.4144.



underdamped, critically damped or overdamped (response) over R (resistance) from 1 ohm to 100 ohm and L (inductance) from 0.01 H to 1 H (logarithmic axes): each of the 31 × 21 cells is one run of electrical-engineering-ai999x, coloured by the verdict it returned — overdamped: 84 cells, underdamped: 567 cells. Other inputs as in the worked example: C = 0.0001 F, V = 12 V, t = 0.005 s. Marked: the worked example, R = 10 ohm, L = 0.1 H → underdamped.

```
electrical_engineering_ai999x.solve('series_rlc_step', R=10, L=0.1, C=0.0001, V=12, t=0.005)
# -> vC = 10.4144 V, i = 0.299285 A, i_mA = 299.285, alpha = 50, omega0 = 316.228 rad/s, zeta =
0.158114, response = underdamped   (SOLVED)
```

```
electrical_engineering_ai999x.solve('series_rlc_step', R=1000, L=0.1, C=1e-06, V=5, t=0.001)
# -> vC = 3.14441 V, i = 0.0020905 A, i_mA = 2.0905, alpha = 5000, omega0 = 3162.28 rad/s, zeta =
1.58114, response = overdamped   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
A series RLC circuit with R = 10 ohm, L = 0.1 H and C = 100 uF is connected to a 12 V step. Find the capacitor voltage at t = 5 ms.	SOLVED	vC = 10.4144 V, i = 0.299285 A, i_mA = 299.285, alpha = 50, omega0 = 316.228 rad/s, zeta = 0.158114, response = underdamped
A series RLC circuit with R = 10 ohm, L = 0.1 H and C = 100 uF is connected to a 12 V step. Find the capacitor voltage.	NEEDS_INPUT	series_rlc_step needs t (time after the step). Example: solve('series_rlc_step', R='10 ohm', L='0.1 H', C='0.0001 F', V=
A series RLC circuit with R = 0 ohm, L = 0.1 H and C = 100 uF is connected to a 12 V step. Find the capacitor voltage at t = 5 ms.	INVALID_INPUT	Step response of a series RLC circuit: R must be positive.

Evaluate a Boolean expression — `boolean_evaluate`

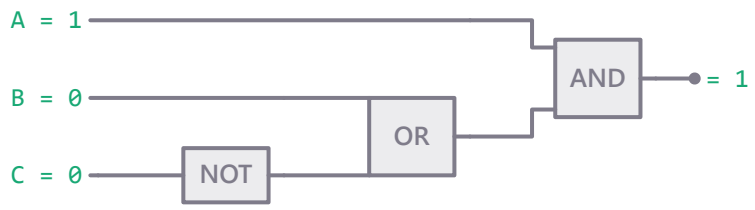
The value (0 or 1) of a Boolean expression for given values of its variables.

$$F(A, B, \ldots) \in \{0, 1\}$$

input	meaning	condition
`expression`	the expression: AND (&, ·, *), OR (|, +), XOR (^), NOT (~, !, NOT, or a postfix '), parentheses; NOT > AND > XOR > OR	
`values`	the variables' values, e.g. '$A = 1$, $B = 0$, $C = 1$'	

output	meaning
`value`	value of the expression (0 or 1)

- **Method:** recursive-descent parse with NOT > AND > XOR > OR, then evaluation
- **Assumes:** every variable given 0 or 1; products written with an operator (A·B, not AB)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The expression A AND (B OR NOT C) drawn as gates, inputs as stated (A=1, B=0, C=0); the output is the value the package returned, 1. Marked: output = 1.

```
electrical_engineering_ai999x.solve('boolean_evaluate', expression='A AND (B OR NOT C)',
values='A=1, B=0, C=0')
# -> value = 1   (SOLVED)
```

```
electrical_engineering_ai999x.solve('boolean_evaluate', expression="A·B' + C", values='A=1, B=1,
C=0')
# -> value = 0   (SOLVED)
```

```
electrical_engineering_ai999x.solve('boolean_evaluate', expression='A + B · C', values='A=0, B=1,
C=0')
# -> value = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Evaluate A AND (B OR NOT C) for A=1, B=0, C=0.	SOLVED	value = 1
Evaluate A·B' + C for A=1, B=0, C=0.	SOLVED	value = 1
Evaluate (A XOR B)' for A=1, B=1.	SOLVED	value = 1
Evaluate A AND B for A=1.	INVALID_INPUT	Evaluate a Boolean expression: give a value for B.
Evaluate A AND B for A=1, B=2.	INVALID_INPUT	Evaluate a Boolean expression: B must be 0 or 1.

Truth table and minimal sum of products of a Boolean expression — `truth_table_sop`

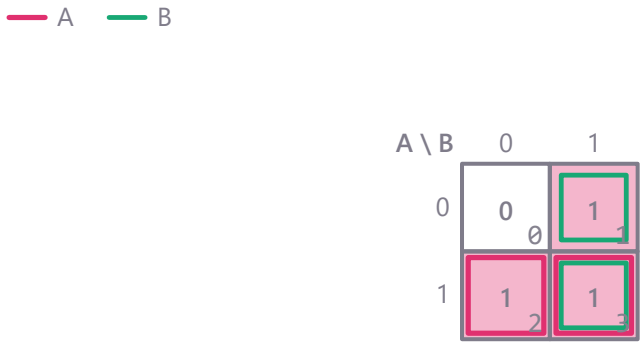
The minterms (truth table), the canonical sum of products and a minimal sum of products (fewest terms, then fewest literals) of an expression in up to 4 variables.

$$F = \sum m(\ldots)$$

input	meaning	condition
`expression`	the expression: AND (&, ·, *), OR (|, +), XOR (^), NOT (~, !, NOT, or a postfix '), parentheses; NOT > AND > XOR > OR	

output	meaning
`variables`	the variables in truth-table order (first = most significant bit)
`minterms`	rows where the expression is 1
`maxterms`	rows where it is 0
`canonical_sop`	sum of all minterms
`minimal_sop`	a minimal sum of products (A' means NOT A, · means AND)
`terms`	number of product terms in the minimal form
`literals`	number of literals in the minimal form

- **Method:** truth table; Quine-McCluskey prime implicants; essential primes, then an exact search for the smallest cover
- **Assumes:** 1 to 4 variables
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Karnaugh map of A·B + A·B' + A'·B over A, B (Gray-coded rows and columns, each cell numbered by its minterm): the minterms the package returned (1, 2, 3) are shaded and marked 1; each term of the returned minimal sum of products, A + B, outlines the cells it covers (one colour per term). Marked: 2 terms, 2 literals.

```
electrical_engineering_ai999x.solve('truth_table_sop', expression="A·B + A·B' + A'·B")
# -> variables = A, B, minterms = [1.0, 2.0, 3.0], maxterms = [0.0], canonical_sop = A'·B + A·B' +
A·B, minimal_sop = A + B, terms = 2, literals = 2   (SOLVED)
```

```
electrical_engineering_ai999x.solve('truth_table_sop', expression='(A + B)·(A + C)')
# -> variables = A, B, C, minterms = [3.0, 4.0, 5.0, 6.0, 7.0], maxterms = [0.0, 1.0, 2.0],
canonical_sop = A'·B·C + A·B'·C' + A·B'·C + A·B·C' + A·B·C, minimal_sop = A + B·C, terms = 2,
literals = 3   (SOLVED)
```

```
electrical_engineering_ai999x.solve('truth_table_sop', expression='A XOR B')
# -> variables = A, B, minterms = [1.0, 2.0], maxterms = [0.0, 3.0], canonical_sop = A'·B + A·B',
minimal_sop = A'·B + A·B', terms = 2, literals = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Simplify A·B + A·B' + A'·B.	SOLVED	variables = A, B, minterms = [1.0, 2.0, 3.0], maxterms = [0.0], canonical_sop = A'·B + A·B' + A·B, minimal_sop = A + B, terms = 2, literals = 2
Find the minimal sum of products of (A + B)·(A + C).	SOLVED	variables = A, B, C, minterms = [3.0, 4.0, 5.0, 6.0, 7.0], maxterms = [0.0, 1.0, 2.0], canonical_sop = A'·B·C + A·B'·C' + A·B'·C + A·B·C' + A·B·C, minimal_sop = A + B·C, terms = 2, literals = 3
Simplify A·B·C·D·E + A'.	INVALID_INPUT	Truth table and minimal sum of products of a Boolean expression: minimization is offered for up to 4 variables.

Minimal sum of products from minterms (with don't-cares) — `minterms_to_sop`

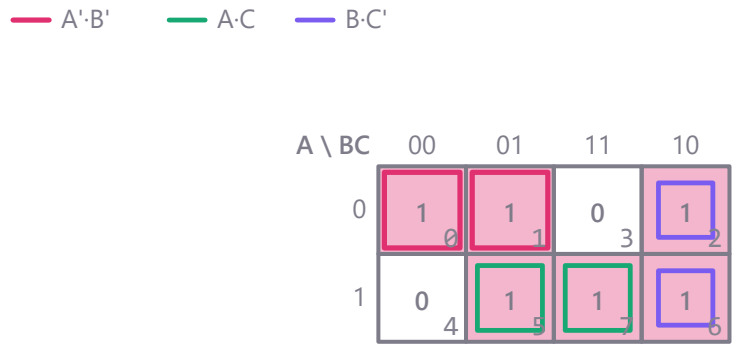
A minimal sum of products (fewest terms, then fewest literals) of F(A, B, C, D) = Σm(...) + d(...), the Karnaugh-map result.

$$F = \sum m(\ldots) + \sum d(\ldots)$$

input	meaning	condition
`minterms`	minterm numbers (A is the most significant bit)	
`variables`	number of variables (1 to 4)	
`dont_cares`	don't-care rows	

output	meaning
`minimal_sop`	a minimal sum of products in A, B, C, D (A' = NOT A)
`terms`	number of product terms
`literals`	number of literals
`variables`	the variables

- **Method:** Quine-McCluskey prime implicants over minterms and don't-cares; exact minimum cover of the minterms
- **Assumes:** 1 to 4 variables; minterms and don't-cares disjoint and in range
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Karnaugh map of the stated minterms 0, 1, 2, 5, 6, 7 over A, B, C: each term of the minimal sum of products the package returned, A'·B' + A·C + B·C', outlines the cells it covers (one colour per term). Marked: 3 terms, 6 literals.

```
electrical_engineering_ai999x.solve('minterms_to_sop', minterms='0, 1, 2, 5, 6, 7', variables=3)
# -> minimal_sop = A'·B' + A·C + B·C', terms = 3, literals = 6, variables = A, B, C   (SOLVED)
```

```
electrical_engineering_ai999x.solve('minterms_to_sop', minterms='1, 3, 7, 11, 15', variables=4,
dont_cares='0, 2, 5')
# -> minimal_sop = A'·B' + C·D, terms = 2, literals = 4, variables = A, B, C, D   (SOLVED)
```

```
electrical_engineering_ai999x.solve('minterms_to_sop', minterms='0, 2, 8, 10', variables=4)
# -> minimal_sop = B'·D', terms = 1, literals = 2, variables = A, B, C, D   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Simplify F(A,B,C) = Σm(0, 1, 2, 5, 6, 7).	SOLVED	minimal_sop = A'·B' + A·C + B·C', terms = 3, literals = 6, variables = A, B, C
Minimize F(A,B,C,D) = Σm(1, 3, 7, 11, 15) + d(0, 2, 5).	SOLVED	minimal_sop = A'·B' + C·D, terms = 2, literals = 4, variables = A, B, C, D
Simplify F(A,B,C) = Σm(0, 1, 9).	INVALID_INPUT	Minimal sum of products from minterms (with don't-cares): minterm numbers must be whole numbers from 0 to 7.
Simplify the function with minterms 1, 3, 5.	NEEDS_INPUT	minterms_to_sop needs variables (number of variables (1 to 4)). Example: solve('minterms_to_sop', minterms='0, 1, 2, 5,

Equivalence of two Boolean expressions — `boolean_equivalence`

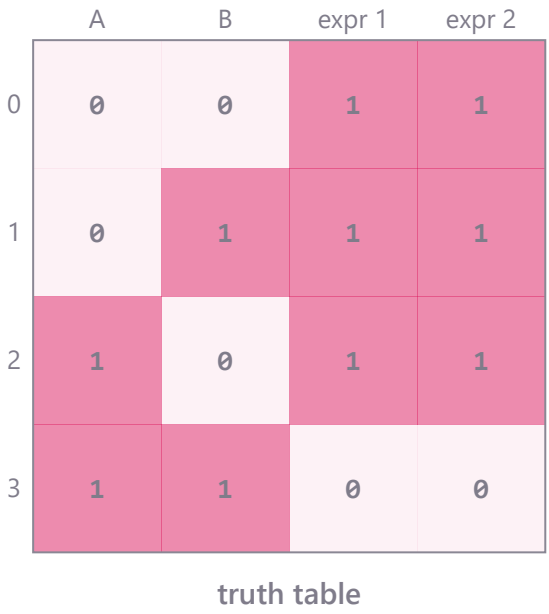
Whether two Boolean expressions are equal for every input, with a counterexample row when they are not.

$$F_1 \equiv F_2 \iff F_1 \oplus F_2 = 0$$

input	meaning	condition
`expression1`	first expression (same notation as boolean_evaluate)	
`expression2`	second expression	

output	meaning
`equivalent`	true when the two agree on every row
`counterexample`	the first row where they differ ('none' if equivalent)

- **Method:** exhaustive comparison over the truth table
- **Assumes:** at most 12 variables
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



truth table

Truth table of expression 1, (A·B)', and expression 2, A' + B', over every assignment of A, B (rows numbered by the assignment as a binary number); each expression's column is the value boolean_evaluate of the package returned (4 × 2 runs). The two columns agree in 4 of 4 rows; the package returned equivalent = yes (counterexample: none). Marked: equivalent = yes.

```
electrical_engineering_ai999x.solve('boolean_equivalence', expression1="(A·B)'", expression2="A' + B'")
# -> equivalent = True, counterexample = none   (SOLVED)
```

```
electrical_engineering_ai999x.solve('boolean_equivalence', expression1='A + B', expression2='A XOR B')
# -> equivalent = False, counterexample = A=1, B=1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Is (A·B)' equivalent to A' + B'?	SOLVED	equivalent = True, counterexample = none
Are A + A'·B and A + B equivalent?	SOLVED	equivalent = True, counterexample = none
Is A + B equivalent to A XOR B?	SOLVED	equivalent = False, counterexample = A=1, B=1

Number base conversion (binary, octal, decimal, hexadecimal) — `base_conversion`

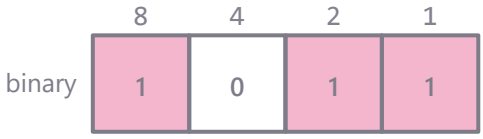
A whole number written in base 2, 8, 10 or 16, given in all four bases.

$$N = \sum_i d_i \, b^i$$

input	meaning	condition
`value`	the digits (0b, 0o, 0x prefixes allowed)	
`base`	the base the value is written in	

output	meaning
`decimal`	value in base 10
`binary`	base 2
`octal`	base 8
`hexadecimal`	base 16 (upper-case digits)

- **Method:** positional value, then repeated division by the target base
- **Assumes:** a non-negative whole number below 2^{53}; digits valid for the base
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



`decimal 11, octal 13, hexadecimal B`

The value 1011 written in base 2 (as stated), shown as the binary digits the package returned with the weight of each bit above it; the package returned decimal 11, octal 13, hexadecimal B. Marked: binary 1011.

```
electrical_engineering_ai999x.solve('base_conversion', value='1011', base='2')
# -> decimal = 11, binary = 1011, octal = 13, hexadecimal = B   (SOLVED)
```

```
electrical_engineering_ai999x.solve('base_conversion', value='255', base='10')
# -> decimal = 255, binary = 11111111, octal = 377, hexadecimal = FF   (SOLVED)
```

```
electrical_engineering_ai999x.solve('base_conversion', value='0x1F', base='16')
# -> decimal = 31, binary = 11111, octal = 37, hexadecimal = 1F   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Convert 1011 from binary to decimal.	SOLVED	decimal = 11, binary = 1011, octal = 13, hexadecimal = B
Convert 255 to hexadecimal.	SOLVED	decimal = 255, binary = 11111111, octal = 377, hexadecimal = FF
Convert 0x1F to binary.	SOLVED	decimal = 31, binary = 11111, octal = 37, hexadecimal = 1F
Convert the octal number 777 to decimal.	SOLVED	decimal = 511, binary = 111111111, octal = 777, hexadecimal = 1FF
Convert 1021 from binary to decimal.	INVALID_INPUT	Number base conversion (binary, octal, decimal, hexadecimal): '1021' is not a whole number in base 2.
Convert 1011 to decimal.	NEEDS_INPUT	base_conversion needs base (the base the value is written in). Example: solve('base_conversion', value='1011', base='2')

Two's complement representation of a signed integer — `twos_complement_encode`

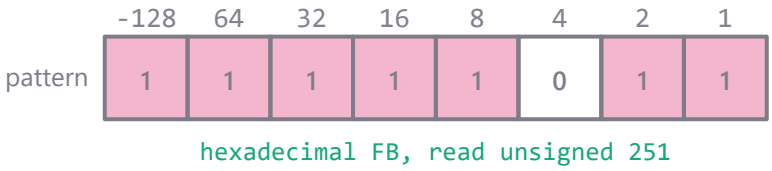
The n-bit two's complement bit pattern (and its hexadecimal form) of a signed whole number.

$$x_{2c} = x \bmod 2^n$$

input	meaning	condition
`value`	the signed whole number	
`bits`	word width in bits	

output	meaning
`pattern`	the bit pattern, most significant bit first
`hexadecimal`	the pattern in hexadecimal
`unsigned`	the pattern read as an unsigned number

- **Method:** value modulo 2^n (for a negative number: invert the bits of |value| and add 1)
- **Assumes:** -2^(n-1) <= value < 2^(n-1)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The stated value -5 in 8 bits: the two's-complement pattern the package returned, 11111011, with each bit's weight above it; hexadecimal FB, read as unsigned 251. Marked: 11111011.

```
electrical_engineering_ai999x.solve('twos_complement_encode', value=-5, bits=8)
# -> pattern = 11111011, hexadecimal = FB, unsigned = 251   (SOLVED)
```

```
electrical_engineering_ai999x.solve('twos_complement_encode', value=100, bits=8)
# -> pattern = 01100100, hexadecimal = 64, unsigned = 100   (SOLVED)
```

```
electrical_engineering_ai999x.solve('twos_complement_encode', value=-1234, bits=16)
# -> pattern = 1111101100101110, hexadecimal = FB2E, unsigned = 64302   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Write -5 in 8-bit two's complement.	SOLVED	pattern = 11111011, hexadecimal = FB, unsigned = 251
Find the 16-bit two's complement representation of -1234.	SOLVED	pattern = 1111101100101110, hexadecimal = FB2E, unsigned = 64302
Write -200 in 8-bit two's complement.	INVALID_INPUT	Two's complement representation of a signed integer: -200 does not fit in 8-bit two's complement (range -128 to 127).

Value of a two's complement bit pattern — `twos_complement_decode`

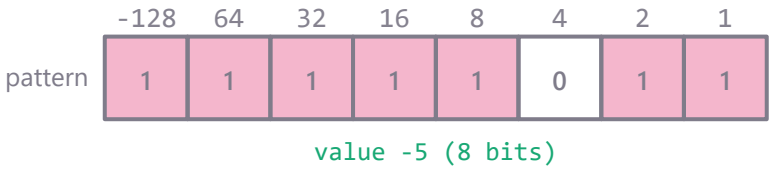
The signed value of an n-bit two's complement pattern (n = the number of digits written).

$$x = -b_{n-1}2^{n-1} + \sum_{i<n-1} b_i 2^i$$

input	meaning	condition
`pattern`	the bits, most significant first	

output	meaning
`value`	signed value
`bits`	word width

- **Method:** unsigned value minus 2^n when the sign bit is 1
- **Assumes:** only 0 and 1; at most 64 bits
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The stated 8-bit pattern 11111011 with the two's-complement weight of each bit above it (the leading bit weighs -128); the package returned the value -5. Marked: value = -5.

```
electrical_engineering_ai999x.solve('twos_complement_decode', pattern='11111011')
# -> value = -5, bits = 8   (SOLVED)
```

```
electrical_engineering_ai999x.solve('twos_complement_decode', pattern='0111')
# -> value = 7, bits = 4   (SOLVED)
```

```
electrical_engineering_ai999x.solve('twos_complement_decode', pattern='1000')
# -> value = -8, bits = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What decimal value does the 8-bit two's complement number 11111011 represent?	SOLVED	value = -5, bits = 8
Decode the two's complement pattern 1000.	SOLVED	value = -8, bits = 4
What decimal value does the 8-bit two's complement number 1111 represent?	OUT_OF_SCOPE	Out of scope: the stated width differs from the number of digits written.

Binary to Gray code and Gray code to binary — `gray_code`

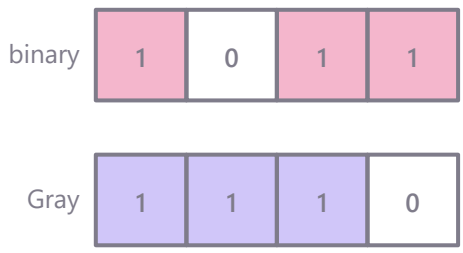
The reflected binary (Gray) code of a binary word, or the binary word of a Gray code, with the same width.

$$g = b \oplus (b \gg 1)$$

input	meaning	condition
`value`	the bits	
`direction`	conversion direction	

output	meaning
`result`	the converted bits
`decimal`	the result read as an unsigned number

- **Method:** $g = b$ XOR (b >> 1); back: $b = g$ XOR (g >> 1) XOR (g >> 2) ...
- **Assumes:** 1 to 32 bits
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The stated bit pattern 1011 and the Gray code the package returned, 1110 (1 bits shaded); its value is 14. Marked: 1110.

```
electrical_engineering_ai999x.solve('gray_code', value='1011', direction='to_gray')
# -> result = 1110, decimal = 14   (SOLVED)
```

```
electrical_engineering_ai999x.solve('gray_code', value='1110', direction='to_binary')
# -> result = 1011, decimal = 11   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Convert the binary number 1011 to Gray code.	SOLVED	result = 1110, decimal = 14
Convert the Gray code 1110 to binary.	SOLVED	result = 1011, decimal = 11
Convert the binary number 1012 to Gray code.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electrical_engineering_ai999x.tem

Voltage divider — `voltage_divider`

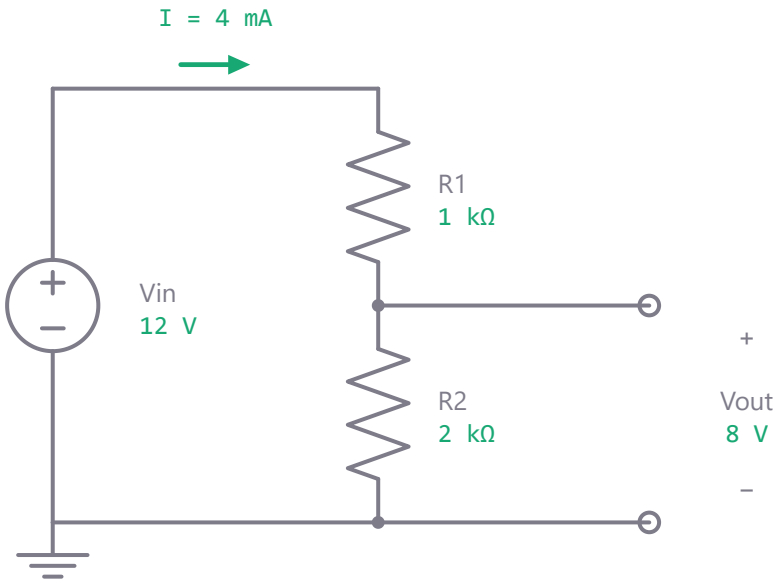
The output voltage across R2 of two resistors in series across Vin (R1 on top, R2 to ground), and the current.

$$V_{out} = V_{in}\frac{R_2}{R_1 + R_2}$$

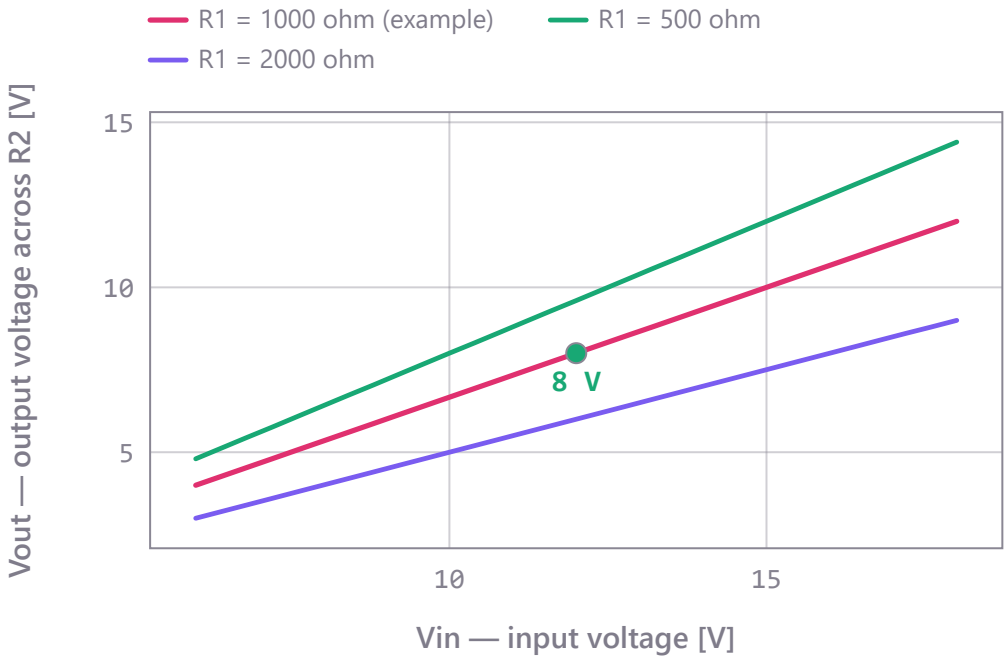
input	meaning	condition
`Vin`	input voltage	
`R1`	top resistor (from the input to the output)	
`R2`	bottom resistor (from the output to ground)	

output	meaning
`Vout`	output voltage across R2 (unloaded)
`I`	current through the divider

- **Method:** Vout = Vin $R_2/(R_1 + R_2)$
- **Assumes:** no load on the output; R1, $R_2 > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Voltage divider as stated: Vin = 12 V across R1 = 1 kΩ (top) and R2 = 2 kΩ (to ground), no load. Labelled with what the package returned: Vout = 8 V across R2, I = 4 mA through both. Marked: Vout = 8 V, I = 0.004 A.



Vout (output voltage across R2 (unloaded)) against Vin (input voltage) from 6 V to 18 V. One curve for each value of R1 (top resistor (from the input to the output)): 1000 ohm, 500 ohm, 2000 ohm. Other inputs as in the worked example: R2 = 2000 ohm. 31 points per curve computed by electrical-engineering-ai999x; marked: the worked example, Vin = 12 V, Vout = 8 V.

```
electrical_engineering_ai999x.solve('voltage_divider', Vin=12, R1=1000, R2=2000)
# -> Vout = 8 V, I = 0.004 A   (SOLVED)
```

```
electrical_engineering_ai999x.solve('voltage_divider', Vin=5, R1=10000, R2=10000)
# -> Vout = 2.5 V, I = 0.00025 A   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the output of a voltage divider with Vin = 12 V, R1 = 1 kohm, R2 = 2 kohm.	SOLVED	Vout = 8 V, I = 0.004 A
A voltage divider has Vin = 5 V, R1 = 10 kohm and R2 = 10 kohm. What is the output voltage?	SOLVED	Vout = 2.5 V, I = 0.00025 A
Find the output of a voltage divider with Vin = 12 V, R1 = 1 kohm.	NEEDS_INPUT	voltage_divider needs R2 (bottom resistor (from the output to ground)). Example: solve('voltage_divider', Vin='12 V', R1
Find the output of a voltage divider with Vin = 12 V, R1 = 1 kohm, R2 = 2 kohm, R3 = 5 kohm.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electrical_engineering_ai999x.tem

Current divider — `current_divider`

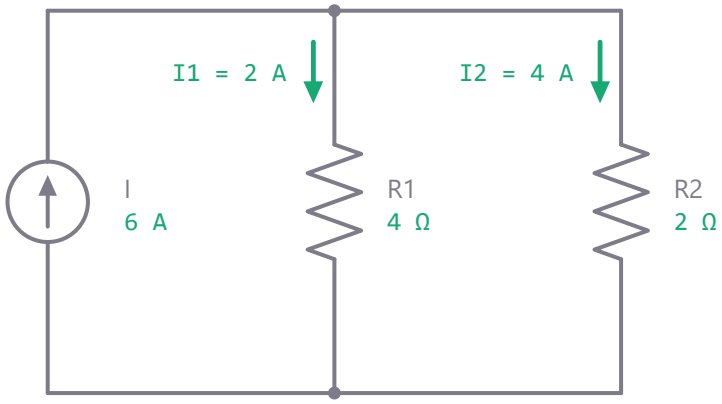
How a current I splits between two resistors in parallel.

$$I_1 = I\frac{R_2}{R_1 + R_2}$$

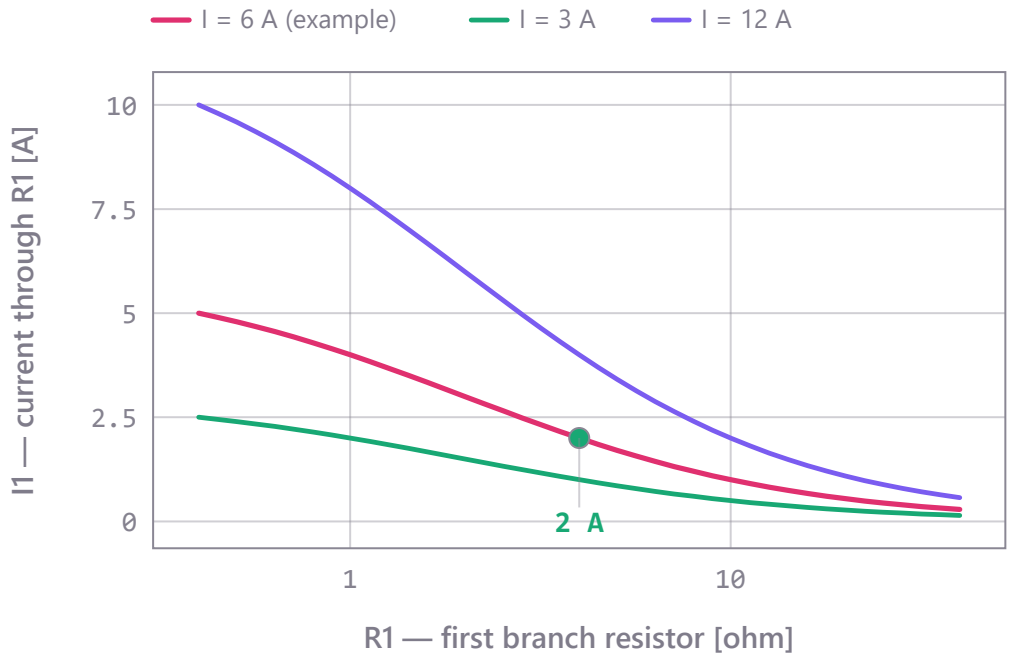
input	meaning	condition
`I`	total current into the parallel pair	
`R1`	first branch resistor	
`R2`	second branch resistor	

output	meaning
`I1`	current through R1
`I2`	current through R2

- **Method:** $I_1 = I\,R_2/(R_1 + R_2)$
- **Assumes:** R1, $R_2 > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Current divider as stated: a source I = 6 A feeding R1 = 4 Ω in parallel with R2 = 2 Ω; the branch currents are those the package returned. Marked: I1 = 2 A, I2 = 4 A.



I1 (current through R1) against R1 (first branch resistor) from 0.4 ohm to 40 ohm on a logarithmic axis. One curve for each value of I (total current into the parallel pair): 6 A, 3 A, 12 A. Other inputs as in the worked example: R2 = 2 ohm. 41 points per curve computed by electrical-engineering-ai999x; marked: the worked example, R1 = 4 ohm, I1 = 2 A.

```
electrical_engineering_ai999x.solve('current_divider', I=6, R1=4, R2=2)
# -> I1 = 2 A, I2 = 4 A   (SOLVED)
```

```
electrical_engineering_ai999x.solve('current_divider', I=1, R1=100, R2=300)
# -> I1 = 0.75 A, I2 = 0.25 A   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the branch currents of a current divider with I = 6 A, R1 = 4 ohm, R2 = 2 ohm.	SOLVED	I1 = 2 A, I2 = 4 A
Find the branch currents of a current divider with I = 6 A, R1 = 4 ohm, R2 = 0 ohm.	INVALID_INPUT	Current divider: R2 must be positive.

Maximum power transfer — `max_power_transfer`

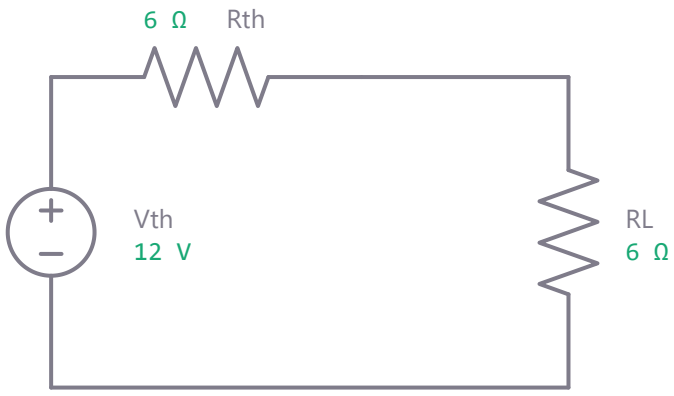
The load resistance that draws the most power from a source with Thevenin voltage Vth and resistance Rth, and that power.

$$P_{max} = \frac{V_{th}^2}{4R_{th}}$$

input	meaning	condition
`Vth`	Thevenin (open-circuit) voltage	
`Rth`	Thevenin resistance	

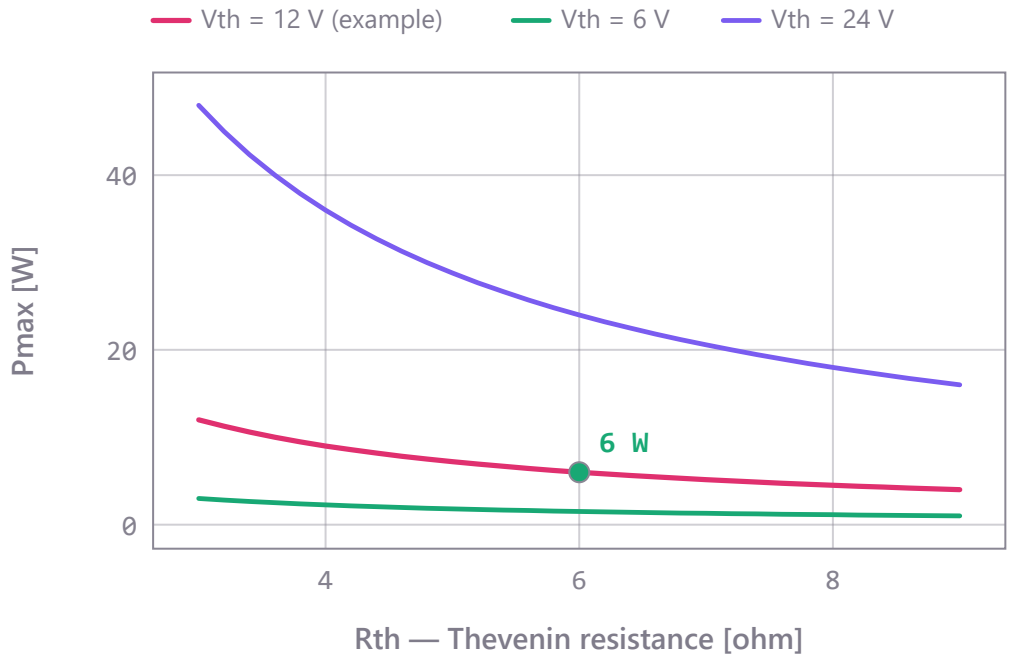
output	meaning
`RL`	load resistance for maximum power (= Rth)
`Pmax`	maximum power delivered to the load
`efficiency`	fraction of the source power reaching the load at that point

- **Method:** RL = Rth, Pmax = Vth^2/(4 Rth)
- **Assumes:** a resistive load; Rth > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



`Pmax = 6 W, efficiency 0.5`

A source Vth = 12 V with Rth = 6 Ω (as stated) and the load the package returned for maximum power, RL = 6 Ω, which then takes Pmax = 6 W at efficiency 0.5. Marked: RL = 6 Ω, Pmax = 6 W.



Pmax (maximum power delivered to the load) against Rth (Thevenin resistance) from 3 ohm to 9 ohm. One curve for each value of Vth (Thevenin (open-circuit) voltage): 12 V, 6 V, 24 V. 31 points per curve computed by electrical-engineering-ai999x; marked: the worked example, Rth = 6 ohm, Pmax = 6 W.

```
electrical_engineering_ai999x.solve('max_power_transfer', Vth=12, Rth=6)
# -> RL = 6 ohm, Pmax = 6 W, efficiency = 0.5   (SOLVED)
```

```
electrical_engineering_ai999x.solve('max_power_transfer', Vth=10, Rth=50)
# -> RL = 50 ohm, Pmax = 0.5 W, efficiency = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the maximum power transfer for Vth = 12 V, Rth = 6 ohm.	SOLVED	RL = 6 ohm, Pmax = 6 W, efficiency = 0.5
What is the maximum power delivered to a load from a source with Vth = 10 V and Rth = 50 ohm?	SOLVED	RL = 50 ohm, Pmax = 0.5 W, efficiency = 0.5

Wheatstone bridge output — `wheatstone_bridge`

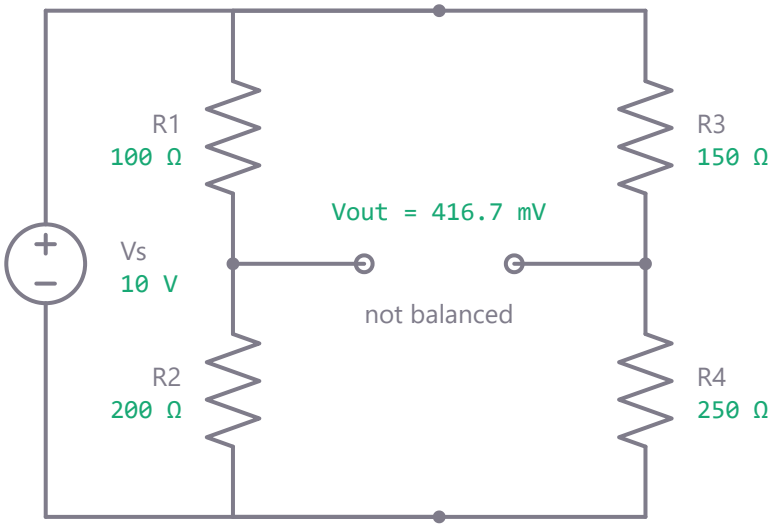
The output voltage of a Wheatstone bridge (left arm R1 over R2, right arm R3 over R4, supply Vs), whether it is balanced, and the R4 that balances it.

$$V_{out} = V_s \left(\frac{R_2}{R_1 + R_2} - \frac{R_4}{R_3 + R_4} \right)$$

input	meaning	condition
`Vs`	bridge supply voltage	
`R1`	left arm, top	
`R2`	left arm, bottom	
`R3`	right arm, top	
`R4`	right arm, bottom	

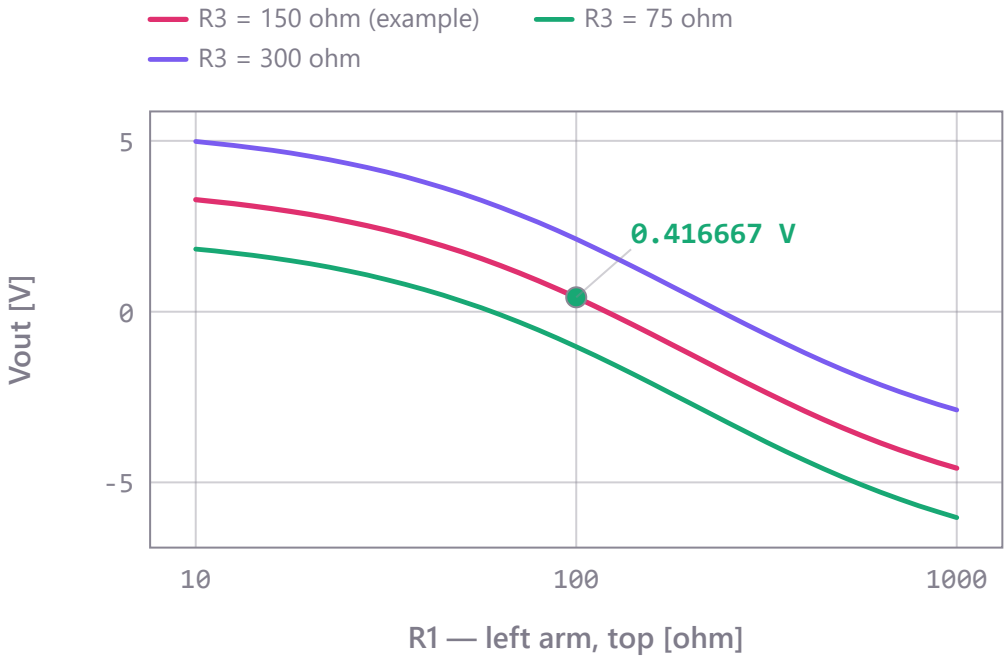
output	meaning
`Vout`	voltage of the left midpoint relative to the right midpoint
`balanced`	true when R1 $R_4 = R_2$ R3
`R4_balance`	the R4 that would balance the bridge

- **Method:** difference of the two divider outputs
- **Assumes:** an unloaded output (ideal voltmeter)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



balance needs R4 = 300 Ω

Wheatstone bridge as stated: Vs = 10 V, R1 = 100 Ω, R2 = 200 Ω (left arm, top to bottom), R3 = 150 Ω, R4 = 250 Ω (right arm). Labelled with what the package returned: the voltage between the arm midpoints Vout = 0.416667 V, not balanced, balance at R4 = 300 Ω. Marked: Vout = 0.416667 V.



Vout (voltage of the left midpoint relative to the right midpoint) against R1 (left arm, top) from 10 ohm to 1000 ohm on a logarithmic axis. One curve for each value of R3 (right arm, top): 150 ohm, 75 ohm, 300 ohm. Other inputs as in the worked example: Vs = 10 V, R2 = 200 ohm, R4 = 250 ohm. 41 points per curve computed by electrical-engineering-ai999x; marked: the worked example, R1 = 100 ohm, Vout = 0.416667 V.

```
electrical_engineering_ai999x.solve('wheatstone_bridge', Vs=10, R1=100, R2=200, R3=150, R4=250)
# -> Vout = 0.416667 V, balanced = False, R4_balance = 300 ohm   (SOLVED)
```

```
electrical_engineering_ai999x.solve('wheatstone_bridge', Vs=5, R1=100, R2=100, R3=220, R4=220)
# -> Vout = 0 V, balanced = True, R4_balance = 220 ohm   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the output of a Wheatstone bridge with Vs = 10 V, R1 = 100 ohm, R2 = 200 ohm, R3 = 150 ohm, R4 = 250 ohm.	SOLVED	Vout = 0.416667 V, balanced = False, R4_balance = 300 ohm
Find the output of a Wheatstone bridge with Vs = 10 V, R1 = 100 ohm, R2 = 200 ohm, R3 = 150 ohm.	NEEDS_INPUT	wheatstone_bridge needs R4 (right arm, bottom). Example: solve('wheatstone_bridge', Vs='10 V', R1='100 ohm', R2='200 ohm

First-order RC filter: cutoff, gain and phase — `rc_filter`

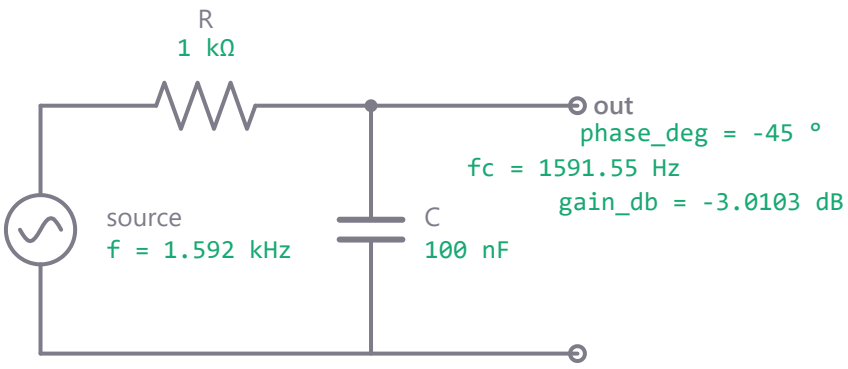
The cutoff frequency 1/(2 pi R C) of an RC low-pass or high-pass filter and its gain (ratio and dB) and phase at a frequency.

$$f_c = \frac{1}{2\pi RC}, \quad H_{LP} = \frac{1}{1 + jf/f_c}$$

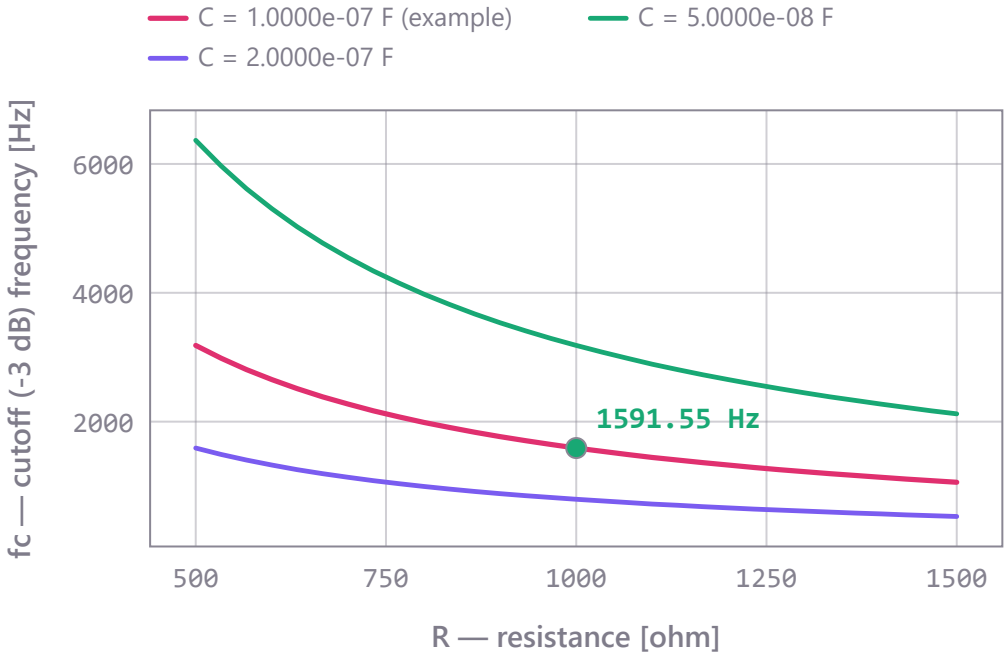
input	meaning	condition
`R`	resistance	
`C`	capacitance	
`f`	signal frequency	
`filter`	first-order low-pass or high-pass	

output	meaning
`fc`	cutoff (-3 dB) frequency
`gain`	|Vout/Vin| at f
`gain_db`	gain in decibels
`phase_deg`	phase of Vout relative to Vin, degrees

- **Method:** H = 1/(1 + j f/fc) (low-pass) or (j $f/fc)/(1 + j$ f/fc) (high-pass)
- **Assumes:** unloaded output; R, C, $f > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The RC filter circuit of the worked example (R = 1000, C = 1e-07, f = 1591.5494309189535, filter = lowpass), labelled with what the package returned: phase_deg = -45; fc = 1591.55; gain_db = -3.0103. Marked: phase_deg = -45.



fc (cutoff (-3 dB) frequency) against R (resistance) from 500 ohm to 1500 ohm. One curve for each value of C (capacitance): 1.0000e-07 F, 5.0000e-08 F, 2.0000e-07 F. Other inputs as in the worked example: f = 1591.5494309189535 Hz, filter = lowpass. 31 points per curve computed by electrical-engineering-ai999x; marked: the worked example, R = 1000 ohm, fc = 1591.55 Hz.

```
electrical_engineering_ai999x.solve('rc_filter', R=1000, C=1e-07, f=1591.5494309189535,
filter='lowpass')
# -> fc = 1591.55 Hz, gain = 0.707107, gain_db = -3.0103, phase_deg = -45   (SOLVED)
```

```
electrical_engineering_ai999x.solve('rc_filter', R=1000, C=1e-06, f=159.15494309189535,
filter='highpass')
# -> fc = 159.155 Hz, gain = 0.707107, gain_db = -3.0103, phase_deg = 45   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
An RC low-pass filter has R = 1 kohm and C = 100 nF. Find the gain at f = 2 kHz.	SOLVED	fc = 1591.55 Hz, gain = 0.622677, gain_db = -4.11474, phase_deg = -51.4881

question	outcome	result
An RC high-pass filter has R = 10 kohm and C = 10 nF. Find its response at f = 100 Hz.	SOLVED	fc = 1591.55 Hz, gain = 0.0627082, gain_db = -24.0535, phase_deg = 86.4047
An RC low-pass filter has R = 1 kohm and C = 100 nF. Find the gain.	NEEDS_INPUT	rc_filter needs f (signal frequency). Example: solve('rc_filter', R='1000 ohm', C='1e-07 F', f='1591.55 Hz', filter='low

Resonance, Q factor and bandwidth of a series RLC circuit — `series_rlc_resonance`

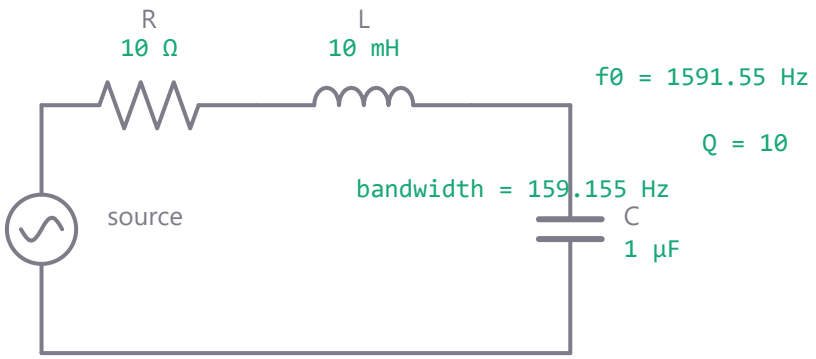
The resonant frequency, quality factor, half-power bandwidth and half-power frequencies of a series RLC circuit.

$$f_0 = \frac{1}{2\pi\sqrt{LC}},\ Q = \frac{1}{R}\sqrt{\frac{L}{C}}$$

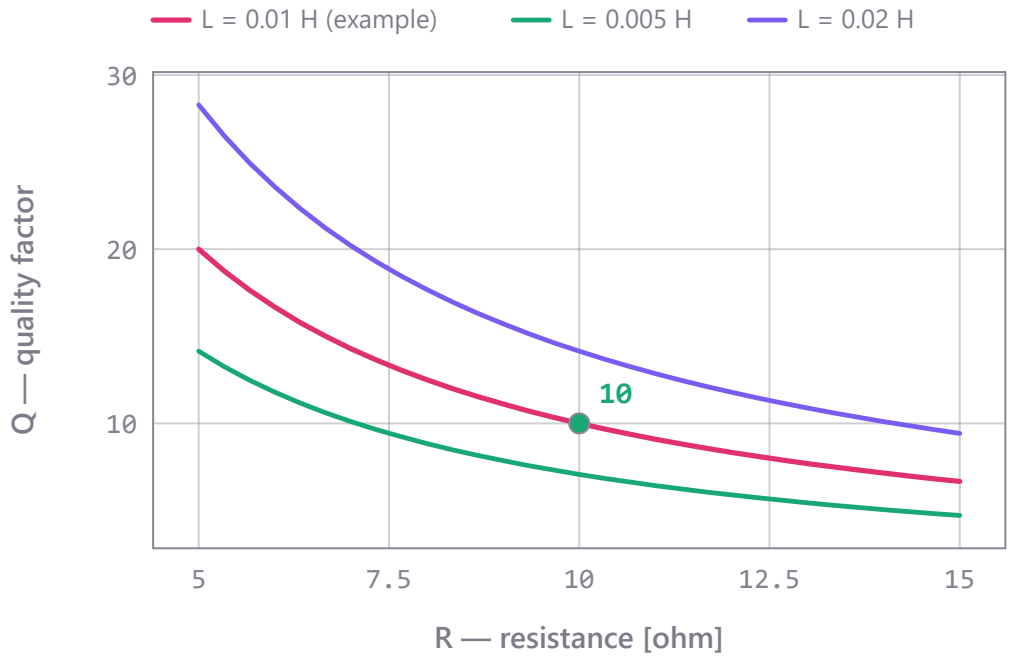
input	meaning	condition
`R`	resistance	
`L`	inductance	
`C`	capacitance	

output	meaning
`f0`	resonant frequency
`Q`	quality factor
`bandwidth`	half-power bandwidth
`f_low`	lower half-power frequency
`f_high`	upper half-power frequency

- **Method:** $Q = (1/R)$ sqrt(L/C), bandwidth = R/(2 pi L)
- **Assumes:** ideal R, L, C
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The series RLC circuit of the worked example (R = 10, L = 0.01, C = 1e-06), labelled with what the package returned: f0 = 1591.55; Q = 10; bandwidth = 159.155. Marked: f0 = 1591.55.



Q (quality factor) against R (resistance) from 5 ohm to 15 ohm. One curve for each value of L (inductance): 0.01 H, 0.005 H, 0.02 H. Other inputs as in the worked example: C = 1e-06 F. 31 points per curve computed by electrical-engineering-ai999x; marked: the worked example, R = 10 ohm, Q = 10.

```
electrical_engineering_ai999x.solve('series_rlc_resonance', R=10, L=0.01, C=1e-06)
# -> f0 = 1591.55 Hz, Q = 10, bandwidth = 159.155 Hz, f_low = 1513.96 Hz, f_high = 1673.12 Hz
(SOLVED)
```

```
electrical_engineering_ai999x.solve('series_rlc_resonance', R=1000, L=0.001, C=1e-09)
# -> f0 = 159155 Hz, Q = 1, bandwidth = 159155 Hz, f_low = 98363.2 Hz, f_high = 257518 Hz
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the resonance and bandwidth of a series RLC circuit with R = 10 ohm, L = 10 mH, C = 1 uF.	SOLVED	f0 = 1591.55 Hz, Q = 10, bandwidth = 159.155 Hz, f_low = 1513.96 Hz, f_high = 1673.12 Hz
What is the Q factor of a series RLC circuit with R = 10 ohm, L = 10 mH, C = 0 uF?	INVALID_INPUT	Resonance, Q factor and bandwidth of a series RLC circuit: C must be positive.

Resonance, Q factor and bandwidth of a parallel RLC circuit — `parallel_rlc_resonance`

The resonant frequency, quality factor, half-power bandwidth and half-power frequencies of a parallel RLC circuit.

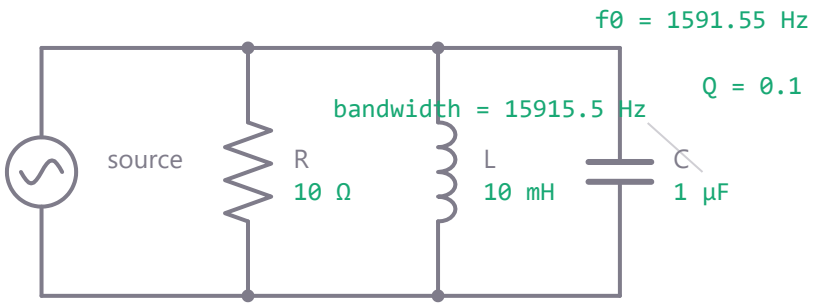
$$f_0 = \frac{1}{2\pi\sqrt{LC}},\ Q = R\sqrt{\frac{C}{L}}$$

input	meaning	condition
`R`	resistance	
`L`	inductance	

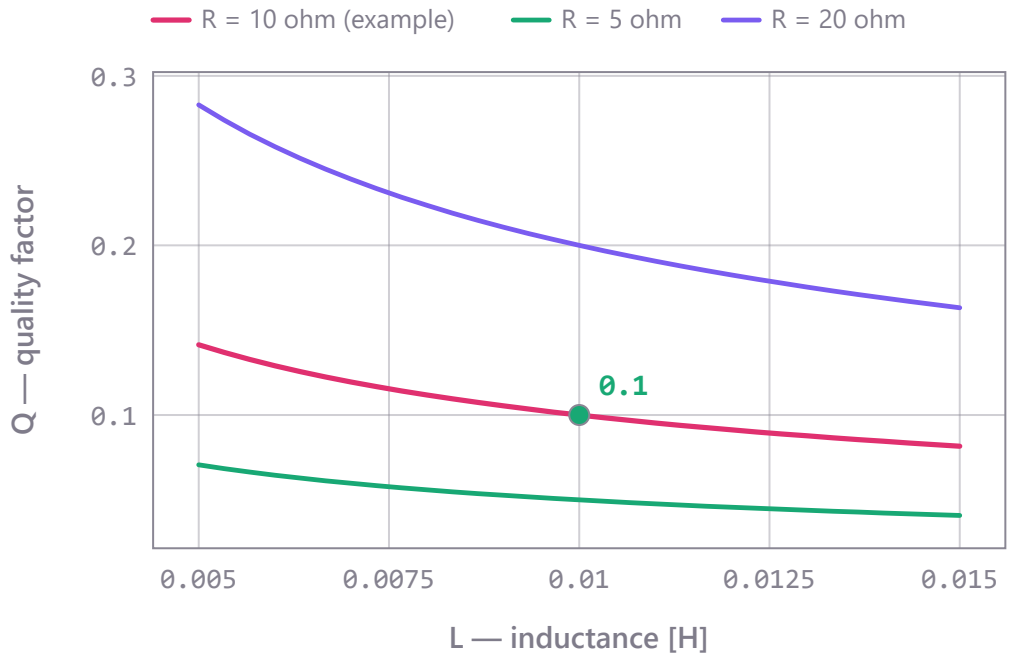
input	meaning	condition
`C`	capacitance	

output	meaning
`f0`	resonant frequency
`Q`	quality factor
`bandwidth`	half-power bandwidth
`f_low`	lower half-power frequency
`f_high`	upper half-power frequency

- **Method:** $Q = R$ sqrt(C/L), bandwidth = 1/(2 pi R C)
- **Assumes:** ideal R, L, C
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The parallel RLC circuit of the worked example (R = 10, L = 0.01, C = 1e-06), labelled with what the package returned: f0 = 1591.55; Q = 0.1; bandwidth = 15915.5. Marked: f0 = 1591.55.



Q (quality factor) against L (inductance) from 0.005 H to 0.015 H. One curve for each value of R (resistance): 10 ohm, 5 ohm, 20 ohm. Other inputs as in the worked example: C = 1e-06 F. 31 points per curve computed by electrical-engineering-ai999x; marked: the worked example, L = 0.01 H, Q = 0.1.

```
electrical_engineering_ai999x.solve('parallel_rlc_resonance', R=10, L=0.01, C=1e-06)
# -> f0 = 1591.55 Hz, Q = 0.1, bandwidth = 15915.5 Hz, f_low = 157.594 Hz, f_high = 16073.1 Hz
(SOLVED)
```

```
electrical_engineering_ai999x.solve('parallel_rlc_resonance', R=1000, L=0.001, C=1e-09)
# -> f0 = 159155 Hz, Q = 1, bandwidth = 159155 Hz, f_low = 98363.2 Hz, f_high = 257518 Hz
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the resonance and bandwidth of a parallel RLC circuit with R = 10 ohm, L = 10 mH, C = 1 uF.	SOLVED	f0 = 1591.55 Hz, Q = 0.1, bandwidth = 15915.5 Hz, f_low = 157.594 Hz, f_high = 16073.1 Hz
What is the Q factor of a parallel RLC circuit with R = 10 ohm, L = 10 mH, C = 0 uF?	INVALID_INPUT	Resonance, Q factor and bandwidth of a parallel RLC circuit: C must be positive.

First-order (RC or RL) transient response — `first_order_transient`

The value at time t of any voltage in a first-order circuit going from its initial value V0 toward its final value Vf with time constant tau.

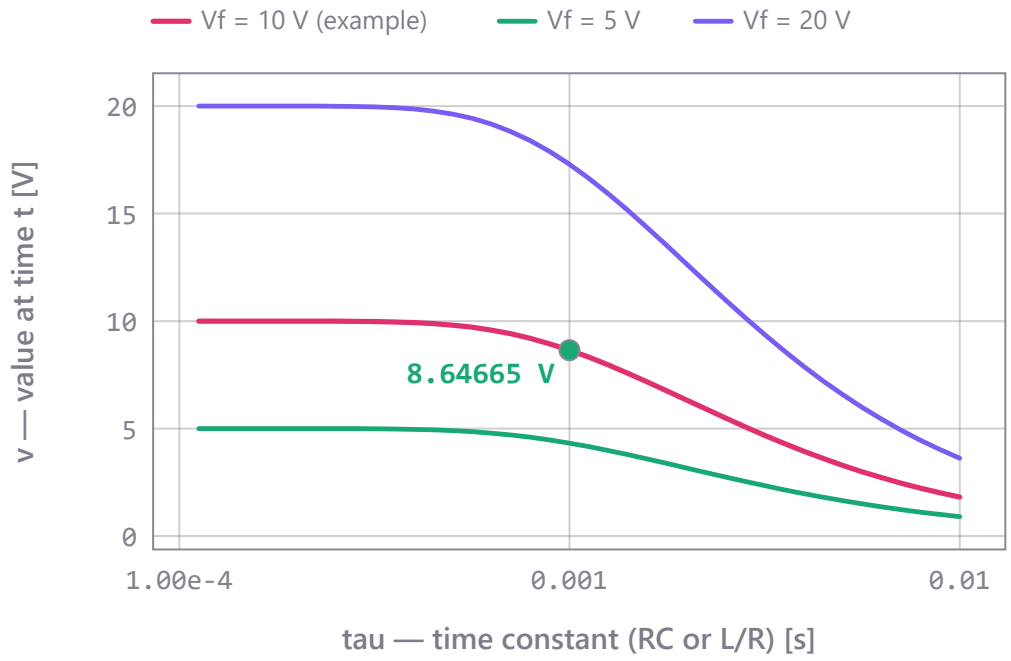
$$v(t) = V_f + (V_0 - V_f)e^{-t/\tau}$$

input	meaning	condition
`V0`	initial value (at $t = 0+$)	
`Vf`	final (steady-state) value	
`tau`	time constant (RC or L/R)	

input	meaning	condition
`t`	time after the switching	

output	meaning
`v`	value at time t
`fraction_complete`	fraction of the change completed, $1 - exp$(-t/tau)

- **Method:** v(t) = Vf + (V0 - Vf) exp(-t/tau)
- **Assumes:** a single time constant; a step change at $t = 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



v (value at time t) against tau (time constant (RC or L/R)) from 0.0001 s to 0.01 s on a logarithmic axis. One curve for each value of Vf (final (steady-state) value): 10 V, 5 V, 20 V. Other inputs as in the worked example: V0 = 0 V, t = 0.002 s. 40 points per curve computed by electrical-engineering-ai999x; marked: the worked example, tau = 0.001 s, v = 8.64665 V.

```
electrical_engineering_ai999x.solve('first_order_transient', V0=0, Vf=10, tau=0.001, t=0.002)
# -> v = 8.64665 V, fraction_complete = 0.864665   (SOLVED)
```

```
electrical_engineering_ai999x.solve('first_order_transient', V0=5, Vf=0, tau=2, t=2)
# -> v = 1.8394 V, fraction_complete = 0.632121   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the first-order transient value with V0 = 0 V, Vf = 10 V, tau = 1 ms, t = 2 ms.	SOLVED	v = 8.64665 V, fraction_complete = 0.864665
Find the first-order transient value with V0 = 0 V, Vf = 10 V, tau = 0 ms, t = 2 ms.	INVALID_INPUT	First-order (RC or RL) transient response: tau must be positive.

Inverting amplifier gain and output — `inverting_amplifier`

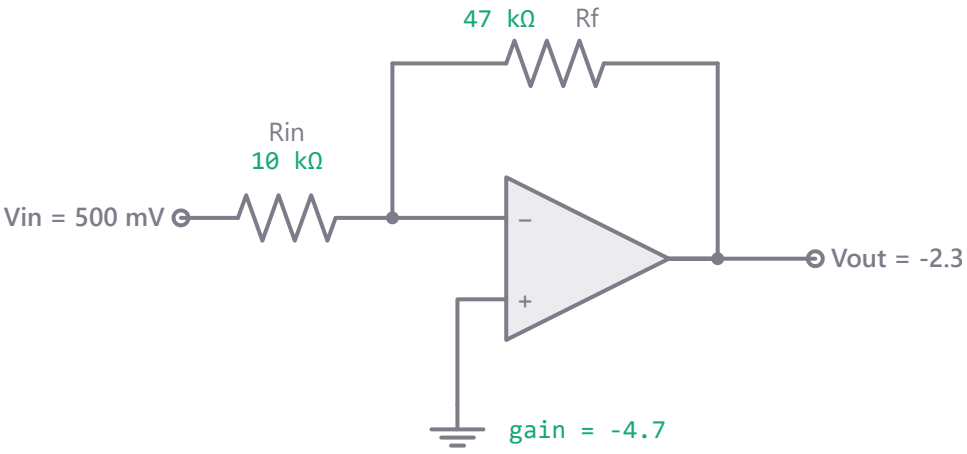
The voltage gain -Rf/Rin of an ideal inverting amplifier and its output for an input voltage.

$$A_v = -\frac{R_f}{R_{in}}$$

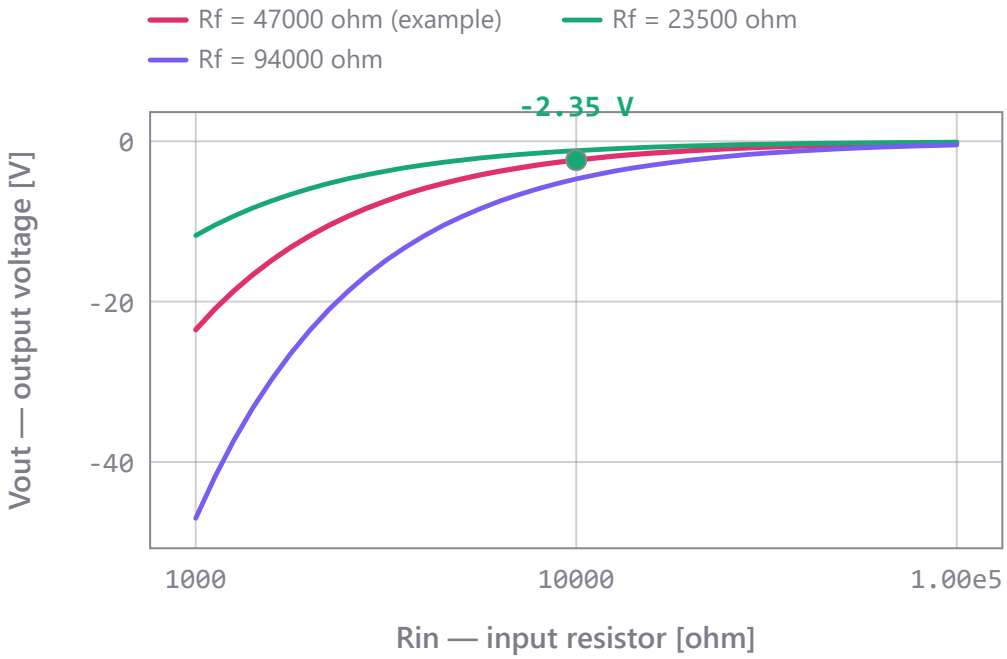
input	meaning	condition
`Rin`	input resistor	
`Rf`	feedback resistor	
`Vin`	input voltage	

output	meaning
`gain`	closed-loop voltage gain
`Vout`	output voltage

- **Method:** gain = -Rf/Rin
- **Assumes:** ideal op-amp, no saturation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Inverting amplifier as stated: Rin = 10 kΩ, feedback Rf = 47 kΩ, input Vin = 500 mV, non-inverting input grounded. Labelled with what the package returned: gain = -4.7, Vout = -2.35 V. Marked: Vout = -2.35 V.



Vout (output voltage) against Rin (input resistor) from 1000 ohm to 100000 ohm on a logarithmic axis. One curve for each value of Rf (feedback resistor): 47000 ohm, 23500 ohm, 94000 ohm. Other inputs as in the worked example: Vin = 0.5 V. 41 points per curve computed by electrical-engineering-ai999x; marked: the worked example, Rin = 10000 ohm, Vout = -2.35 V.

```
electrical_engineering_ai999x.solve('inverting_amplifier', Rin=10000.0, Rf=47000.0, Vin=0.5)
# -> gain = -4.7, Vout = -2.35 V   (SOLVED)
```

```
electrical_engineering_ai999x.solve('inverting_amplifier', Rin=1000.0, Rf=10000.0, Vin=-0.2)
# -> gain = -10, Vout = 2 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the output of an inverting amplifier with Rin = 10 kohm, Rf = 47 kohm, Vin = 0.5 V.	SOLVED	gain = -4.7, Vout = -2.35 V
Find the output of an inverting amplifier with Rin = 10 kohm, Rf = 47 kohm.	NEEDS_INPUT	inverting_amplifier needs Vin (input voltage). Example: solve('inverting_amplifier', Rin='10000 ohm', Rf='47000 ohm', Vi

Non-inverting amplifier gain and output — `noninverting_amplifier`

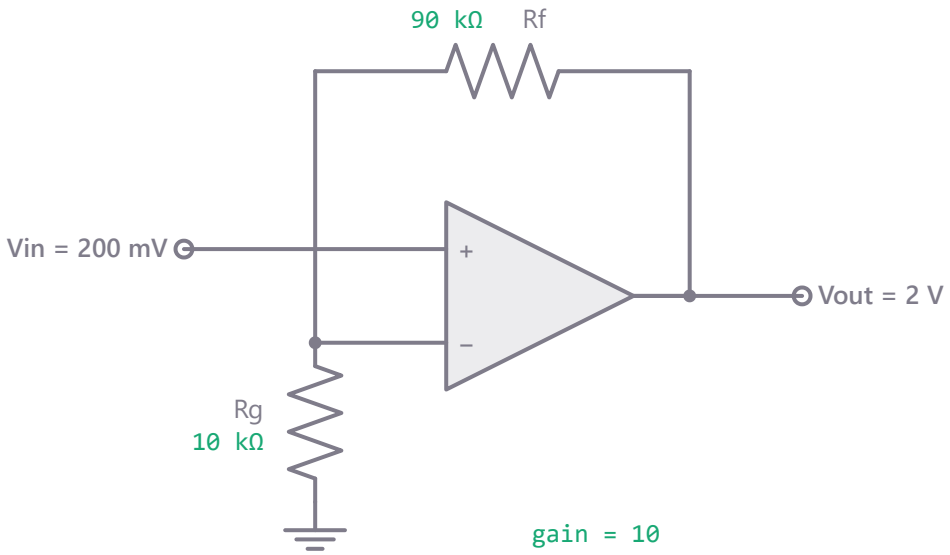
The voltage gain $1 + Rf/Rg$ of an ideal non-inverting amplifier and its output.

$$A_v = 1 + \frac{R_f}{R_g}$$

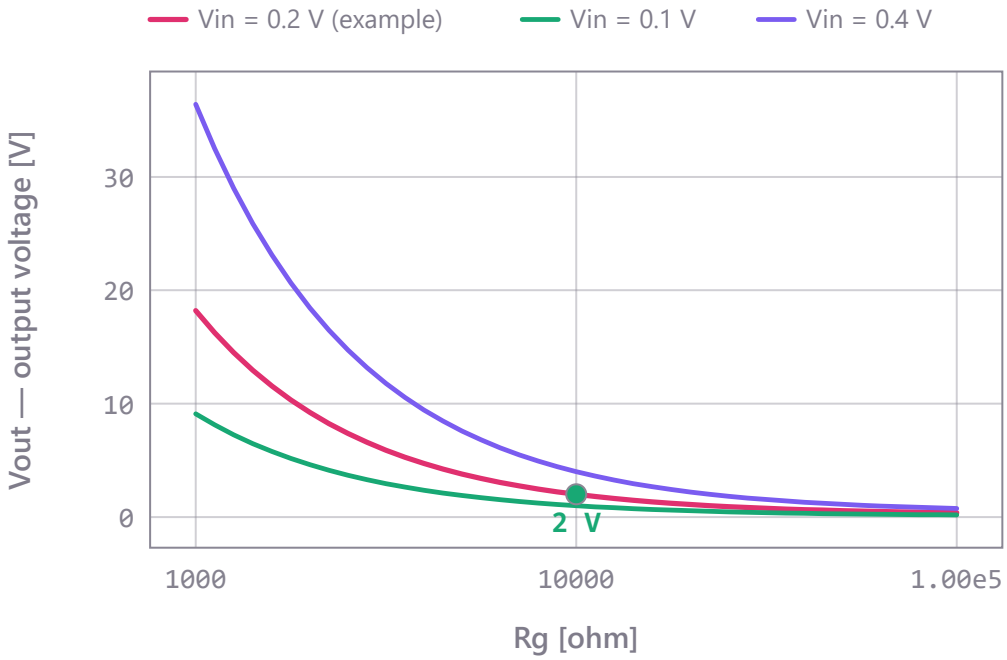
input	meaning	condition
`Rg`	resistor from the inverting input to ground	
`Rf`	feedback resistor	
`Vin`	input voltage (at the non-inverting input)	

output	meaning
`gain`	closed-loop voltage gain
`Vout`	output voltage

- **Method:** gain = 1 + Rf/Rg
- **Assumes:** ideal op-amp, no saturation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Non-inverting amplifier as stated: Rg = 10 kΩ to ground, feedback Rf = 90 kΩ, input Vin = 200 mV on the + input. Labelled with what the package returned: gain = 10, Vout = 2 V. Marked: Vout = 2 V.



Vout (output voltage) against Rg (resistor from the inverting input to ground) from 1000 ohm to 100000 ohm on a logarithmic axis. One curve for each value of Vin (input voltage (at the non-inverting input)): 0.2 V, 0.1 V, 0.4 V. Other inputs as in the worked example: Rf = 90000.0 ohm. 41 points per curve computed by electrical-engineering-ai999x; marked: the worked example, Rg = 10000 ohm, Vout = 2 V.

```
electrical_engineering_ai999x.solve('noninverting_amplifier', Rg=10000.0, Rf=90000.0, Vin=0.2)
# -> gain = 10, Vout = 2 V   (SOLVED)
```

```
electrical_engineering_ai999x.solve('noninverting_amplifier', Rg=1000.0, Rf=1000.0, Vin=1)
# -> gain = 2, Vout = 2 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the output of a non-inverting amplifier with Rg = 10 kohm, Rf = 90 kohm, Vin = 0.2 V.	SOLVED	gain = 10, Vout = 2 V
Find the output of a non-inverting amplifier with Rg = 0 kohm, Rf = 90 kohm, Vin = 0.2 V.	INVALID_INPUT	Non-inverting amplifier gain and output: Rg must be positive.

Op-amp integrator output for a constant input — `opamp_integrator`

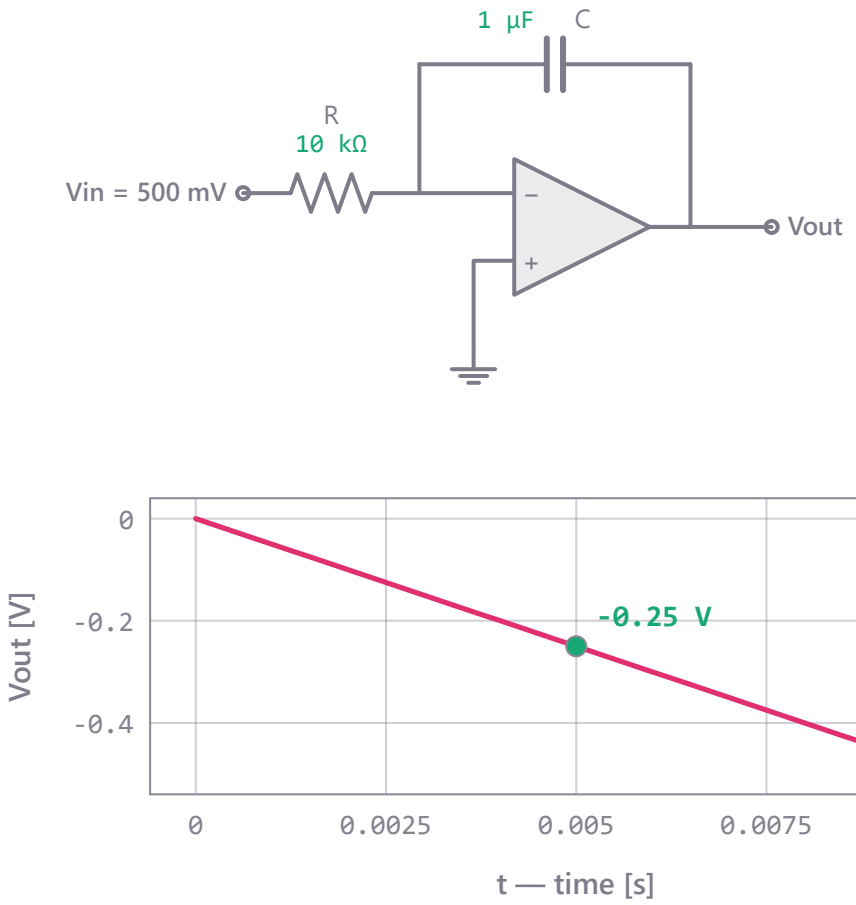
The output of an ideal inverting integrator (R in, C feedback) after a constant input Vin has been applied for time t, from an initial output V0.

$$V_{out}(t) = V_0 - \frac{1}{RC}\int_0^t V_{in}\,dt$$

input	meaning	condition
`R`	input resistor	
`C`	feedback capacitor	
`Vin`	constant input voltage	
`t`	integration time	
`V0`	output voltage at $t = 0$	

output	meaning
`Vout`	output voltage at t
`slope`	output slope dVout/dt in V/s

- **Method:** Vout = V0 - Vin t/(RC)
- **Assumes:** ideal op-amp, no saturation; constant input
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Op-amp integrator as stated (R = 10 kΩ, C = 1 µF, constant input Vin = 500 mV, starting at V0 = 0 V) and, below, Vout computed by the package at 41 instants from 0 to 0.01 s (slope -50 V/s). Marked: t = 0.005 s, Vout = -0.25 V.

```
electrical_engineering_ai999x.solve('opamp_integrator', R=10000.0, C=1e-06, Vin=0.5, t=0.005, V0=0)
# -> Vout = -0.25 V, slope = -50   (SOLVED)
```

```
electrical_engineering_ai999x.solve('opamp_integrator', R=100000.0, C=1e-07, Vin=-1, t=0.02, V0=1)
# -> Vout = 3 V, slope = 100   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the output of an op-amp integrator with R = 10 kohm, C = 1 uF, Vin = 0.5 V, t = 5 ms, V0 = 0 V.	SOLVED	Vout = -0.25 V, slope = -50
Find the output of an op-amp integrator with R = 10 kohm, C = 1 uF, Vin = 0.5 V, t = 5 ms.	NEEDS_INPUT	opamp_integrator needs V0 (output voltage at t = 0). Example: solve('opamp_integrator', R='10000 ohm', C='1e-06 F', Vin=

Ideal transformer: voltages, currents and reflected resistance — `transformer`

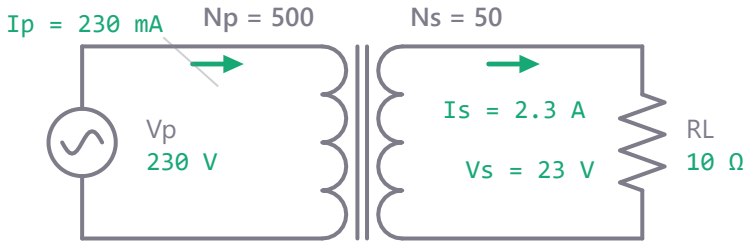
Secondary voltage and current, primary current and the load resistance seen from the primary for an ideal transformer with a resistive load.

$$\frac{V_s}{V_p} = \frac{N_s}{N_p}, \quad R_{in} = R_L\left(\frac{N_p}{N_s}\right)^2$$

input	meaning	condition
`Np`	primary turns	
`Ns`	secondary turns	
`Vp`	primary (rms) voltage	
`RL`	load resistance on the secondary	

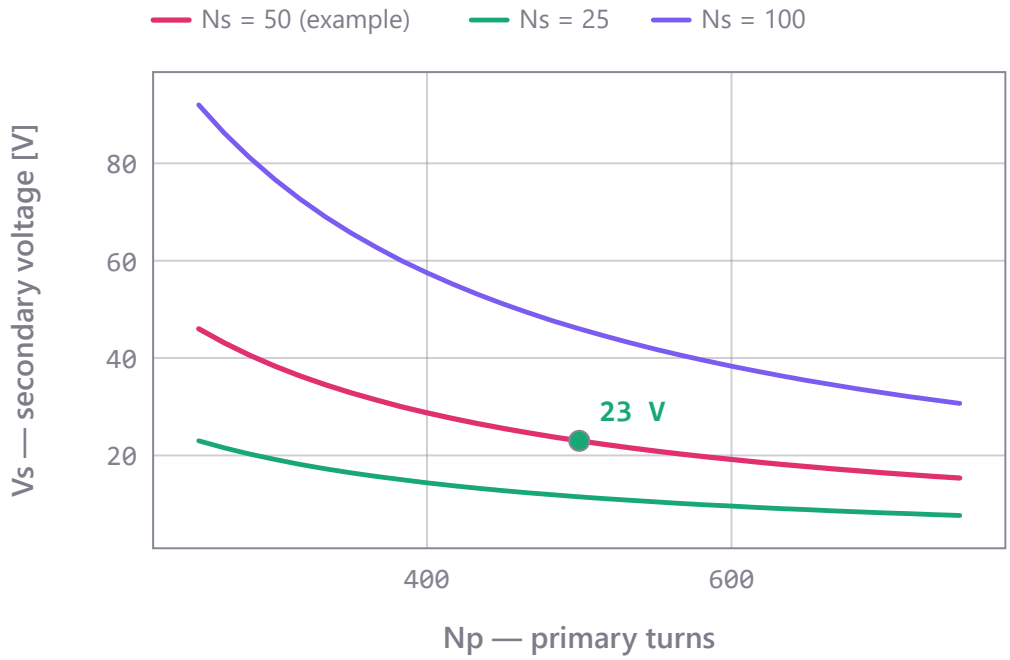
output	meaning
`Vs`	secondary voltage
`Is`	secondary (load) current
`Ip`	primary current
`Rin`	load resistance reflected to the primary
`turns_ratio`	Np/Ns

- **Method:** $Vs = Vp$ Ns/Np, $Ip = Is$ Ns/Np, Rin = RL $(Np/Ns)^2$
- **Assumes:** ideal transformer (no losses, no magnetizing current)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



turns ratio 10, input resistance 1 kΩ

Ideal transformer as stated: Np = 500, Ns = 50 turns, primary Vp = 230 V, load RL = 10 Ω. Labelled with what the package returned: Vs = 23 V, Is = 2.3 A, Ip = 0.23 A, turns ratio 10, resistance seen from the primary 1000 Ω. Marked: Vs = 23 V.



Vs (secondary voltage) against Np (primary turns) from 250 to 750. One curve for each value of Ns (secondary turns): 50, 25, 100. Other inputs as in the worked example: Vp = 230 V, RL = 10 ohm. 31 points per curve computed by electrical-engineering-ai999x; marked: the worked example, Np = 500, Vs = 23 V.

```
electrical_engineering_ai999x.solve('transformer', Np=500, Ns=50, Vp=230, RL=10)
# -> Vs = 23 V, Is = 2.3 A, Ip = 0.23 A, Rin = 1000 ohm, turns_ratio = 10   (SOLVED)
```

```
electrical_engineering_ai999x.solve('transformer', Np=100, Ns=200, Vp=12, RL=48)
# -> Vs = 24 V, Is = 0.5 A, Ip = 1 A, Rin = 12 ohm, turns_ratio = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the secondary voltage of a transformer with Np = 500, Ns = 50, Vp = 230 V, RL = 10 ohm.	SOLVED	Vs = 23 V, Is = 2.3 A, Ip = 0.23 A, Rin = 1000 ohm, turns_ratio = 10
Find the secondary voltage of a transformer with Np = 500, Ns = 0, Vp = 230 V, RL = 10 ohm.	INVALID_INPUT	Ideal transformer: voltages, currents and reflected resistance: Ns must be positive.

MOSFET drain current in saturation (square law) — `mosfet_saturation`

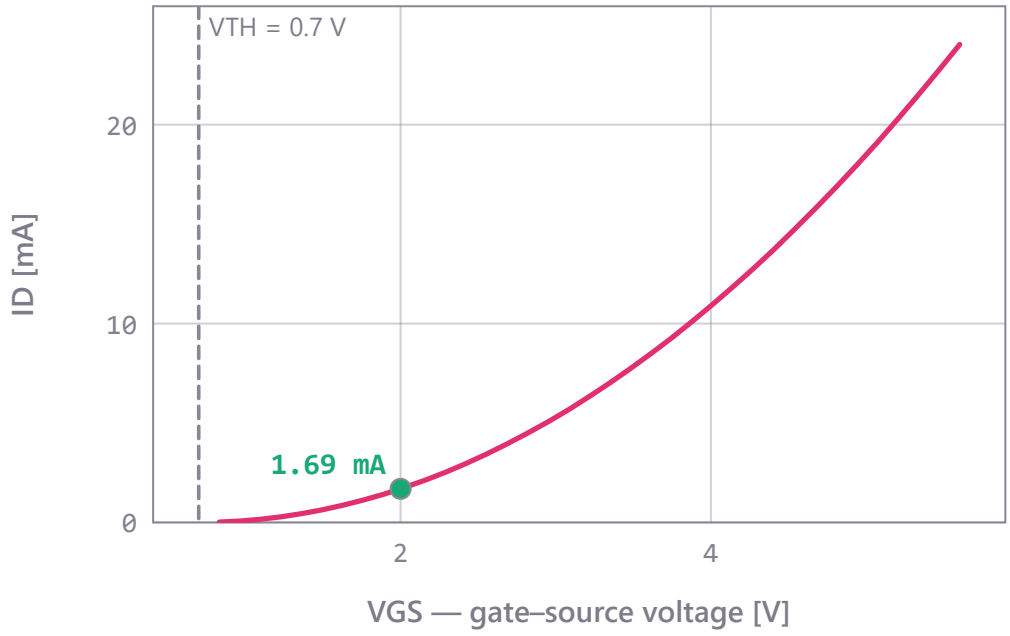
Drain current, overdrive voltage and transconductance of an n-channel MOSFET in saturation.

$$I_D = \tfrac{1}{2} k'_n \frac{W}{L} (V_{GS} - V_{TH})^2$$

input	meaning	condition
`kn`	process transconductance parameter mu_n Cox in A/V^2	
`WL`	aspect ratio W/L	
`VGS`	gate-source voltage	
`VTH`	threshold voltage	
`VDS`	drain-source voltage	

output	meaning
`ID`	drain current
`ID_mA`	drain current in milliamperes
`VOV`	overdrive VGS - VTH
`gm`	transconductance in A/V (siemens)

- **Method:** $ID = (1/2)$ kn (W/L) (VGS - VTH)^2
- **Assumes:** saturation: VGS > VTH and VDS >= VGS - VTH; no channel-length modulation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Drain current in saturation against gate–source voltage, from VTH = 0.7 V to 6 V (kn = 0.0002 A/V², W/L = 10, VDS = 5 V as stated), 40 runs of the package. Marked: the worked example VGS = 2 V, ID = 1.69 mA (gm = 0.0026 S, overdrive 1.3 V).

```
electrical_engineering_ai999x.solve('mosfet_saturation', kn=0.0002, WL=10, VGS=2, VTH=0.7, VDS=5)
# -> ID = 0.00169 A, ID_mA = 1.69, VOV = 1.3 V, gm = 0.0026   (SOLVED)
```

```
electrical_engineering_ai999x.solve('mosfet_saturation', kn=0.0001, WL=20, VGS=1.5, VTH=0.5, VDS=1)
# -> ID = 0.001 A, ID_mA = 1, VOV = 1 V, gm = 0.002   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the drain current of a MOSFET with kn = 200 uA/V^2, WL = 10, VGS = 2 V, VTH = 0.7 V, VDS = 5 V.	SOLVED	ID = 0.00169 A, ID_mA = 1.69, VOV = 1.3 V, gm = 0.0026
Find the drain current of a MOSFET with kn = 200 uA/V^2, WL = 10, VGS = 2 V, VTH = 0.7 V, VDS = 0.5 V.	INVALID_INPUT	MOSFET drain current in saturation (square law): VDS < VGS - VTH: the transistor is in the triode region, not saturation
Find the drain current of a MOSFET with kn = 200 uA/V^2, WL = 10, VGS = 0.5 V, VTH = 0.7 V, VDS = 5 V.	INVALID_INPUT	MOSFET drain current in saturation (square law): VGS does not exceed the threshold voltage: the transistor is off.

Operating point of a diode in series with a resistor (Shockley equation) — `diode_circuit`

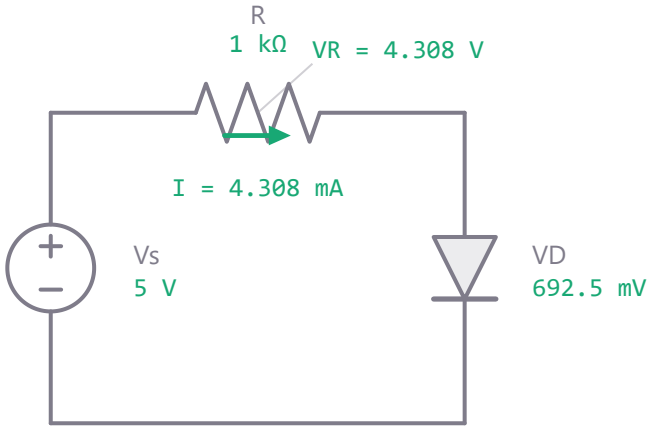
The current and the diode voltage of a forward-biased diode $I = Is$ (exp(VD/(n VT)) - 1) in series with a resistor across a DC source.

$$V_s = IR + nV_T \ln\left(1 + \frac{I}{I_s}\right)$$

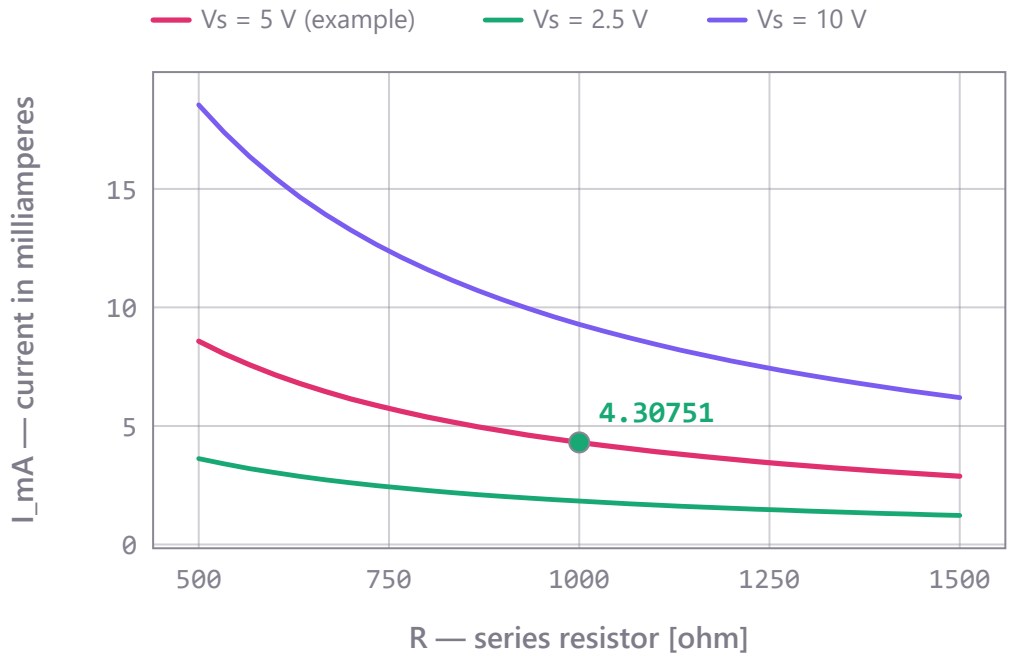
input	meaning	condition
`Vs`	source voltage (forward)	
`R`	series resistor	
`Is`	saturation current	
`n`	ideality factor	
`VT`	thermal voltage kT/q (about 25.85 mV at 300 K)	

output	meaning
`I`	circuit current
`I_mA`	current in milliamperes
`VD`	diode voltage
`VR`	resistor voltage

- **Method:** bisection then Newton on $Vs = I$ R + n VT ln(1 + I/Is)
- **Assumes:** forward bias $(Vs > 0)$; the Shockley model without series resistance or breakdown
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A diode in series with R = 1 kΩ across Vs = 5 V (saturation current Is = 1.0000e-14 A, ideality n = 1, VT = 0.02585 V, as stated). Labelled with the operating point the package returned: I = 0.00430751 A, diode voltage VD = 0.69249 V, VR = 4.30751 V. Marked: I = 4.30751 mA.



I_mA (current in milliamperes) against R (series resistor) from 500 ohm to 1500 ohm. One curve for each value of Vs (source voltage (forward)): 5 V, 2.5 V, 10 V. Other inputs as in the worked example: Is = 1e-14 A, n = 1, VT = 0.02585 V. 31 points per curve computed by electrical-engineering-ai999x; marked: the worked example, R = 1000 ohm, I_mA = 4.30751.

```
electrical_engineering_ai999x.solve('diode_circuit', Vs=5, R=1000, Is=1e-14, n=1, VT=0.02585)
# -> I = 0.00430751 A, I_mA = 4.30751, VD = 0.69249 V, VR = 4.30751 V   (SOLVED)
```

```
electrical_engineering_ai999x.solve('diode_circuit', Vs=10, R=10000, Is=1e-12, n=2, VT=0.02585)
# -> I = 0.000893443 A, I_mA = 0.893443, VD = 1.06557 V, VR = 8.93443 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the current of a diode circuit with Vs = 5 V, R = 1 kohm, Is = 1e-14 A, n = 1, VT = 25.85 mV.	SOLVED	I = 0.00430751 A, I_mA = 4.30751, VD = 0.69249 V, VR = 4.30751 V
Find the current of a diode circuit with Vs = 5 V, R = 1 kohm, n = 1, VT = 25.85 mV.	NEEDS_INPUT	diode_circuit needs Is (saturation current). Example: solve('diode_circuit', Vs='5 V', R='1000 ohm', Is='1e-14 A', n=1,

RMS and average value of a periodic waveform — `rms_waveform`

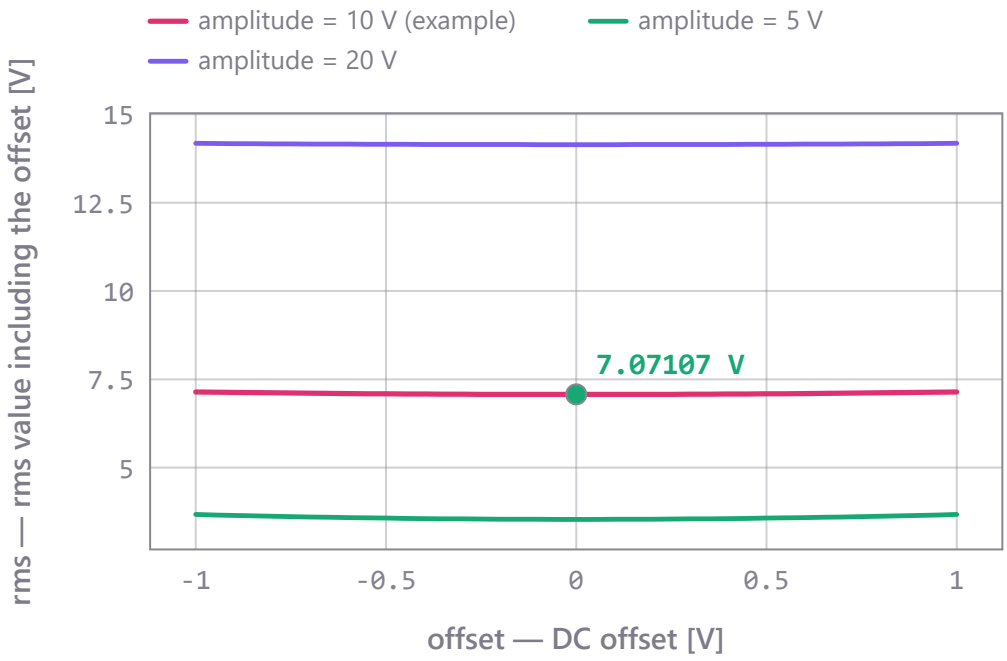
The rms value (with DC offset), average, AC rms and crest factor of a sine, square, triangle or sawtooth wave.

$$V_{rms} = \sqrt{\frac{1}{T}\int_0^T v^2\,dt}$$

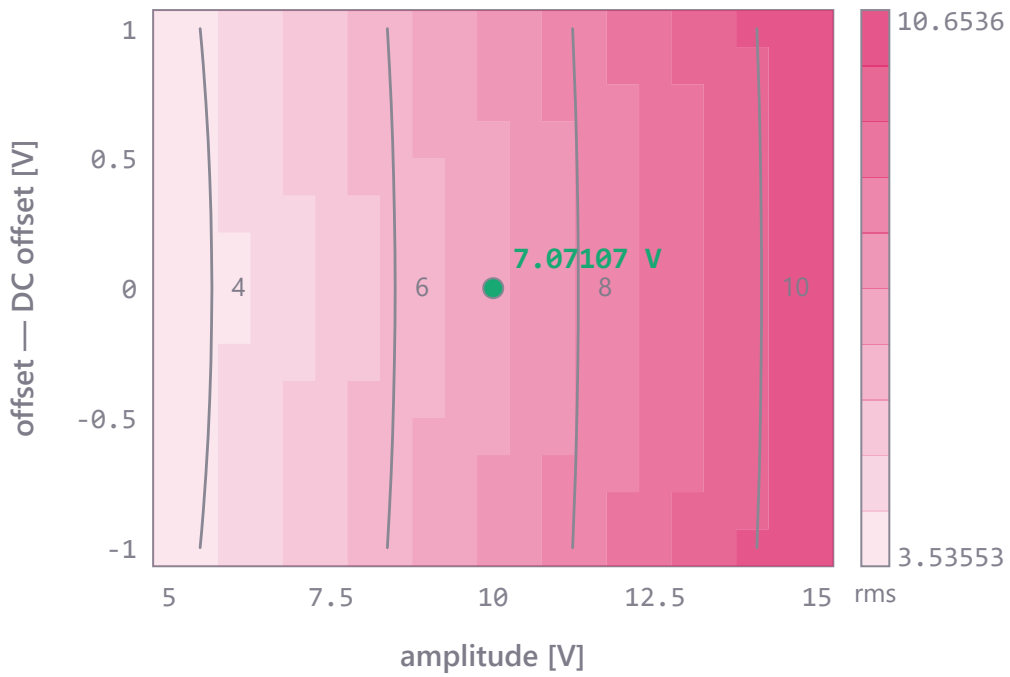
input	meaning	condition
`shape`	waveform shape	
`amplitude`	peak amplitude about the average	
`offset`	DC offset (average value); 0 when not given	

output	meaning
`rms`	rms value including the offset
`average`	average value
`ac_rms`	rms of the AC part
`crest_factor`	peak over rms

- **Method:** Vrms = sqrt(offset^2 + k A^2) with $k = 1/2$ (sine), 1 (square), $1/3$ (triangle, sawtooth)
- **Assumes:** symmetric waveform of the stated shape about its offset
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



rms (rms value including the offset) against offset (DC offset (average value); 0 when not given) from -1 V to 1 V. One curve for each value of amplitude (peak amplitude about the average): 10 V, 5 V, 20 V. Other inputs as in the worked example: shape = sine. 41 points per curve computed by electrical-engineering-ai999x; marked: the worked example, offset = 0 V, rms = 7.07107 V.



rms (rms value including the offset) over amplitude (peak amplitude about the average) from 5 V to 15 V and offset (DC offset (average value); 0 when not given) from -1 V to 1 V: each of the 21 × 15 cells is one run of electrical-engineering-ai999x, shaded by its value (light 3.53553 V to dark 10.6536 V); iso-lines at rms = 4, 6, 8, 10 V. Other inputs as in the worked example: shape = sine. Marked: the worked example, amplitude = 10 V, offset = 0 V, rms = 7.07107 V.

```
electrical_engineering_ai999x.solve('rms_waveform', shape='sine', amplitude=10, offset=0)
# -> rms = 7.07107 V, average = 0 V, ac_rms = 7.07107 V, crest_factor = 1.41421   (SOLVED)
```

```
electrical_engineering_ai999x.solve('rms_waveform', shape='square', amplitude=5, offset=0)
# -> rms = 5 V, average = 0 V, ac_rms = 5 V, crest_factor = 1   (SOLVED)
```

```
electrical_engineering_ai999x.solve('rms_waveform', shape='triangle', amplitude=3, offset=4)
# -> rms = 4.3589 V, average = 4 V, ac_rms = 1.73205 V, crest_factor = 1.60591   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the rms value of a square wave with amplitude = 5 V, offset = 0 V.	SOLVED	rms = 5 V, average = 0 V, ac_rms = 5 V, crest_factor = 1
Find the rms value of a triangle wave with amplitude = 3 V, offset = 4 V.	SOLVED	rms = 4.3589 V, average = 4 V, ac_rms = 1.73205 V, crest_factor = 1.60591
Find the rms value of a sine wave with amplitude = 10 V.	SOLVED	rms = 7.07107 V, average = 0 V, ac_rms = 7.07107 V, crest_factor = 1.41421

Analog-to-digital conversion: output code and resolution — `adc_conversion`

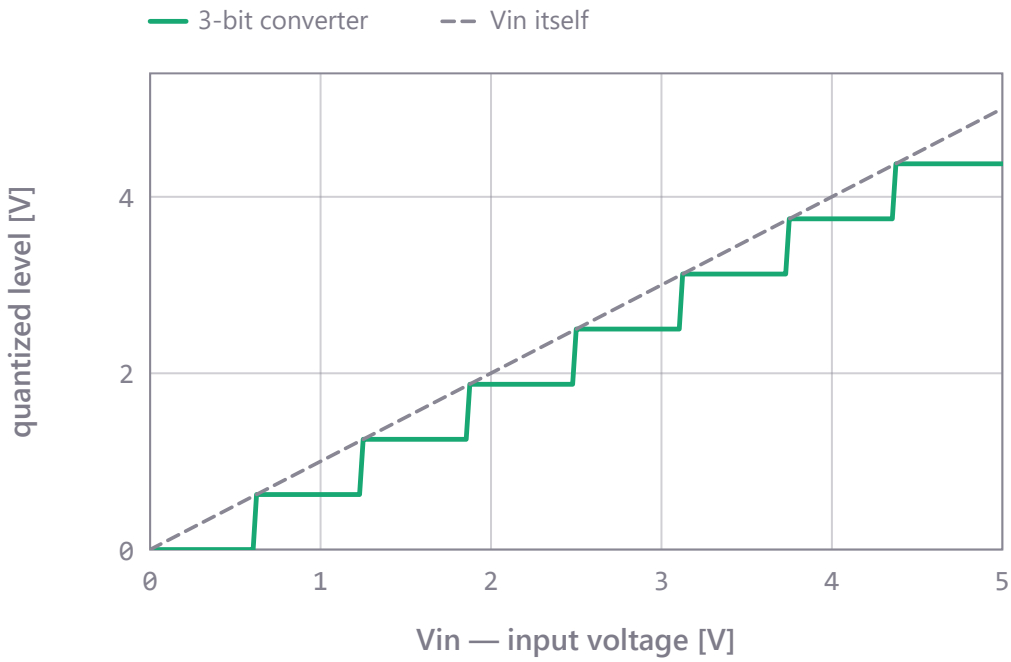
The output code (truncating converter), its binary form, the LSB size and the quantized voltage of an ideal n-bit ADC with reference Vref.

$$\text{code} = \left\lfloor \frac{V_{in}}{V_{ref}/2^{n}} \right\rfloor$$

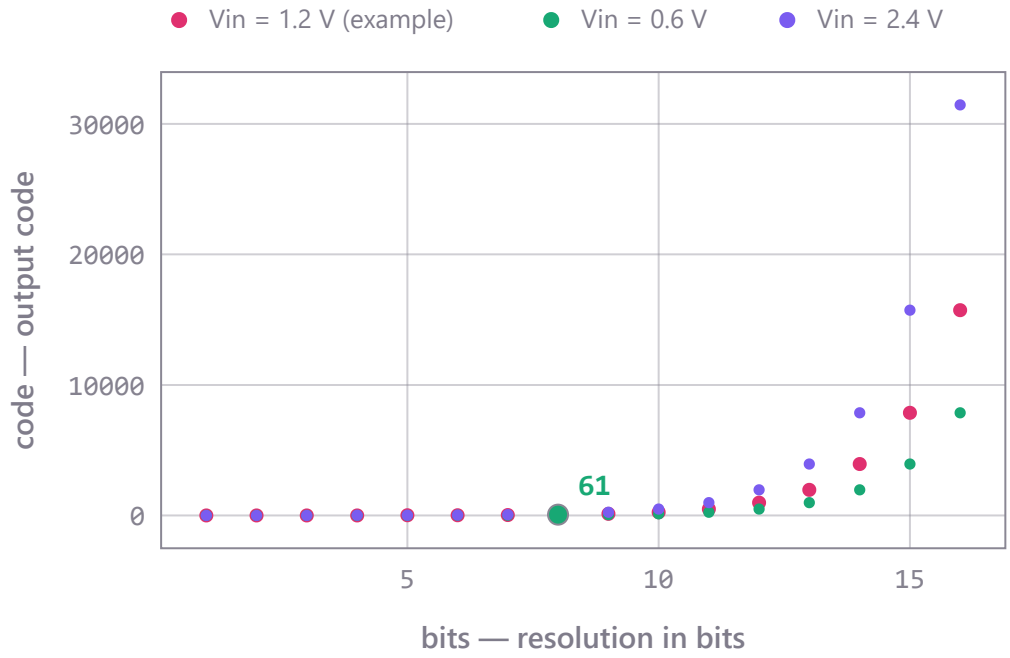
input	meaning	condition
`Vin`	input voltage (0 to Vref)	
`Vref`	full-scale reference voltage	
`bits`	resolution in bits	

output	meaning
`code`	output code
`binary`	code in binary
`lsb`	LSB size Vref/2^n
`lsb_mV`	LSB in millivolts
`quantized`	voltage of the code, code x LSB

- **Method:** code = floor(Vin / LSB), capped at $2^n - 1$
- **Assumes:** 0 <= Vin <= Vref; truncating (floor) quantizer; Vref itself gives the top code
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Transfer staircase of an analogue-to-digital converter: the quantized level the package returned for 241 input voltages from 0 to Vref = 5 V, with a 3-bit converter (chosen so the steps are visible; the worked example uses 8 bits: Vin = 1.2 V gives code 61 = 00111101, LSB 0.0195312 V, level 1.19141 V). The dashed line is Vin itself.



code (output code) against bits (resolution in bits) from 1 to 16. One curve for each value of Vin (input voltage (0 to Vref)): 1.2 V, 0.6 V, 2.4 V. Other inputs as in the worked example: Vref = 5 V. 16 points per curve computed by electrical-engineering-ai999x; marked: the worked example, bits = 8, code = 61.

```
electrical_engineering_ai999x.solve('adc_conversion', Vin=1.2, Vref=5, bits=8)
# -> code = 61, binary = 00111101, lsb = 0.0195312 V, lsb_mV = 19.5312, quantized = 1.19141 V
(SOLVED)
```

```
electrical_engineering_ai999x.solve('adc_conversion', Vin=5, Vref=5, bits=4)
# -> code = 15, binary = 1111, lsb = 0.3125 V, lsb_mV = 312.5, quantized = 4.6875 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the ADC output code for Vin = 1.2 V, Vref = 5 V, bits = 8.	SOLVED	code = 61, binary = 00111101, lsb = 0.0195312 V, lsb_mV = 19.5312, quantized = 1.19141 V
Find the ADC output code for Vin = 6 V, Vref = 5 V, bits = 8.	INVALID_INPUT	Analog-to-digital conversion: output code and resolution: the input must be between 0 and Vref.

Transfer function of a single-input single-output state-space model — `state_space_to_tf`

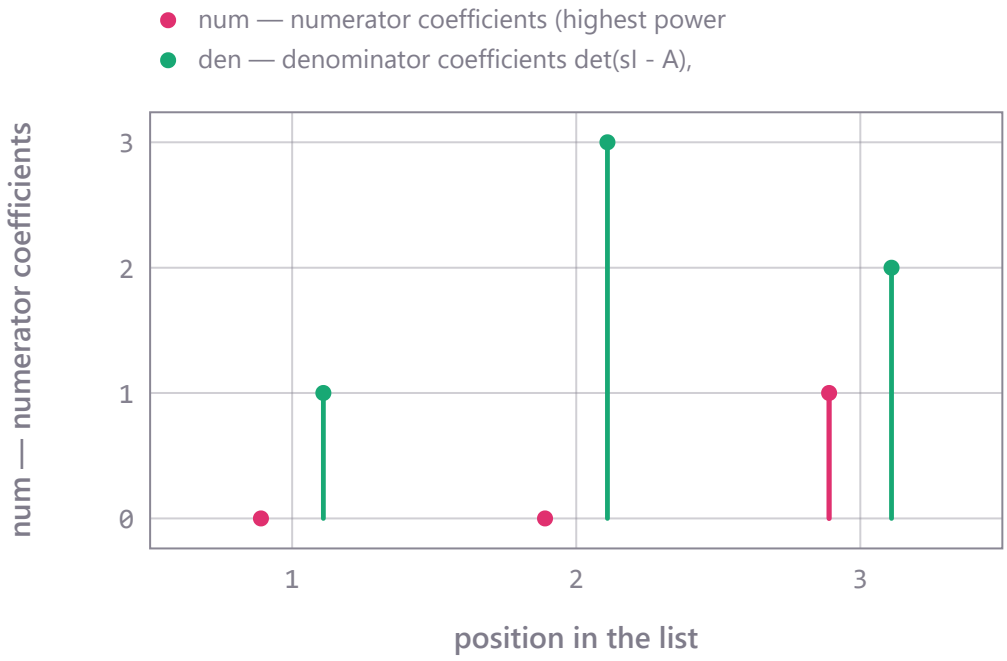
G(s) = C $(sI - A)^{-1}$ B + D as numerator and denominator coefficients for x' = Ax + Bu, $y = Cx + Du$.

$$G(s) = C(sI - A)^{-1}B + D$$

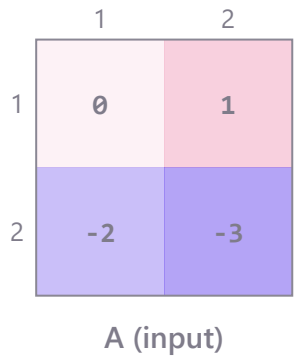
input	meaning	condition
`A`	state matrix (n x n)	
`B`	input vector (n entries)	
`C`	output vector (n entries)	
`D`	feedthrough	

output	meaning
`num`	numerator coefficients (highest power first, length n + 1)
`den`	denominator coefficients det(sI - A), monic
`order`	system order n

- **Method:** Faddeev-LeVerrier recursion for det(sI - A) and adj(sI - A)
- **Assumes:** a square A with B and C of matching size; coefficients below $1e-12$ of the largest are written as 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



numerator coefficients (highest power first, length n + 1) (num) returned by electrical-engineering-ai999x for the worked example, one value for each position in the list (3 values, first 0, last 1, largest 1); with den (denominator coefficients det(sI - A), monic). Inputs: A = [[0, 1], [-2, -3]], B = [0, 1], C = [1, 0], D = 0.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input). Returned with them by electrical-engineering-ai999x: order = 2. Other inputs: B = [0, 1], C = [1, 0], D = 0.

```
electrical_engineering_ai999x.solve('state_space_to_tf', A='[[0, 1], [-2, -3]]', B='[0, 1]', C='[1, 0]', D=0)
# -> num = [0.0, 0.0, 1.0], den = [1.0, 3.0, 2.0], order = 2   (SOLVED)
```

```
electrical_engineering_ai999x.solve('state_space_to_tf', A=[[-1, 0], [0, -2]], B=[1, 1], C=[1, 1],
D=1)
# -> num = [1.0, 5.0, 5.0], den = [1.0, 3.0, 2.0], order = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the transfer function of the state-space model A = [[0, 1], [-2, -3]], B = [0, 1], C = [1, 0], D = 0.	SOLVED	num = [0.0, 0.0, 1.0], den = [1.0, 3.0, 2.0], order = 2
Find the transfer function of the state-space model A = [[0, 1], [-2, -3]], B = [0, 1, 2], C = [1, 0], D = 0.	INVALID_INPUT	Transfer function of a single-input single-output state-space model: A must be n x n and B, C must have n entries.

Controllability and observability of a single-input single-output system — `controllability_observability`

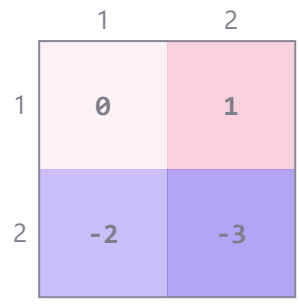
The ranks of the controllability matrix [B AB ... A^(n-1)B] and the observability matrix [C; CA; ...; CA^{n-1}], and whether the system is controllable and observable.

$$\mathcal{C} = [B\ AB\ \cdots\ A^{n-1}B],\ \mathcal{O} = [C;\ CA;\ \cdots;\ CA^{n-1}]$$

input	meaning	condition
`A`	state matrix (n x n)	
`B`	input vector	
`C`	output vector	

output	meaning
`controllability_rank`	rank of the controllability matrix
`observability_rank`	rank of the observability matrix
`controllable`	full controllability rank
`observable`	full observability rank

- **Method:** Kalman rank tests by Gaussian elimination with partial pivoting
- **Assumes:** small integer or well-scaled entries (rank decided at a relative tolerance of 1e-9)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input). Returned with them by electrical-engineering-ai999x: controllability_rank = 2, observability_rank = 2. Other inputs: B = [0, 1], C = [1, 0].

```
electrical_engineering_ai999x.solve('controllability_observability', A='[[0, 1], [-2, -3]]', B='[0, 1]', C='[1, 0]')
# -> controllability_rank = 2, observability_rank = 2, controllable = True, observable = True (SOLVED)
```

```
electrical_engineering_ai999x.solve('controllability_observability', A=[[-1, 0], [0, -2]], B=[1, 0], C=[1, 1])
# -> controllability_rank = 1, observability_rank = 2, controllable = False, observable = True (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Is the system with A = [[0, 1], [-2, -3]], B = [0, 1], C = [1, 0] controllable and observable?	SOLVED	controllability_rank = 2, observability_rank = 2, controllable = True, observable = True
Is the system with A = [[-1, 0], [0, -2]], B = [1, 0], C = [1, 1] controllable?	SOLVED	controllability_rank = 1, observability_rank = 2, controllable = False, observable = True

Energy stored in an inductor — `inductor_energy`

The magnetic energy L $I^2/2$ and the flux linkage L I of an inductor carrying current I.

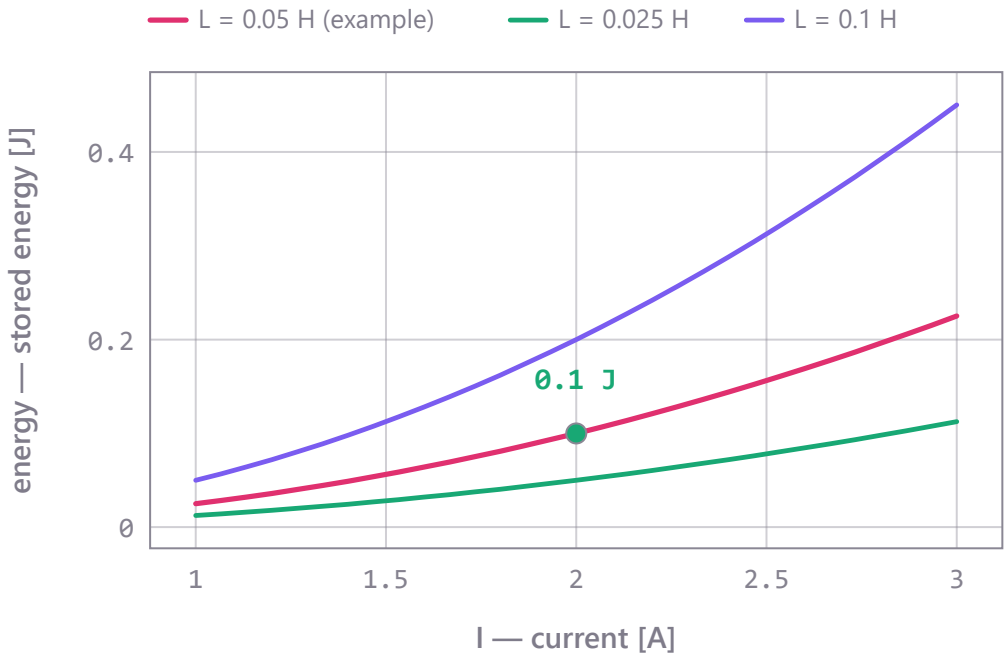
$$W = \tfrac{1}{2} L I^2$$

input	meaning	condition
`L`	inductance	
`I`	current	

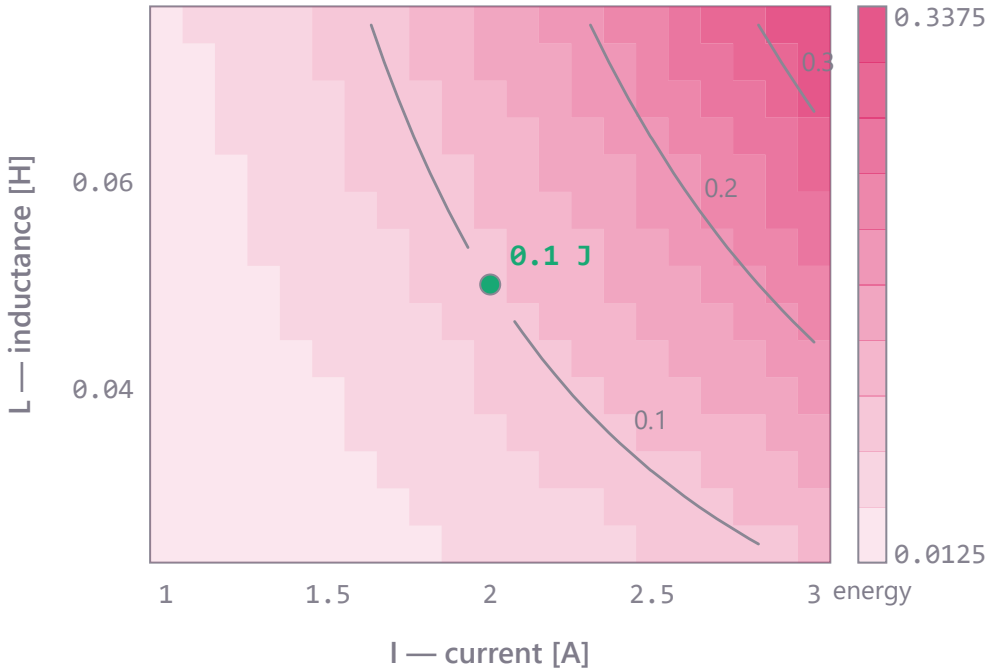
output	meaning
`energy`	stored energy
`flux_linkage`	flux linkage L I in weber-turns

- **Method:** $W = L\,I^2/2$
- **Assumes:** a linear inductor (no saturation)

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



energy (stored energy) against I (current) from 1 A to 3 A. One curve for each value of L (inductance): 0.05 H, 0.025 H, 0.1 H. 31 points per curve computed by electrical-engineering-ai999x; marked: the worked example, I = 2 A, energy = 0.1 J.



energy (stored energy) over I (current) from 1 A to 3 A and L (inductance) from 0.025 H to 0.075 H: each of the 21 × 15 cells is one run of electrical-engineering-ai999x, shaded by its value (light 0.0125 J to dark 0.3375 J); iso-lines at energy = 0.1, 0.2, 0.3 J. Marked: the worked example, I = 2 A, L = 0.05 H, energy = 0.1 J.

```
electrical_engineering_ai999x.solve('inductor_energy', L=0.05, I=2)
# -> energy = 0.1 J, flux_linkage = 0.1   (SOLVED)
```

```
electrical_engineering_ai999x.solve('inductor_energy', L=0.001, I=10)
# -> energy = 0.05 J, flux_linkage = 0.01   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the energy stored in an inductor with L = 50 mH, I = 2 A.	SOLVED	energy = 0.1 J, flux_linkage = 0.1
Find the energy stored in an inductor with L = 50 mH.	NEEDS_INPUT	inductor_energy needs I (current). Example: solve('inductor_energy', L='0.05 H', I='2 A').

Time for a first-order response to reach a value — `first_order_time_to_value`

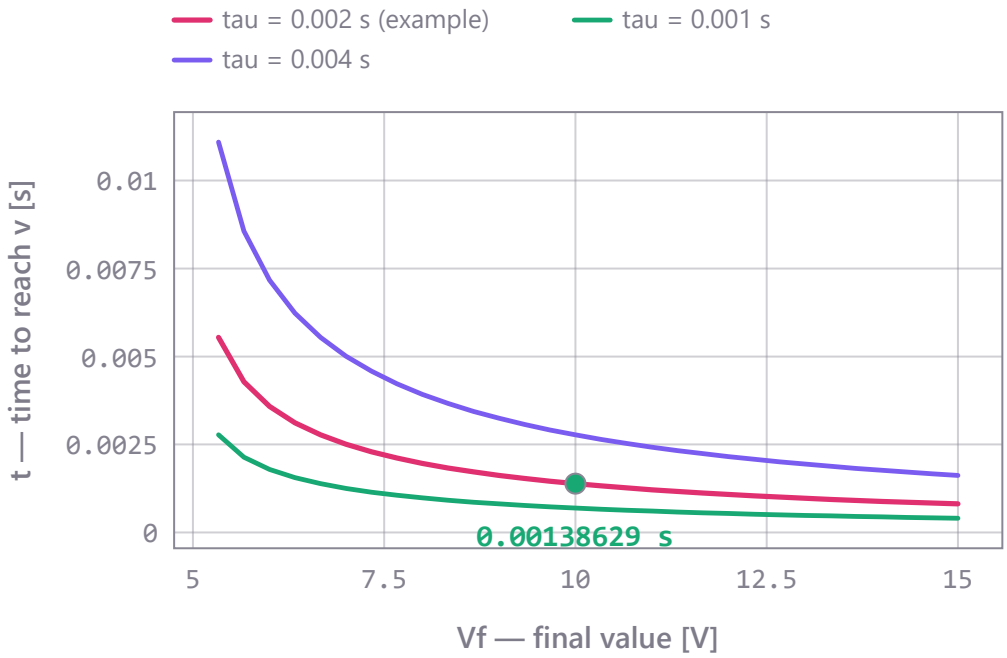
How long a first-order (RC or RL) response from V0 toward Vf with time constant tau takes to reach a value v.

$$t = \tau \ln \frac{V_f - V_0}{V_f - v}$$

input	meaning	condition
`V0`	initial value	
`Vf`	final value	
`tau`	time constant	
`v`	value to reach	

output	meaning
`t`	time to reach v
`time_constants`	the same time in time constants

- **Method:** $t = -\tau \ln(1 - (v - V_0)/(Vf - V_0))$
- **Assumes:** v between V0 and Vf
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



t (time to reach v) against Vf (final value) from 5 V to 15 V. One curve for each value of tau (time constant): 0.002 s, 0.001 s, 0.004 s. Other inputs as in the worked example: V0 = 0 V, v = 5 V. 30 points per curve computed by electrical-engineering-ai999x; marked: the worked example, Vf = 10 V, t = 0.00138629 s.

```
electrical_engineering_ai999x.solve('first_order_time_to_value', V0=0, Vf=10, tau=0.002, v=5)
# -> t = 0.00138629 s, time_constants = 0.693147   (SOLVED)
```

```
electrical_engineering_ai999x.solve('first_order_time_to_value', V0=12, Vf=0, tau=1, v=3)
# -> t = 1.38629 s, time_constants = 1.38629   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How long does a first-order response with V0 = 0 V, Vf = 10 V, tau = 2 ms take to reach v = 5 V?	SOLVED	t = 0.00138629 s, time_constants = 0.693147
How long does a first-order response with V0 = 0 V, Vf = 10 V, tau = 2 ms take to reach v = 12 V?	INVALID_INPUT	Time for a first-order response to reach a value: the value is never reached: it must lie between V0 and Vf (Vf itself o

Maximum phase lead of a lead compensator — `lead_compensator`

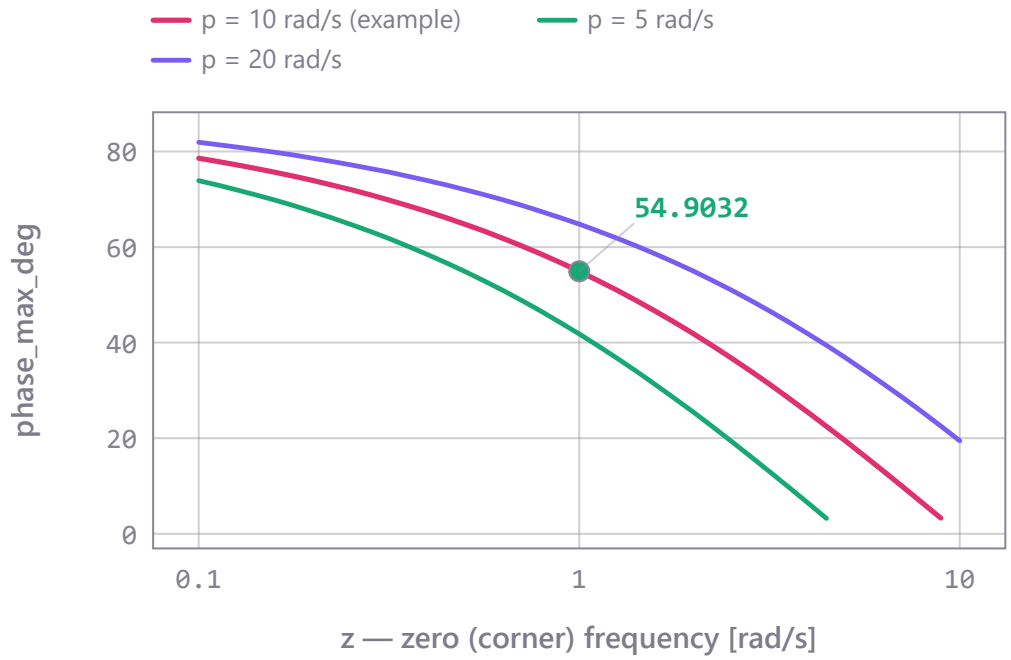
For C(s) = (s + z)/(s + p) with $p > z$: the maximum phase lead, the frequency where it occurs, and the gain there.

$$\phi_{max} = \sin^{-1}\frac{p-z}{p+z},\ \omega_m = \sqrt{zp}$$

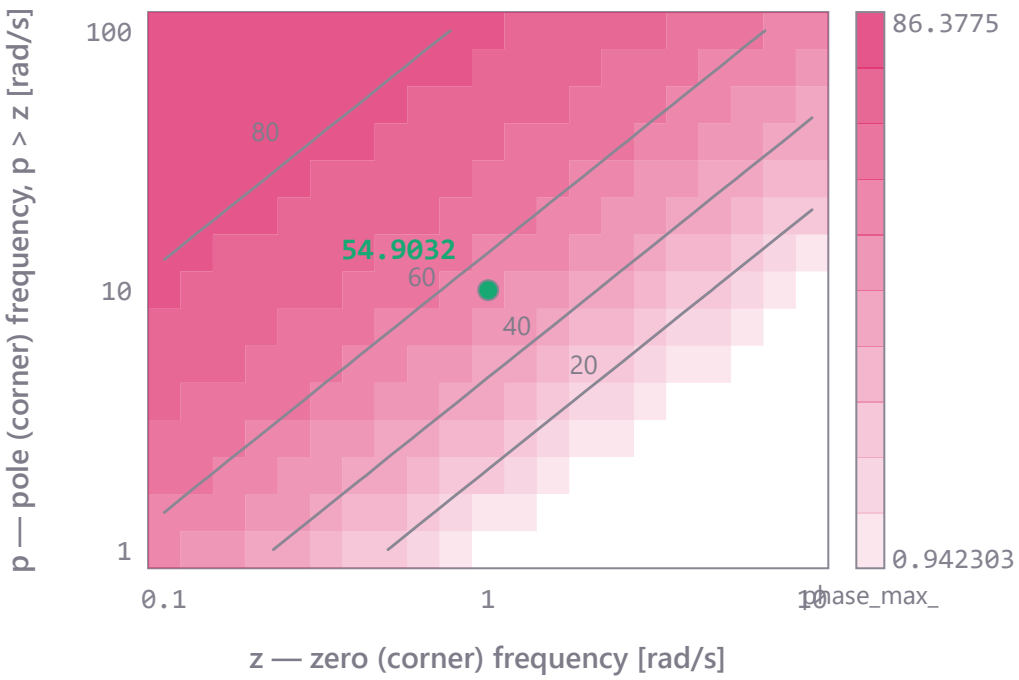
input	meaning	condition
`z`	zero (corner) frequency	
`p`	pole (corner) frequency, $p > z$	

output	meaning
`phase_max_deg`	maximum phase lead in degrees
`w_max`	frequency of maximum phase, sqrt(z p)
`gain_at_w_max_db`	gain of (s+z)/(s+p) there, in dB

- **Method:** phi_max = asin((p - z)/(p + z)) at $w = sqrt$(z p)
- **Assumes:** $p > z > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



phase_max_deg (maximum phase lead in degrees) against z (zero (corner) frequency) from 0.1 rad/s to 10 rad/s on a logarithmic axis. One curve for each value of p (pole (corner) frequency, p > z): 10 rad/s, 5 rad/s, 20 rad/s. 40 points per curve computed by electrical-engineering-ai999x; marked: the worked example, z = 1 rad/s, phase_max_deg = 54.9032.



phase_max_deg (maximum phase lead in degrees) over z (zero (corner) frequency) from 0.1 rad/s to 10 rad/s and p (pole (corner) frequency, p > z) from 1 rad/s to 100 rad/s (logarithmic axes): each of the 21 × 15 cells is one run of electrical-engineering-ai999x, shaded by its value (light 0.942303 to dark 86.3775); iso-lines at phase_max_deg = 20, 40, 60, 80; blank cells: inputs the capability refuses. Marked: the worked example, z = 1 rad/s, p = 10 rad/s, phase_max_deg = 54.9032.

```
electrical_engineering_ai999x.solve('lead_compensator', z=1, p=10)
# -> phase_max_deg = 54.9032, w_max = 3.16228 rad/s, gain_at_w_max_db = -10   (SOLVED)
```

```
electrical_engineering_ai999x.solve('lead_compensator', z=2, p=18)
# -> phase_max_deg = 53.1301, w_max = 6 rad/s, gain_at_w_max_db = -9.54243   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the maximum phase lead of a lead compensator with z = 1 rad/s, p = 10 rad/s.	SOLVED	phase_max_deg = 54.9032, w_max = 3.16228 rad/s, gain_at_w_max_db = -10
Find the maximum phase lead of a lead compensator with z = 10 rad/s, p = 1 rad/s.	INVALID_INPUT	Maximum phase lead of a lead compensator: a lead compensator has its zero below its pole (p > z).

Steady-state speed and current of a permanent-magnet DC motor — `dc_motor`

Speed, current, powers and efficiency of a DC motor with armature resistance R, torque constant Kt, back-emf constant Ke, viscous friction b and load torque T at supply voltage V.

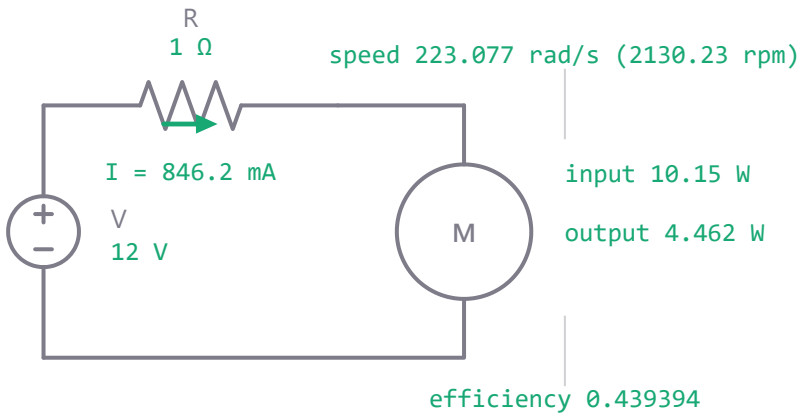
$$\omega = \frac{K_t V - RT}{K_t K_e + Rb}$$

input	meaning	condition
`V`	armature supply voltage	
`R`	armature resistance	

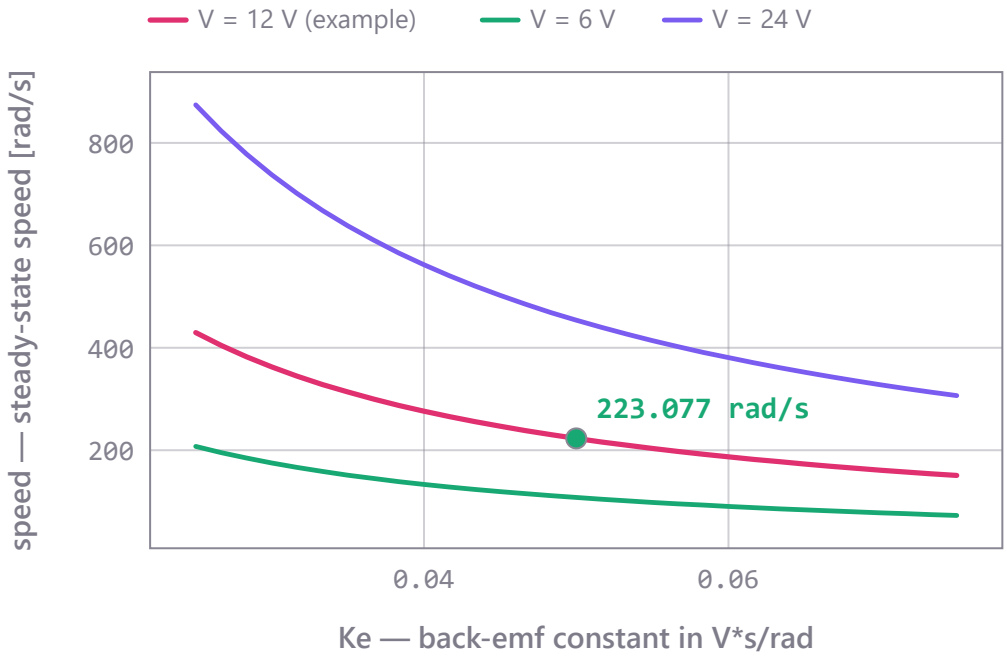
input	meaning	condition
`Kt`	torque constant in Nm/A	
`Ke`	back-emf constant in V*s/rad	
`T`	load torque	
`b`	viscous friction in N*m*s/rad	

output	meaning
`speed`	steady-state speed
`rpm`	speed in revolutions per minute
`current`	armature current
`input_power`	electrical input power
`output_power`	mechanical output power to the load
`efficiency`	output over input power

- **Method:** solve $V = R$ I + Ke w and Kt $I = b$ w + T
- **Assumes:** steady state; linear model (no saturation, constant parameters)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Permanent-magnet DC motor as stated: supply V = 12 V, armature R = 1 Ω, Kt = 0.05 N·m/A, Ke = 0.05 V·s/rad, load torque 0.02 N·m, friction b = 0.0001 N·m·s. Labelled with the steady state the package returned: current 0.846154 A, speed 223.077 rad/s, input 10.1538 W, output 4.46154 W, efficiency 0.439394. Marked: I = 0.846154 A.



speed (steady-state speed) against Ke (back-emf constant in V*s/rad) from 0.025 to 0.075. One curve for each value of V (armature supply voltage): 12 V, 6 V, 24 V. Other inputs as in the worked example: R = 1 ohm, Kt = 0.05, T = 0.02 N*m, b = 0.0001. 31 points per curve computed by electrical-engineering-ai999x; marked: the worked example, Ke = 0.05, speed = 223.077 rad/s.

```
electrical_engineering_ai999x.solve('dc_motor', V=12, R=1, Kt=0.05, Ke=0.05, T=0.02, b=0.0001)
# -> speed = 223.077 rad/s, rpm = 2130.23, current = 0.846154 A, input_power = 10.1538 W,
output_power = 4.46154 W, efficiency = 0.439394   (SOLVED)
```

```
electrical_engineering_ai999x.solve('dc_motor', V=24, R=2, Kt=0.1, Ke=0.1, T=0, b=0)
# -> speed = 240 rad/s, rpm = 2291.83, current = 0 A, input_power = 0 W, output_power = 0 W,
efficiency = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the steady-state speed of a DC motor with V = 12 V, R = 1 ohm, Kt = 0.05, Ke = 0.05, T = 0.02 N*m, b = 0.0001.	SOLVED	speed = 223.077 rad/s, rpm = 2130.23, current = 0.846154 A, input_power = 10.1538 W, output_power = 4.46154 W, efficiency = 0.439394
Find the steady-state speed of a DC motor with V = 12 V, R = 1 ohm, Kt = 0.05, Ke = 0.05, T = 0.02 N*m.	NEEDS_INPUT	dc_motor needs b (viscous friction in N*m*s/rad). Example: solve('dc_motor', V='12 V', R='1 ohm', Kt=0.05, Ke=0.05, T='0